



Modelos de Computación

Tema 3: Propiedades de los Lenguajes Regulares

Salvador Gutiérrez
salvaguti@ugr.es

Curso 2026/2027

Contenido

Lema de bombeo y aplicaciones

Operaciones con lenguajes regulares

Algoritmos para autómatas

Minimización de autómatas: estados indistinguibles.

Contenido

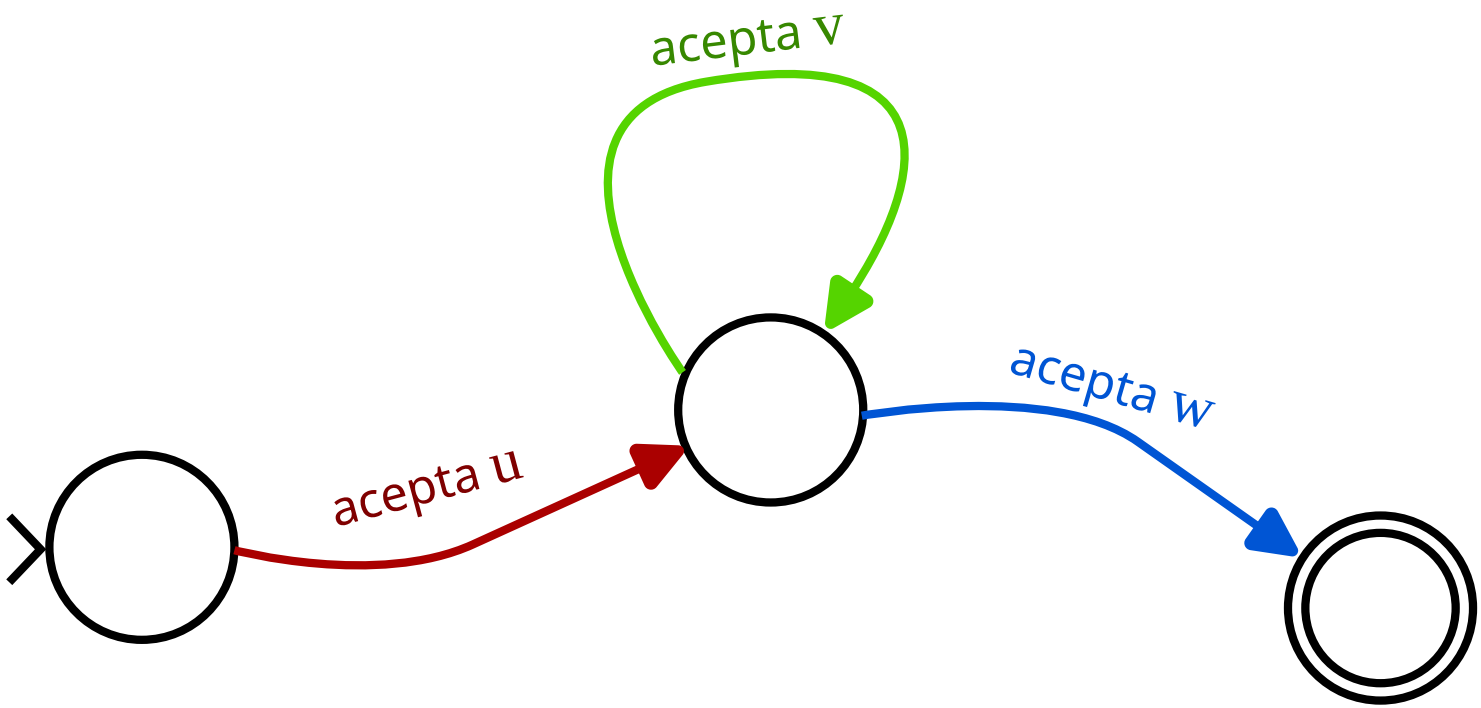
Lema de bombeo y aplicaciones

Operaciones con lenguajes regulares

Algoritmos para autómatas

Minimización de autómatas: estados indistinguibles.

Lema de bombeo



Lema de bombeo

Lema de bombeo

Sea L un lenguaje regular, entonces *existe* un $n \in \mathbb{N}$ tal que $\forall z \in L$, *si* $|z| \geq n$, entonces z *se puede* expresar de la forma $z = uvw$ donde

1. $|uv| \leq n$
2. $|v| \geq 1$
3. $(\forall i \geq 0) \quad uv^i w \in L$

además n puede ser el número de estados de cualquier autómata que acepte el lenguaje L.

Comentarios

- Es un lema.

Comentarios

- Es un lema.
- Es útil para demostrar que un determinado lenguaje no es regular.

Comentarios

- Es un lema.
- Es útil para demostrar que un determinado lenguaje no es regular.
- Es una condición necesaria para los lenguajes regulares.

Comentarios

- Es un lema.
- Es útil para demostrar que un determinado lenguaje no es regular.
- Es una condición necesaria para los lenguajes regulares.
- No es una buena guía para descubrir si un lenguaje es o no regular.

Demostración

Sea $M = (Q, A, \delta, q_0, F)$ un autómata finito determinista que acepta el lenguaje L y n su número de estados.

Supongamos $z \in L, \quad z = a_1 a_2 \dots a_m$ con $m \geq n$.

Como M acepta el lenguaje L y $z \in L$, tenemos que al leer z en M se llega desde el estado inicial a un estado final.

Demostración

Sea $M = (Q, A, \delta, q_0, F)$ un autómata finito determinista que acepta el lenguaje L y n su número de estados.

Supongamos $z \in L, \quad z = a_1 a_2 \dots a_m$ con $m \geq n$.

Como M acepta el lenguaje L y $z \in L$, tenemos que al leer z en M se llega desde el estado inicial a un estado final.

Sea $z' = a_1 a_2 \dots a_n$ la palabra formada por los n primeros símbolos de z.

Consideremos el vector de estados $(q_{i_0}, q_{i_1}, \dots, q_{i_n})$ donde q_{i_0} es el estado inicial, q_0, y $q_{i_j} = \delta(q_{i_{j-1}}, a_j)$

Demostración

Sea $M = (Q, A, \delta, q_0, F)$ un autómata finito determinista que acepta el lenguaje L y n su número de estados.

Supongamos $z \in L, \quad z = a_1 a_2 \dots a_m$ con $m \geq n$.

Como M acepta el lenguaje L y $z \in L$, tenemos que al leer z en M se llega desde el estado inicial a un estado final.

Sea $z' = a_1 a_2 \dots a_n$ la palabra formada por los n primeros símbolos de z.

Consideremos el vector de estados $(q_{i_0}, q_{i_1}, \dots, q_{i_n})$ donde q_{i_0} es el estado inicial, q_0, y $q_{i_j} = \delta(q_{i_{j-1}}, a_j)$

Hay n estados distintos y el vector es de longitud $n+1$: algún estado se debe de repetir. Supongamos que se repiten $q_{i_k} = q_{i_l}$.

Demostración

Sea $M = (Q, A, \delta, q_0, F)$ un autómata finito determinista que acepta el lenguaje L y n su número de estados.

Supongamos $z \in L, \quad z = a_1 a_2 \dots a_m$ con $m \geq n$.

Como M acepta el lenguaje L y $z \in L$, tenemos que al leer z en M se llega desde el estado inicial a un estado final.

Sea $z' = a_1 a_2 \dots a_n$ la palabra formada por los n primeros símbolos de z.

Consideremos el vector de estados $(q_{i_0}, q_{i_1}, \dots, q_{i_n})$ donde q_{i_0} es el estado inicial, q_0, y $q_{i_j} = \delta(q_{i_{j-1}}, a_j)$

Hay n estados distintos y el vector es de longitud $n+1$: algún estado se debe de repetir. Supongamos que se repiten $q_{i_k} = q_{i_l}$.

La descomposición es
$u = a_1 \dots a_k$, $v = a_{k+1} \dots a_l$, $w = a_{l+1} \dots a_m$

Demostración

Sea $M = (Q, A, \delta, q_0, F)$ un autómata finito determinista que acepta el lenguaje L y n su número de estados.

Supongamos $z \in L, \quad z = a_1 a_2 \dots a_m$ con $m \geq n$.

Como M acepta el lenguaje L y $z \in L$, tenemos que al leer z en M se llega desde el estado inicial a un estado final.

Sea $z' = a_1 a_2 \dots a_n$ la palabra formada por los n primeros símbolos de z.

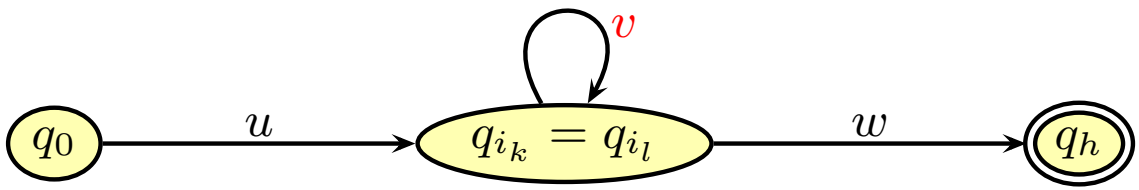
Consideremos el vector de estados $(q_{i_0}, q_{i_1}, \dots, q_{i_n})$ donde q_{i_0} es el estado inicial, q_0, y $q_{i_j} = \delta(q_{i_{j-1}}, a_j)$

Hay n estados distintos y el vector es de longitud $n+1$: algún estado se debe de repetir. Supongamos que se repiten $q_{i_k} = q_{i_l}$.

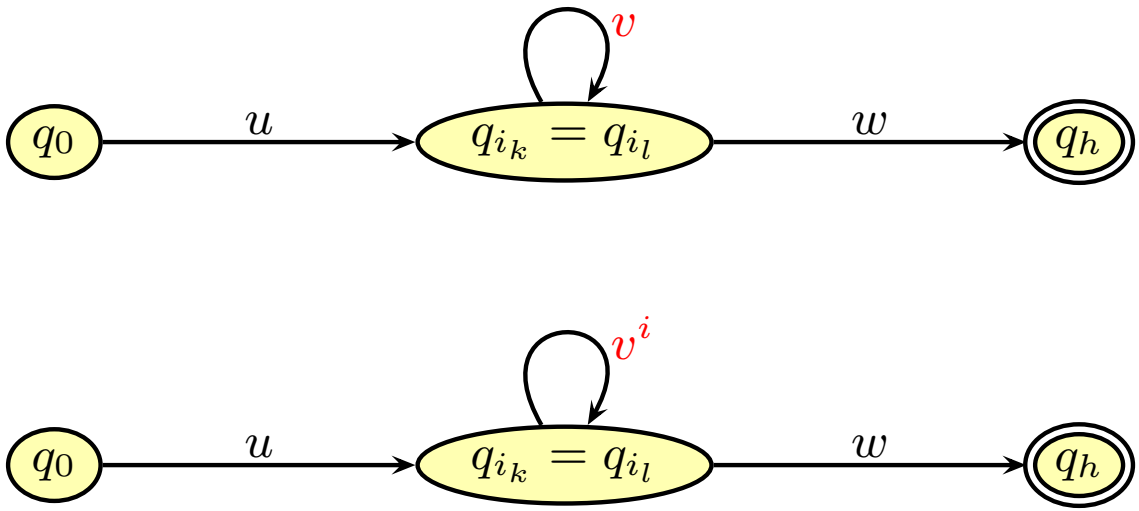
La descomposición es
$u = a_1 \dots a_k$, $v = a_{k+1} \dots a_l$, $w = a_{l+1} \dots a_m$

Como al leer v pasamos por un ciclo (nos movemos de un estado a él mismo), tenemos que $\delta^*(q_0, uv^i w) = \delta^*(q_0, uvw) = \delta^*(q_0, z) \in F$. Y como M acepta L, tenemos que $uv^i w \in L$.

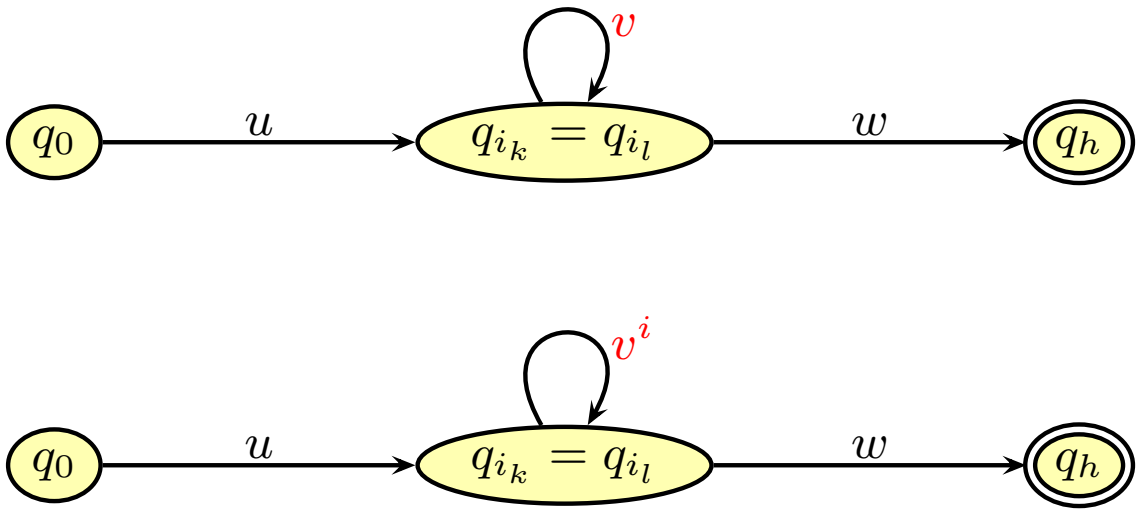
En el diagrama de transición



En el diagrama de transición



En el diagrama de transición



Pertenecen al lenguaje porque llega al mismo estado final que z.

$|uv| \leq n$ porque el ciclo se produce como máximo al leer n símbolos.

$|v| = l - k \geq 1$ porque en el vector de estados siempre se leía un símbolo para pasar al siguiente estado.

Un lenguaje no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$ tal que para toda descomposición

$$z = uvw$$

Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces

$$\exists i \in \mathbb{N}, \text{tal que } uv^i w \notin L$$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$,

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^n 1^n$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^n 1^n$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces tenemos que $u = 0^k, v = 0^l, w = 0^{n-k-l}1^n$, con $l \geq 1$.

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^n 1^n$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces tenemos que $u = 0^k, v = 0^l, w = 0^{n-k-l}1^n$, con $l \geq 1$.

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

Haciendo $i = 2, uv^2w = 0^k 0^{2l} 0^{n-k-l} 1^n = 0^{n+l}1^n \notin L$.

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$,

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces

$\exists i \in \mathbb{N}$, tal que $uv^i w \notin L$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^{n^2}$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces

$\exists i \in \mathbb{N}$, tal que $uv^i w \notin L$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^{n^2}$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces tenemos que $u = 0^k, v = 0^l, w = 0^{n^2-l-k}$, con $l \geq 1, l \leq n$.

$\exists i \in \mathbb{N}$, tal que $uv^iw \notin L$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^{n^2}$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces tenemos que $u = 0^k, v = 0^l, w = 0^{n^2-l-k}$, con $l \geq 1, l \leq n$.

$\exists i \in \mathbb{N}$, tal que $uv^iw \notin L$

Haciendo $i = 2, uv^2w = 0^k 0^{2l} 0^{n^2-l-k} = 0^{n^2+l}$
Como $(n+1)^2 - n^2 = n^2 + 2n + 1 - n^2 = 2n + 1 > n \geq l$, tenemos que
$n^2 < n^2 + l < (n+1)^2$ y $uv^2w = 0^{n^2+l} \notin L$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$,

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^n 1^n 0^n$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^n 1^n 0^n$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces tenemos que $u = 0^k, v = 0^l, w = 0^{n-k-l} 1^n 0^n$, con $l \geq 1$.

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

no es regular

$\forall n \in \mathbb{N}$, existe una palabra $z \in L$, con $|z| \geq n$, $\quad z = 0^n 1^n 0^n$

tal que para toda descomposición $z = uvw$. Si se verifica

- $|uv| \leq n$
- $|v| \geq 1$

entonces tenemos que $u = 0^k, v = 0^l, w = 0^{n-k-l}1^n 0^n$, con $l \geq 1$.

$$\exists i \in \mathbb{N}, \text{ tal que } uv^i w \notin L$$

Haciendo $i = 2, uv^2 w = 0^k 0^{2l} 0^{n-k-l} 1^n 0^n = 0^{n+l} 1^n 0^n \notin L$

Contraejemplo

La condición del lema de bombeo es necesaria, pero no suficiente

El lenguaje $L = \{a^l b^j c^k \ : \ (l = 0) \vee (j = k)\}$ no es regular, pero satisface la condición necesaria que aparece en el lema de bombeo

Se verifica la condición del lema de bombeo para $n = 2$.

Si $z \in L$ y $|z| \geq 2$ entonces $z = a^l b^j c^k$ con $l = 0$ ó $j = k$.

Una descomposición de z se puede obtener de la siguiente forma:

- $u = \epsilon$
- v es el primer símbolo de z
- w es z menos su primer símbolo

Contraej. Cont.

Caben dos posibilidades:

a) $l = 0$. En este caso $z = b^j c^k$ Se verifican las tres condiciones exigidas en el

lema de bombeo.

1. $|uv| = 1 \leq n = 2$
2. $|v| = 1 \geq 1$
3. Si $i \geq 0$ entonces $uv^i w$ sigue siendo una sucesión de b seguida de una sucesión de c y por tanto una palabra de L.

Contraej. Cont.

b) $l \geq 1$. En ese caso $z = a^l b^j c^j (l \geq 1)$, y también aquí se verifican las tres condiciones:

1. $|uv| = 1 \leq n = 2$
2. $|v| = 1 \geq 1$
3. Si $i \geq 0$ entonces $uv^i w$ sigue siendo una sucesión de a seguida de una sucesión de b y otra de c, en la que la cantidad de b es igual que la cantidad de c, y por tanto, una palabra de L.

Contenido

Lema de bombeo y aplicaciones

Operaciones con lenguajes regulares

Algoritmos para autómatas

Minimización de autómatas: estados indistinguibles.

Lenguajes regulares: operaciones

Ya conocemos las siguientes propiedades:

- *Unión:* Si L_1 y L_2 son lenguajes regulares, entonces $L_1 \cup L_2$ es regular.
- *Concatenación:* Si L_1 y L_2 son regulares, entonces $L_1 L_2$ es regular.
- *Clausura de Kleene:* Si L es regular, entonces L^* es regular.

La primera propiedad se obtiene considerando que si L_1 y L_2 son lenguajes regulares entonces tienen dos expresiones regulares, $\mathbf{r_1}$ y $\mathbf{r_2}$. Entonces $\mathbf{r_1} + \mathbf{r_2}$ es una expresión regular para $L_1 \cup L_2$, y como la unión tiene una expresión regular, entonces es regular.

Análogamente se demuestran las otras dos propiedades.

Complementario

Propiedad

Si $L \subseteq A^*$ es un lenguaje regular entonces $\overline{L} = A^* - L$ es regular.

Complementario

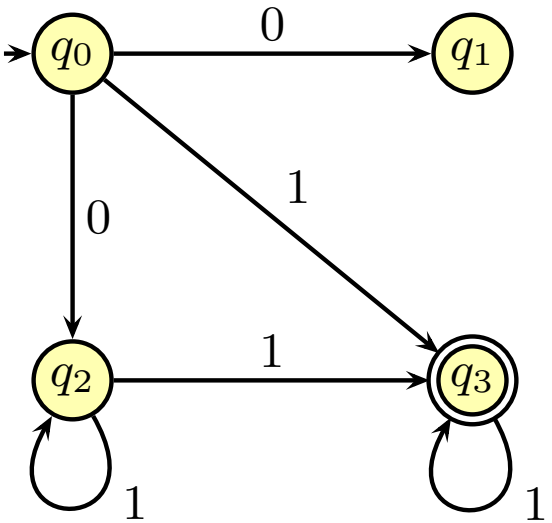
Propiedad

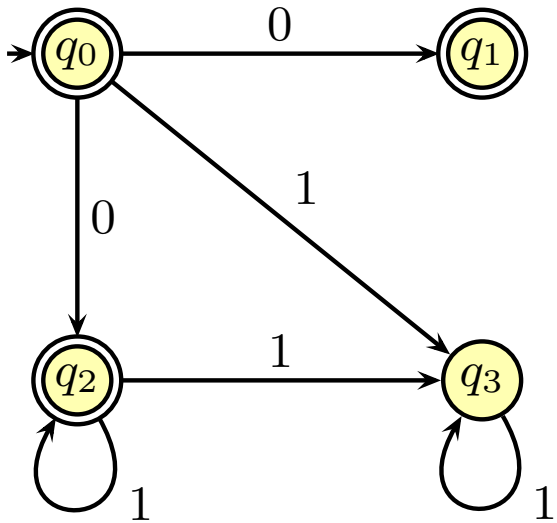
Si $L \subseteq A^*$ es un lenguaje regular entonces $\overline{L} = A^* - L$ es regular.

Basta con considerar que si $M = (Q, A, \delta, q_0, F)$ es un autómata finito determista que acepta el lenguaje L, entonces $M' = (Q, A, \delta, q_0, Q - F)$ acepta el lenguaje complementario $A^* \setminus L$.

No-determinismo

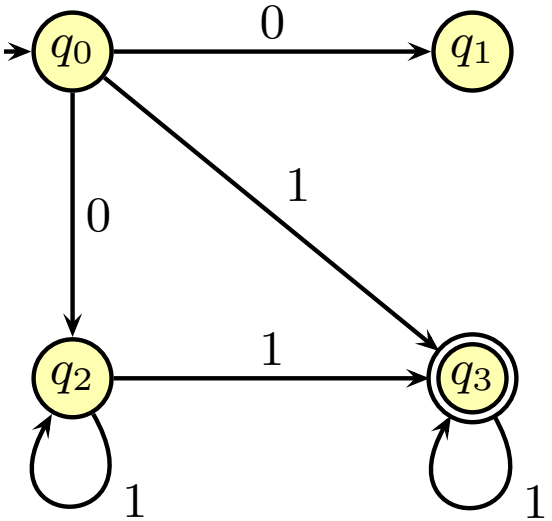
Esta operación no es válida en autómatas no deterministas

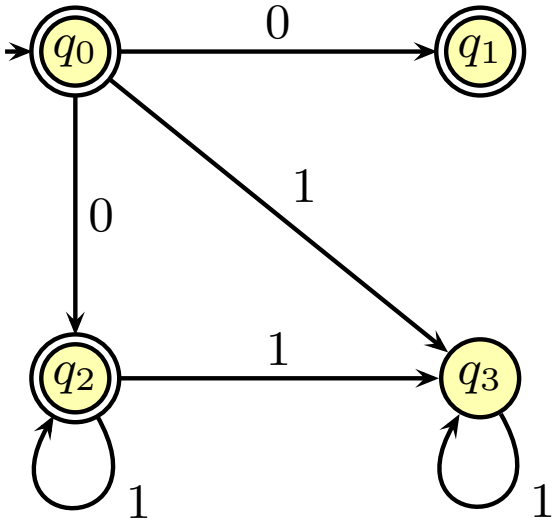




No-determinismo

Esta operación no es válida en autómatas no deterministas

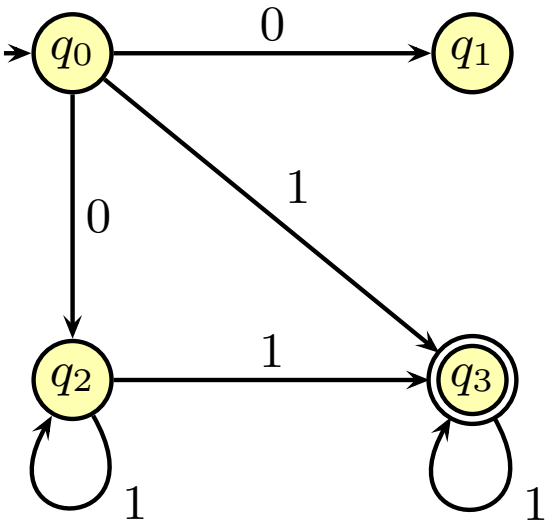


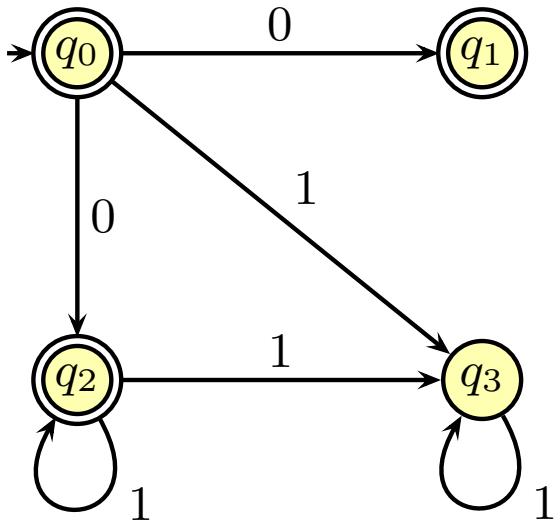


011 es aceptada en ambos autómatas

No-determinismo

Esta operación no es válida en autómatas no deterministas





011 es aceptada en ambos autómatas

100 no es aceptada en ninguno de los autómatas

Intersección

Propiedad

Si L_1 y L_2 son dos lenguajes regulares sobre el alfabeto A, entonces $L_1 \cap L_2$ es regular.

Es inmediato ya que $L_1 \cap L_2 = \overline{(\overline{L_1} \cup \overline{L_2})}$.

Existe también una demostración constructiva. Si $M_1 = (Q_1, A, \delta_1, q_0^1, F_1)$ es un autómata finito determinístico que acepta L_1, y $M_2 = (Q_2, A, \delta_2, q_0^2, F_2)$ es un autómata que acepta L_2, entonces

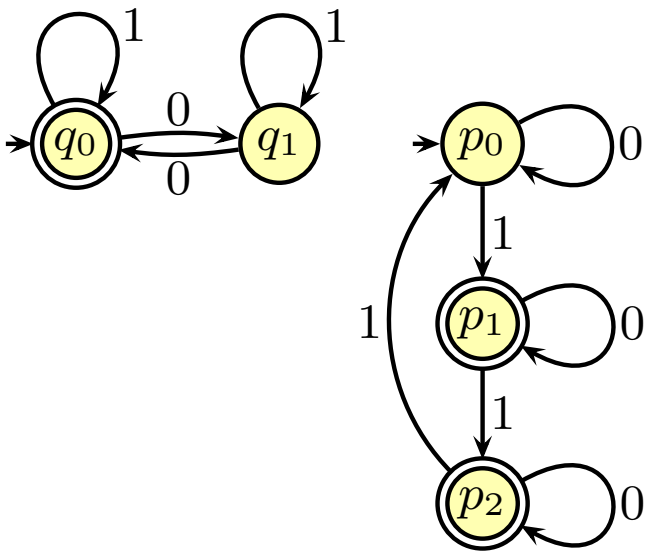
$M = (Q_1 \times Q_2, A, \delta, (q_0^1, q_0^2), F_1 \times F_2)$

donde $\delta((q_i, q_j), a) = (\delta_1(q_i, a), \delta_2(q_j, a))$, acepta el lenguaje $L_1 \cap L_2$.

A este autómata se le llama autómata producto.

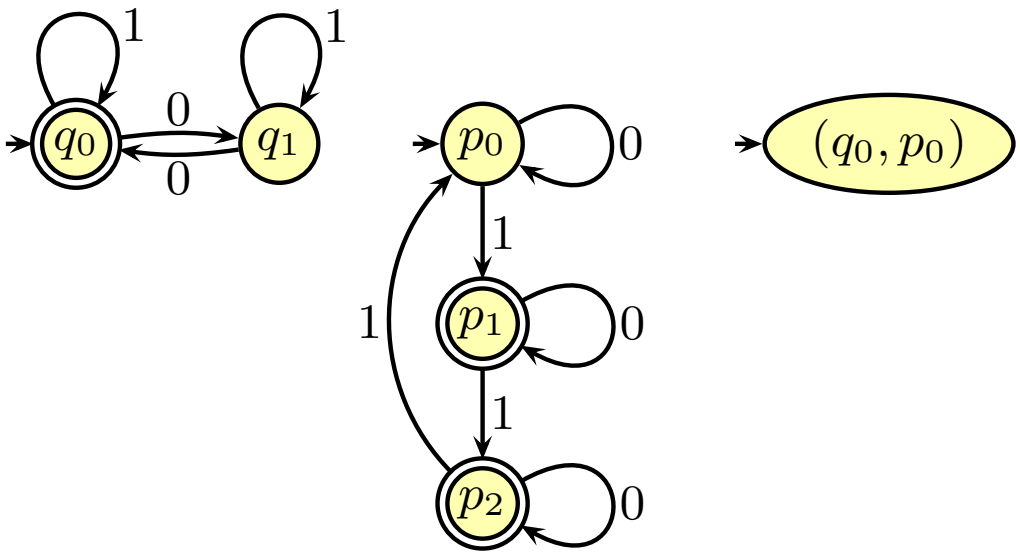
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



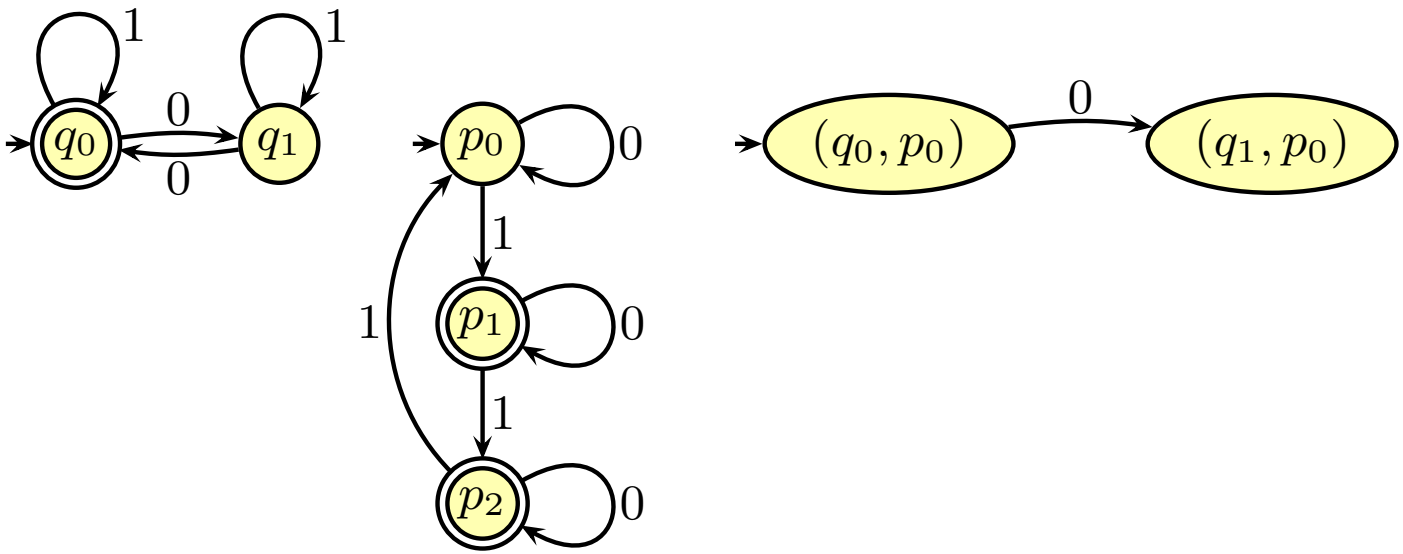
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



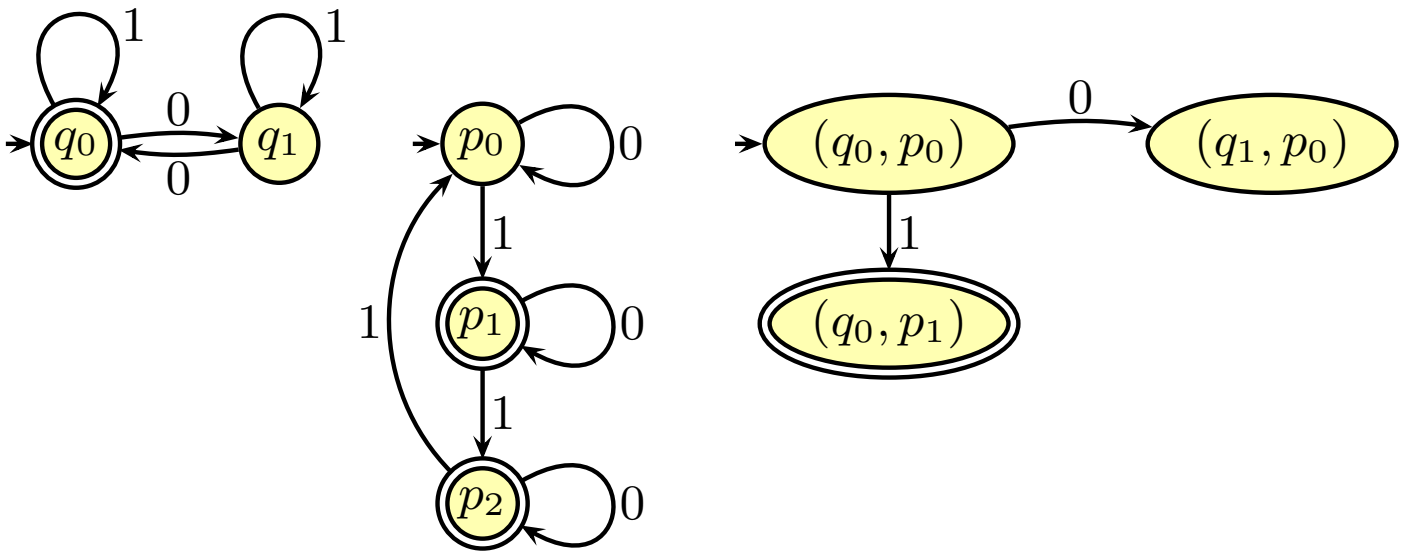
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



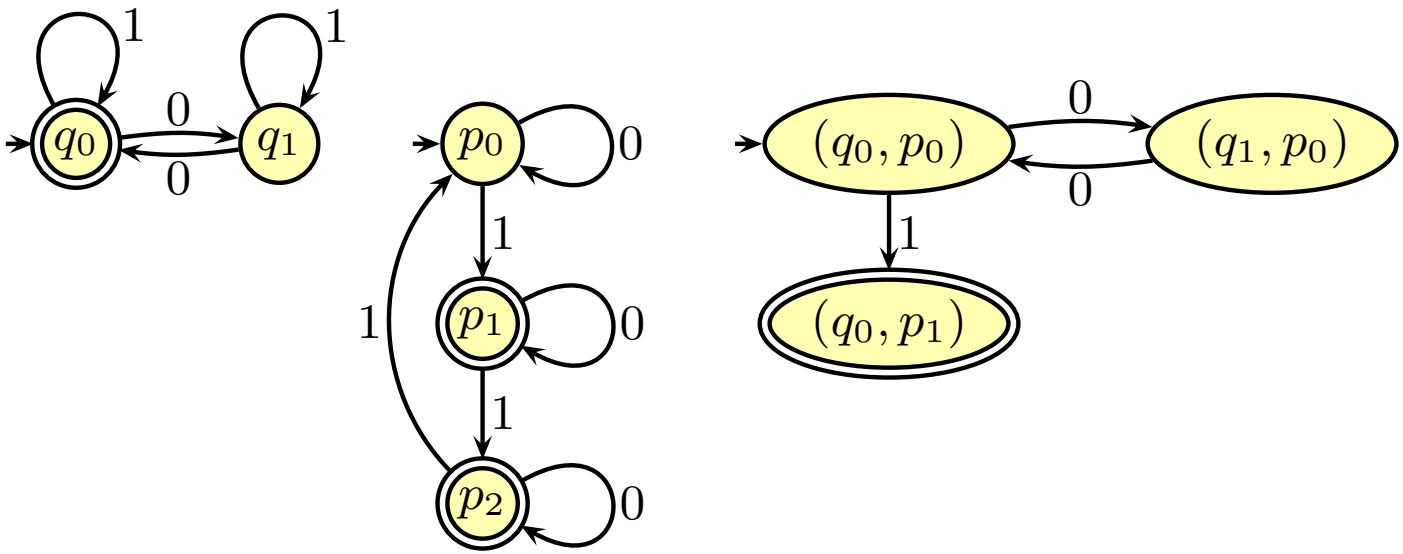
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



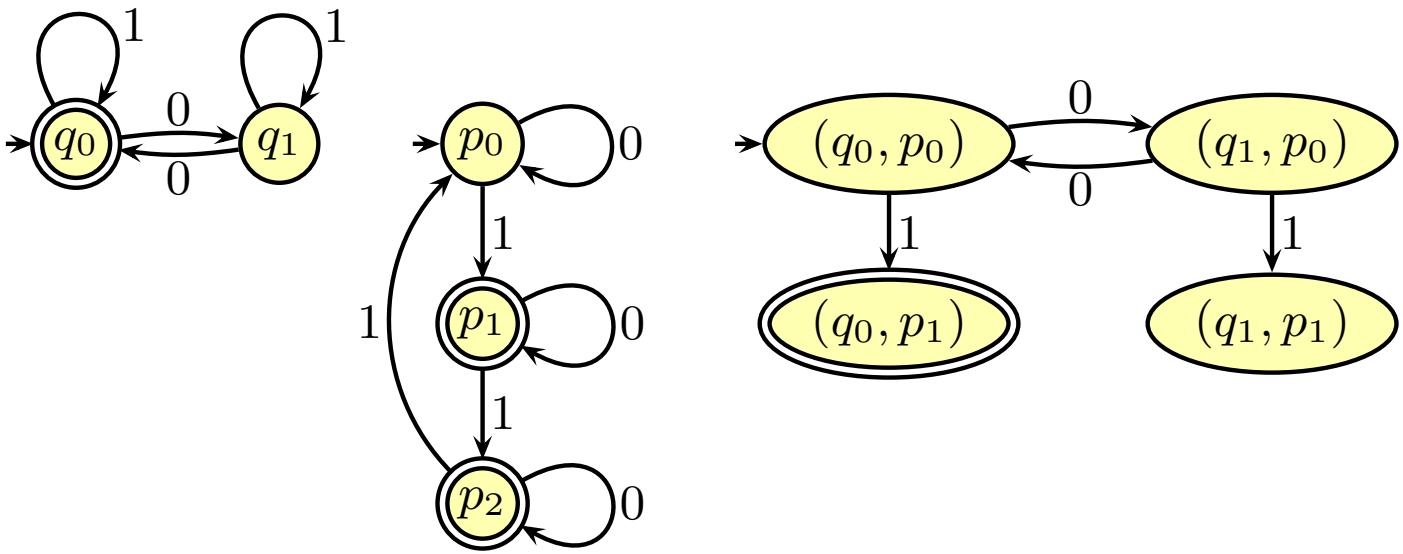
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



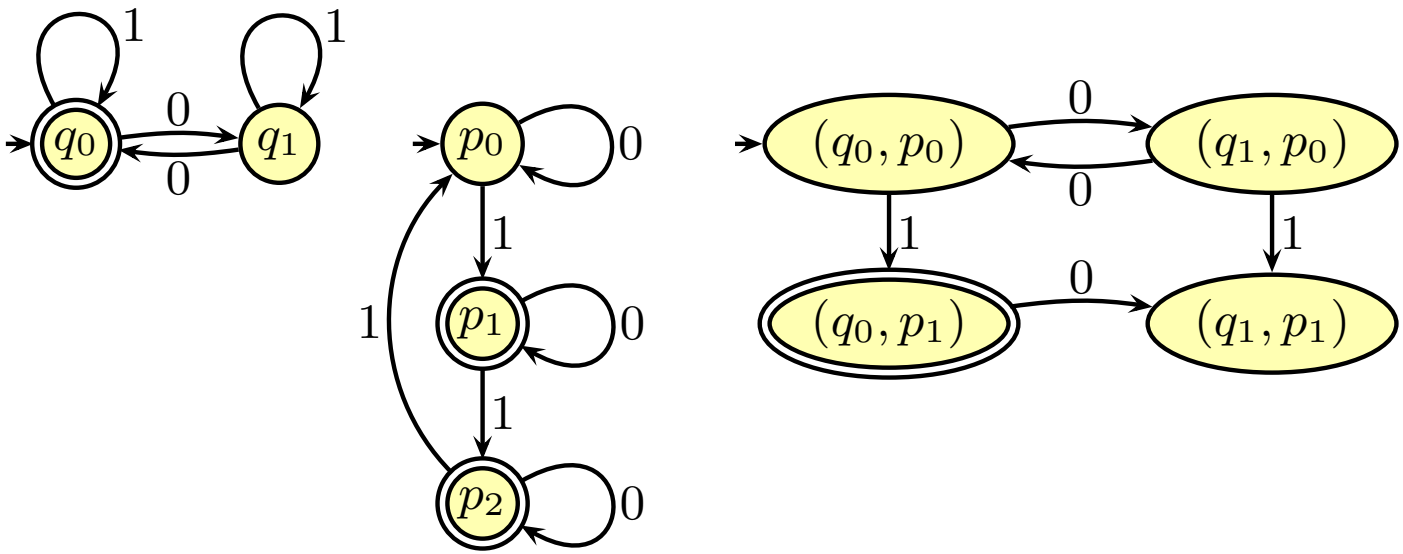
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



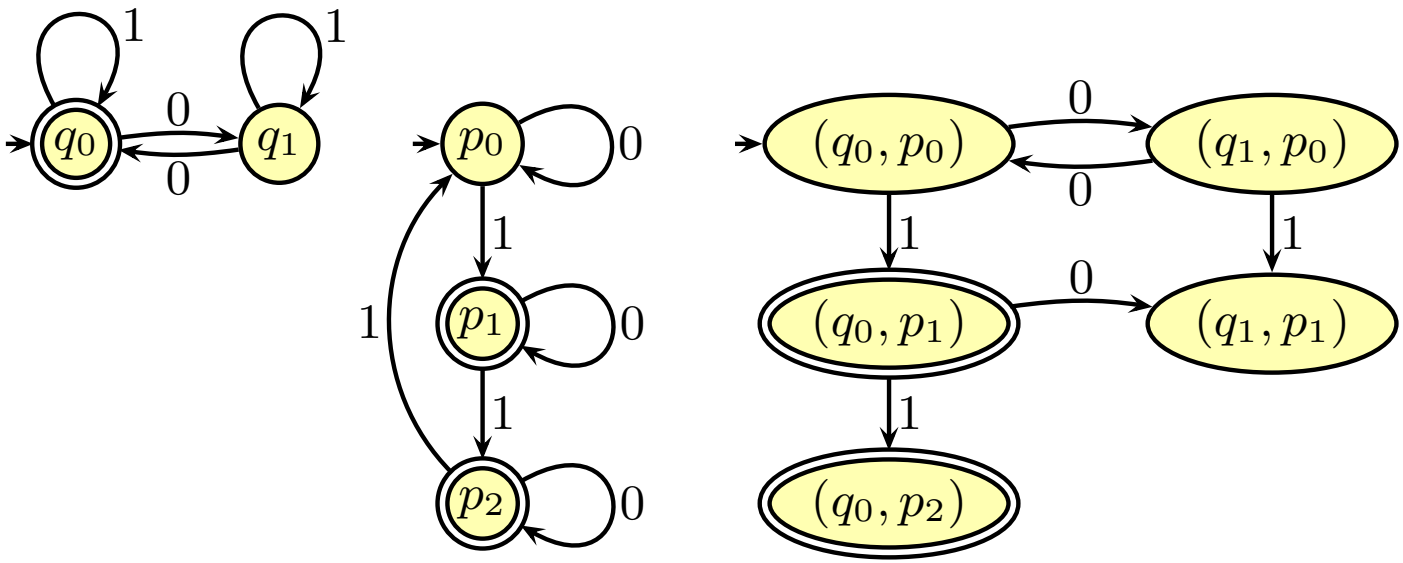
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



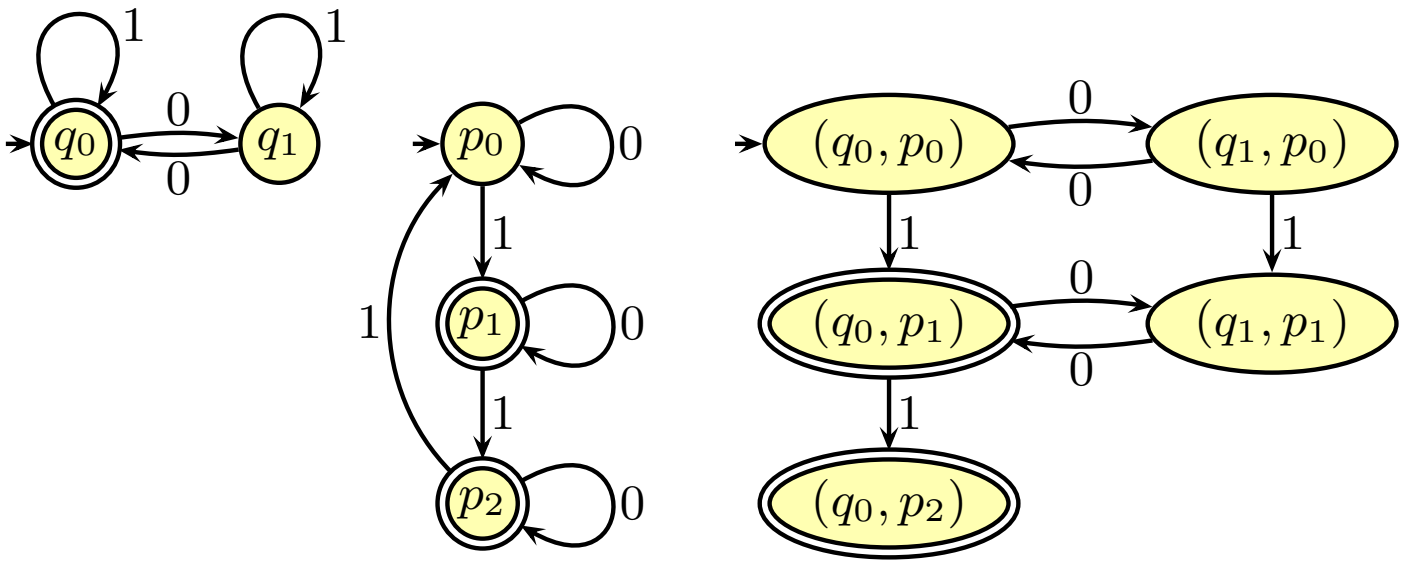
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



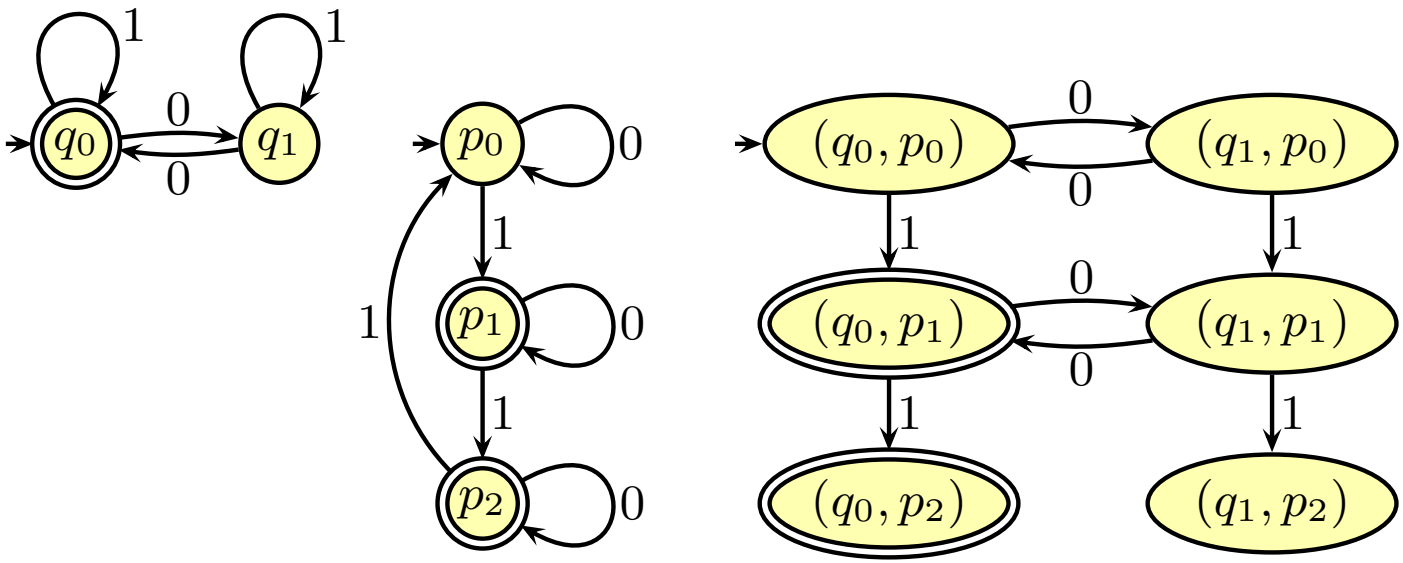
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



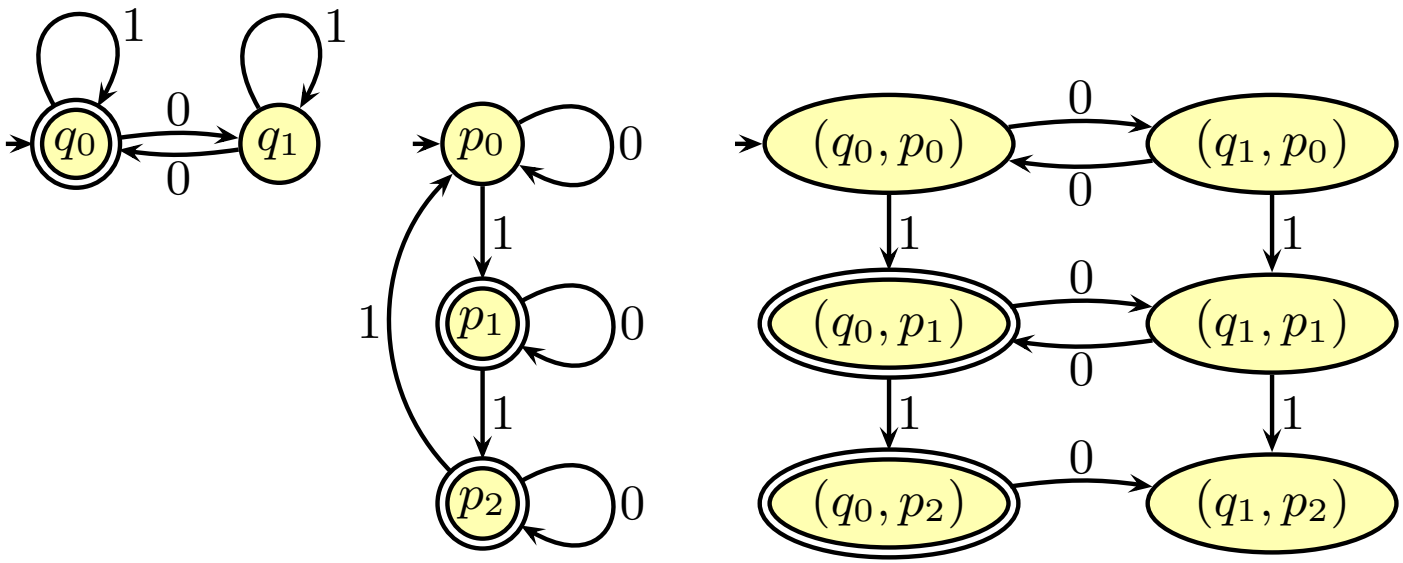
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



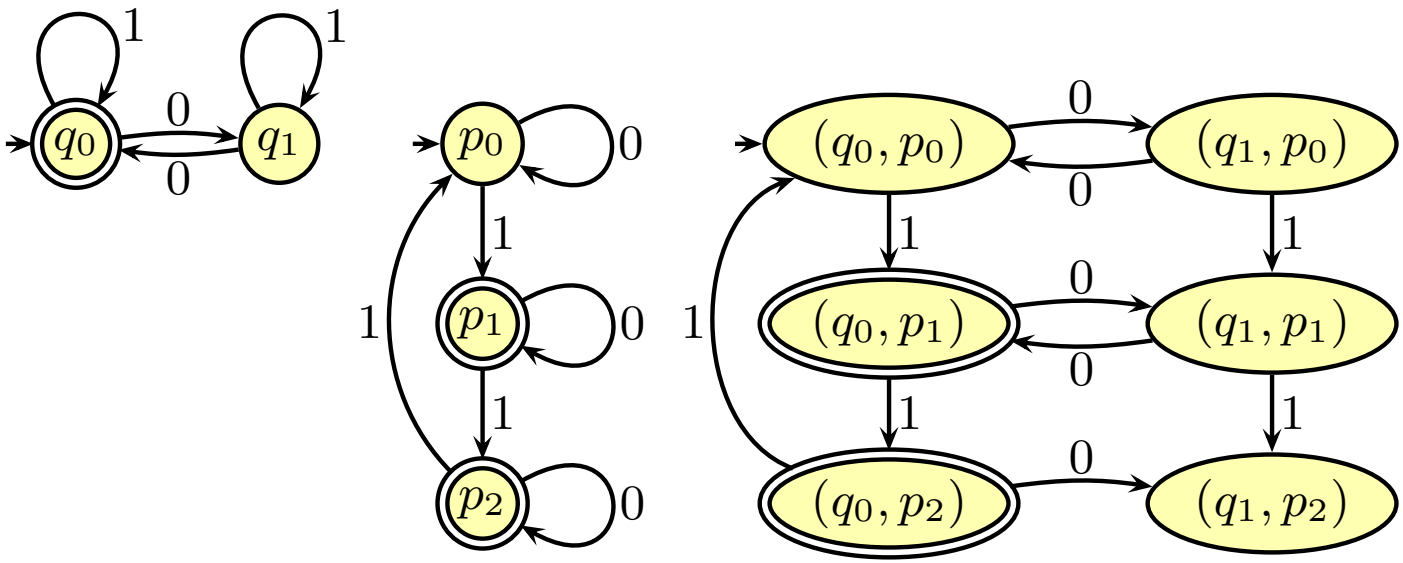
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



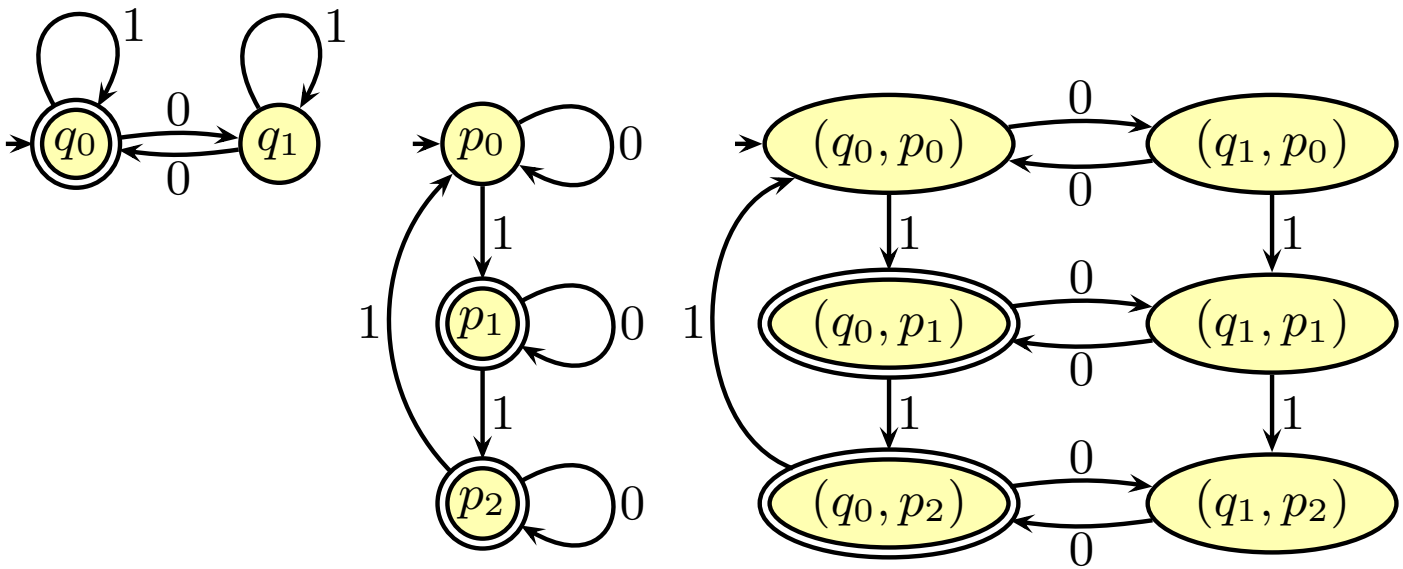
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



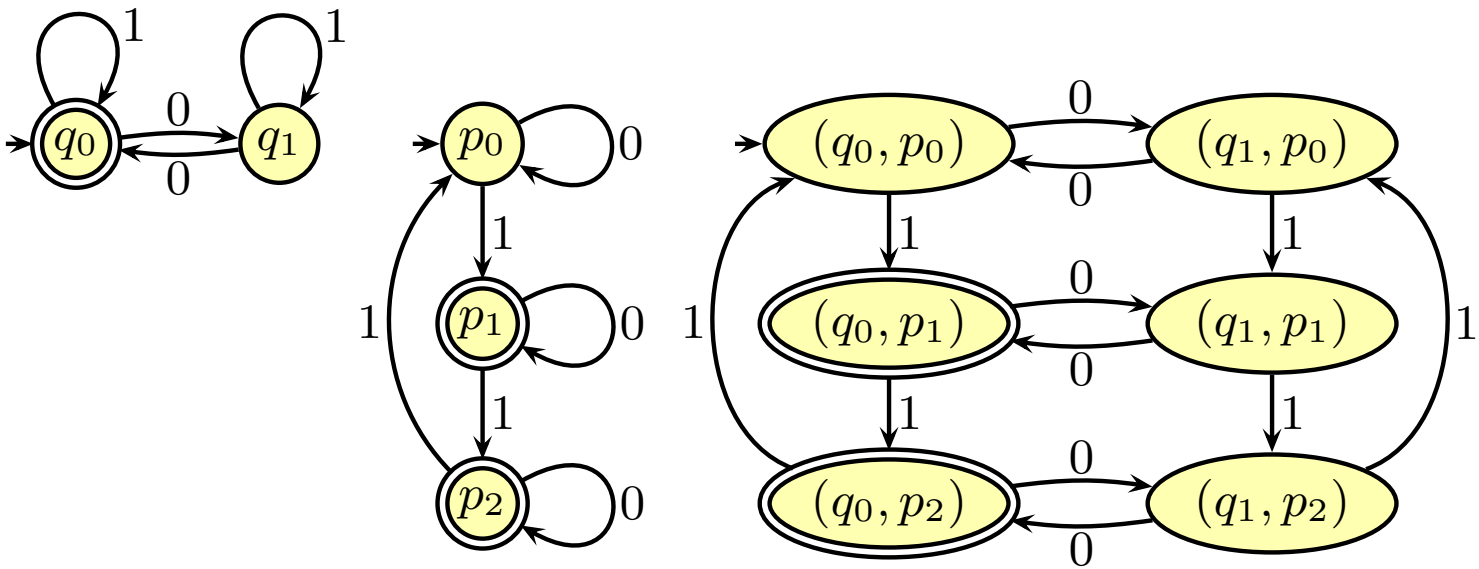
Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.



Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **Y** un número de unos que no sea múltiplo de 3.

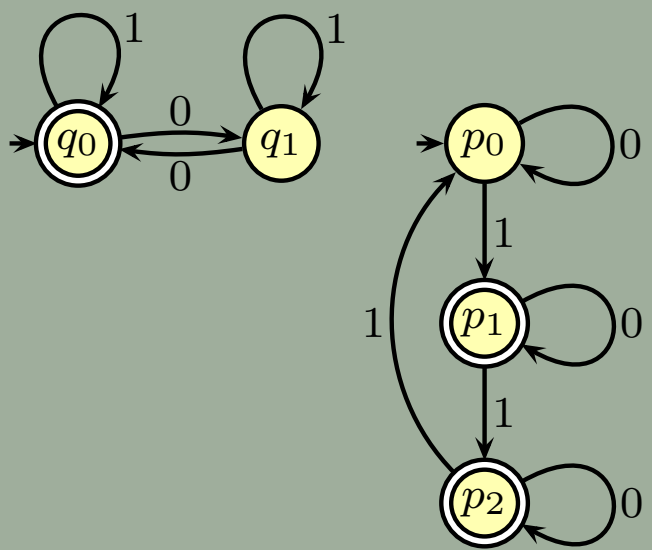


Autómata producto para la unión

Para construir un autómata que acepte la unión de los lenguajes aceptados por dos autómatas con el autómata producto, basta con hacer finales las parejas de estados en las que, al menos, uno de ellos es final.

Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **O** un número de unos que no sea múltiplo de 3.

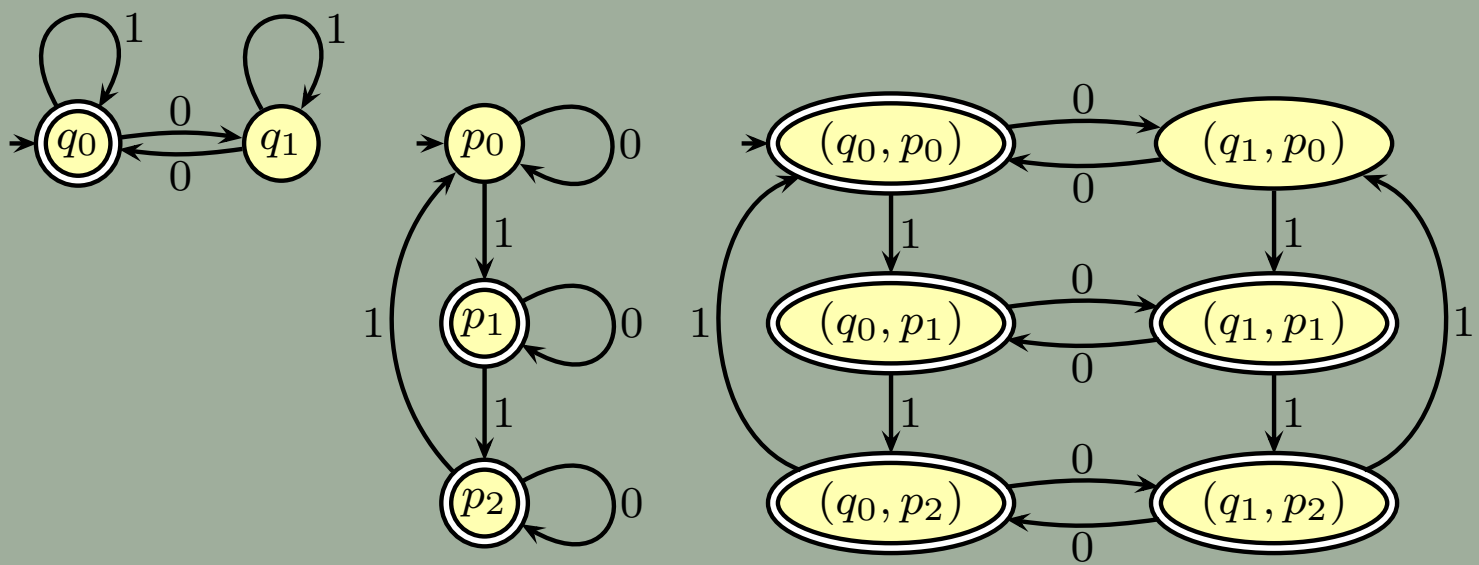


Autómata producto para la unión

Para construir un autómata que acepte la unión de los lenguajes aceptados por dos autómatas con el autómata producto, basta con hacer finales las parejas de estados en las que, al menos, uno de ellos es final.

Ejemplo

Construir el autómata que acepta las palabras con un número de ceros que es múltiplo de 2 **O** un número de unos que no sea múltiplo de 3.



Homomorfismo

Propiedad

Si A y B son alfabetos y $f : A^* \longrightarrow B^*$ un homomorfismo entre ellos, **entonces** si $L \subseteq A^*$ es un lenguaje regular, $f(L) = \{u \in B^* : \exists v \in A^* \text{ verificando } f(v) = u\}$ es también un lenguaje regular.

Basta con comprobar que se puede conseguir una expresión regular para $f(L)$ partiendo de una expresión regular para L: Basta con substituir cada símbolo, a, de L, por la correspondiente palabra $f(a)$.

Ejemplo

Si $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ y $B = \{0, 1\}$ y f es el homomorfismo dado por

$$f(0) = 0000, \quad f(1) = 0001, \quad f(2) = 0010, \quad f(3) = 0011$$
$$f(4) = 0100, \quad f(5) = 0101, \quad f(6) = 0110, \quad f(7) = 0111$$
$$f(8) = 1000, \quad f(9) = 1001$$

Si $L \subseteq A^*$ es el lenguaje regular dado por la expresión regular $(1 + 2)^*9$,

$f(L)$ viene dado por la expresión regular $(0001 + 0010)^*1001$.

Homomorfismo inverso

Propiedad

Si A y B son alfabetos y $f : A^* \longrightarrow B^*$ es un homomorfismo, entonces si $L \subseteq B^*$ es un lenguaje regular, también lo es $f^{-1}(L) = \{u \in A^* \ : \ f(u) \in L\}$.

Supongamos que $M = (Q, B, \delta, q_0, F)$ que acepta el lenguaje L

Entonces el autómata $\overline{M} = (Q, A, \overline{\delta}, q_0, F)$ donde

$$\overline{\delta}(q, a) = \delta^*(q, f(a)),$$

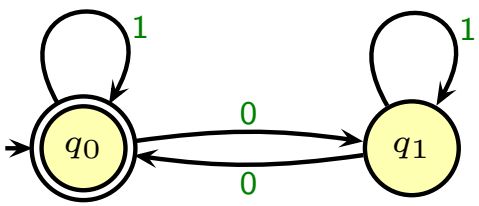
acepta el lenguaje $f^{-1}(L)$.

Ejemplo

En el homomorfismo f entre $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ y $B = \{0, 1\}$ dado por:

$$f(0) = 0000, \quad f(1) = 0001, \quad f(2) = 0010, \quad f(3) = 0011, \quad f(4) = 0100,$$
$$f(5) = 0101, \quad f(6) = 0110, \quad f(7) = 0111, \quad f(8) = 1000, \quad f(9) = 1001$$

El conjunto L de las palabras con un número par de ceros es:

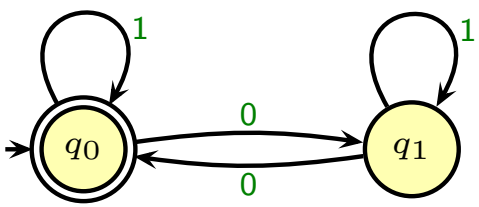


Ejemplo

En el homomorfismo f entre $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ y $B = \{0, 1\}$ dado por:

$$f(0) = 0000, \quad f(1) = 0001, \quad f(2) = 0010, \quad f(3) = 0011, \quad f(4) = 0100,$$
$$f(5) = 0101, \quad f(6) = 0110, \quad f(7) = 0111, \quad f(8) = 1000, \quad f(9) = 1001$$

El conjunto L de las palabras con un número par de ceros es:



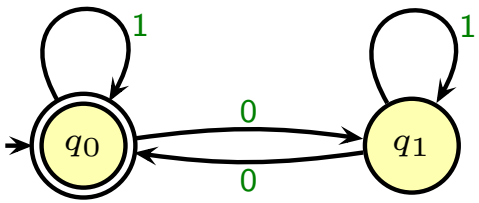
y el conjunto de palabras $u \in A^*$ tales que $f(u)$ tiene un número par de ceros (es decir $f^{-1}(L)$) es regular con autómata:

Ejemplo

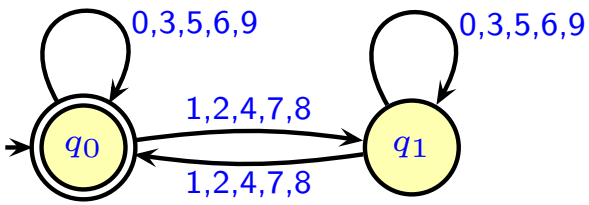
En el homomorfismo f entre $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ y $B = \{0, 1\}$ dado por:

$$f(0) = 0000, \quad f(1) = 0001, \quad f(2) = 0010, \quad f(3) = 0011, \quad f(4) = 0100,$$
$$f(5) = 0101, \quad f(6) = 0110, \quad f(7) = 0111, \quad f(8) = 1000, \quad f(9) = 1001$$

El conjunto L de las palabras con un número par de ceros es:



y el conjunto de palabras $u \in A^*$ tales que $f(u)$ tiene un número par de ceros (es decir $f^{-1}(L)$) es regular con autómata:



Aplicación

Esta propiedad también se puede usar para demostrar que un lenguaje no es regular por reducción al absurdo.

Ejemplo

Si $A = B = \{0, 1\}$ y f es el homomorfismo dado por

$$f(0) = 00, \qquad f(1) = 11$$

entonces el lenguaje $L = \{0^{2k}1^{2k} : k \geq 0\}$ no es regular, porque si lo fuese su imagen inversa, $f^{-1}(L) = \{0^k 1^k : k \geq 0\}$ sería también regular y no lo es.

Contenido

Lema de bombeo y aplicaciones

Operaciones con lenguajes regulares

Algoritmos para autómatas

Minimización de autómatas: estados indistinguibles.

Algoritmos

Algoritmo

Existe un algoritmo para determinar si el lenguaje aceptado por un autómata es vacío

Algoritmos

Algoritmo

Existe un algoritmo para determinar si el lenguaje aceptado por un autómata es vacío

Basta eliminar estados inaccesibles (mediante un recorrido por el grafo a partir del estado inicial) y comprobar si quedan estados finales.

Algoritmos

Algoritmo

Existe un algoritmo para determinar si el lenguaje aceptado por un autómata es vacío

Basta eliminar estados inaccesibles (mediante un recorrido por el grafo a partir del estado inicial) y comprobar si quedan estados finales.

El lenguaje aceptado por un autómata es finito o infinito:

Algoritmos

Algoritmo

Existe un algoritmo para determinar si el lenguaje aceptado por un autómata es vacío

Basta eliminar estados inaccesibles (mediante un recorrido por el grafo a partir del estado inicial) y comprobar si quedan estados finales.

El lenguaje aceptado por un autómata es finito o infinito:

Se suponen eliminados los estados inaccesibles y se eliminan los estados de error o estados desde los que no se pueden llegar a estado finales.

Se puede hacer recorriendo el grafo en sentido contrario a los arcos y empezando en los estados finales.

Se comprueba si en el grafo resultante quedan ciclos.

Algoritmos: igualdad

Algoritmo

Existe un algoritmo para el problema: **Dados dos autómatas finitos M_1 y M_2 comprobar si aceptan el mismo lenguaje:**

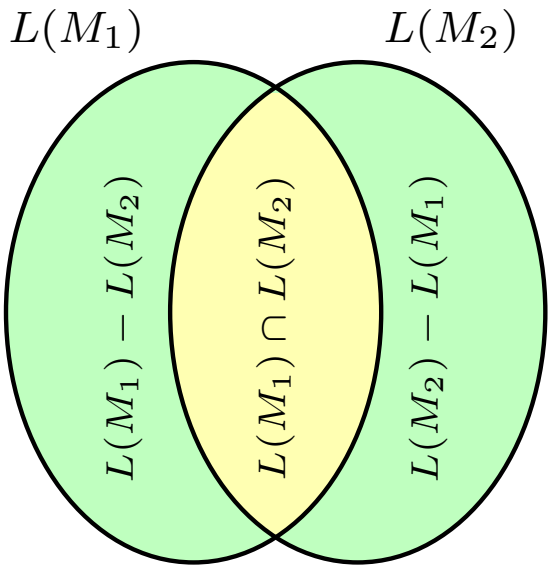
Algoritmos: igualdad

Algoritmo

Existe un algoritmo para el problema: **Dados dos autómatas finitos M_1 y M_2 comprobar si aceptan el mismo lenguaje:**

Basta con construir el autómata que acepta el lenguaje
$(L(M_1) - L(M_2)) \cup (L(M_2) - L(M_1)) = (L(M_1) \cap \overline{L(M_2)}) \cup (L(M_2) \cap \overline{L(M_1)})$

Después se comprueba si el lenguaje aceptado por este autómata es vacío.



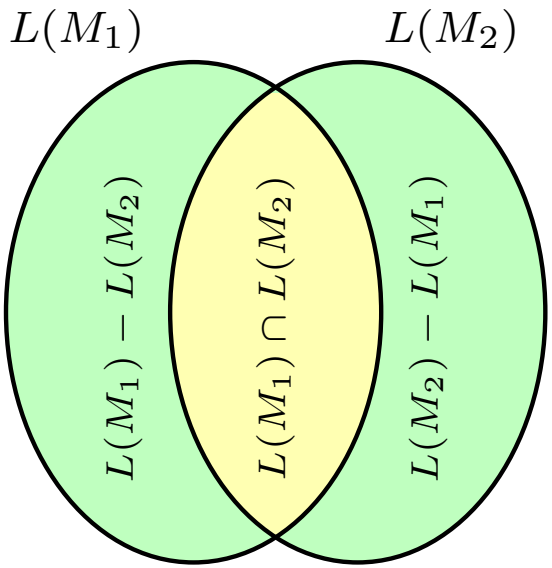
Algoritmos: igualdad

Algoritmo

Existe un algoritmo para el problema: **Dados dos autómatas finitos M_1 y M_2 comprobar si aceptan el mismo lenguaje:**

Basta con construir el autómata que acepta el lenguaje
$(L(M_1) - L(M_2)) \cup (L(M_2) - L(M_1)) = (L(M_1) \cap \overline{L(M_2)}) \cup (L(M_2) \cap \overline{L(M_1)})$

Después se comprueba si el lenguaje aceptado por este autómata es vacío.



La mejor forma de hacer este autómata es con el autómata producto, haciendo finales las parejas de estados en las que un estado es final y el otro no.

Contenido

Lema de bombeo y aplicaciones

Operaciones con lenguajes regulares

Algoritmos para autómatas

Minimización de autómatas: estados indistinguibles.

Autómata Minimal

Un autómata finito determinista M se dice minimal si no exite otro autómata con menos estados que él y que acepte el mismo lenguaje.

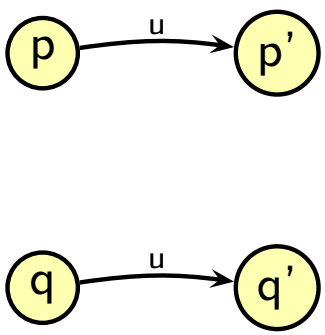
- La primera condición para que un autómata sea minimal es que no tenga estados inaccesibles.
- Vamos a ver primero una condición para que un autómata sin estados indistinguibles sea minimal: que no tenga estados indistinguibles.
- A continuación veremos un algoritmo que, dado un autómata M calcula un autómata minimal que acepta el mismo lenguaje.

Minimización de autómatas

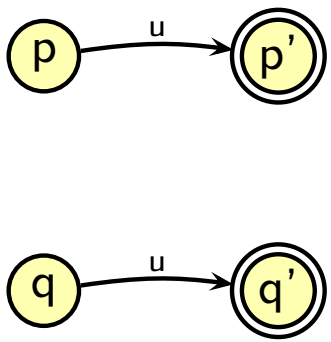
Un concepto básico para minimizar autómatas es el de estados indistinguibles.

Si $M = (Q, A, \delta, q_0, F)$ es un autómata finito determinista y p, q son dos estados de Q, decimos que p y q son indistinguibles si y solo si se cumple que

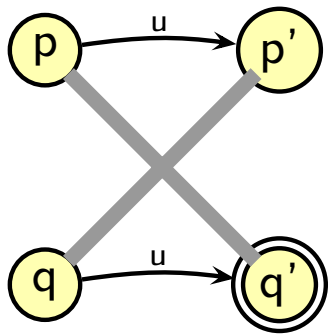
$$\forall u \in A^*, \quad (\delta^*(p, u) \in F \Leftrightarrow \delta^*(q, u) \in F)$$



SÍ



SÍ



NO

Relación de indistingubilidad

Propiedades

La relación ser indistinguible de es una relación de equivalencia en el conjunto Q de estados: es reflexiva, simétrica y transitiva.

Cuando dos estados no son indistinguibles se dice que son distinguibles.

Propiedad

Un estado final y un no final son siempre distinguibles: al leer la palabra vacía en un caso se llega a un estado final y en otro a un no final.

Propiedad

Si p y q son dos estados indistinguibles, entonces para todo $a \in A$, los estados $\delta(p, a)$ y $\delta(q, a)$ son también indistinguibles.

Propiedad

Si p y q son dos estados indistinguibles, entonces para todo $u \in A^*$, los estados $\delta^*(p, u)$ y $\delta^*(q, u)$ son también indistinguibles.

Minimización de autómatas

Teorema

Si $M = (Q, A, \delta, q_0, F)$ es un autómata finito determinista y $q_1, q_2 \in Q$ son una pareja de estados indistiguibles distintos, entonces el autómata $M' = (Q', A, \delta', q_0', F')$ donde

- $Q' = Q \setminus \{q_1\}$
- $\delta'(p, a) = \delta(p, a)$ si $\delta(p, a) \neq q_1$ y $\delta'(p, a) = q_2$ si $\delta(p, a) = q_1$.
- $q_0' = q_0$ si $q_0 \neq q_1$ y $q_0' = q_2$ si $q_0 = q_1$
- $F' = F \setminus \{q_1\}$

acepta el mismo lenguaje que M.

La demostración se basa en comprobar que $\delta^*(p, u)$ y $\delta'^*(q, u)$ son siempre dos estados indistinguibles en M para toda pareja p y q de estados indistinguibles en M y $q \in Q'$.

Para $u = \epsilon$, $\delta^*(p, \epsilon) = p$ y $\delta^*(q, \epsilon) = q = \delta'^*(q, \epsilon)$ que son indistinguibles.

Si es cierto para palabras de longitud n, entonces una palabra u de longitud $n + 1$ es de la forma $u = av$ donde $a \in$ y v es de longitud n.

Como $\delta^*(p, av) = \delta^*(\delta(p, a), v)$ y $\delta'^*(q, av) = \delta'^*(\delta'(q, a), v)$.

p y q son indistinguibles en M y por lo tanto $\delta(q, a)$ y $\delta(q, a)$ son indistinguibles en M, como como $\delta(q, a)$ y $\delta'(q, a)$ son indistinguibles (son el mismos estado o q_1 y q_2), entonces por la propiedad transitiva de la indisguibilidad $\delta(p, a)$ y $\delta'(q, a)$ son también indistinguibles.

Ahora se aplica la hipótesis de inducción y se comprueba que $\delta^*(p, av)$ y $\delta'^*(q, av)$ son también indistinguibles.

Como q_0 y q_0' son indistinguibles, entonces $\delta^*(q_0, u)$ y $\delta'^*(q_0', u)$ son indistinguibles y como en una pareja de estados indistinguibles no puede ser uno final y otro no final, los dos autómatas aceptan el mismo lenguaje.

Minimización de autómatas

Propiedad

Un autómata sin estados inaccesibles es **minimal** si y solo si no tiene una pareja de estados distintos indistinguibles.

Demostraremos que un autómata no es minimal $\Leftrightarrow$ tiene estados indistinguibles.

$\Leftarrow$ Si q_1 y q_2 son estados indistinguibles de M, entonces el autómata M' del teorema anterior acepta el mismo lenguaje y tiene el mismo estado. Por tanto, M no sería minimal.

Minimización de autómatas

Propiedad

Un autómata sin estados inaccesibles es **minimal** si y solo si no tiene estados indistinguibles.

Demostraremos que un autómata no es minimal $\Leftrightarrow$ tiene estados indistinguibles.

$\Rightarrow$ Si M no es minimal existe otro autómata M' con menos estados que acepta el mismo lenguaje.
Es fácil comprobar que exiten dos palabras $u, v \in A^*$ tales que al leerlas en M se llega a dos estados distintos p y q; y al leerlas en M' se llega al mismo estado p'.
Se puede comprobar que p y q tienen que ser indistinguibles, ya que M y M' aceptan el mismo lenguaje.

Unicidad del autómata minimal

Teorema de unicidad

Si $M = (Q, A, \delta, q_0, F)$ y $M' = (Q', A, \delta', q_0', F')$ son dos autómatas minimales que aceptan el mismo lenguaje, entonces son isoformos, es decir existe una aplicación biyectiva $f : Q \to Q'$, tal que

- $f(q_0) = q_0'$
- $f(\delta(q, a)) = \delta(f(q), a), \forall a \in A, q \in Q$
- $f(F) = F'$

La demostración se basa en lo siguiente: para cualquier estado $q \in Q$, este estado es accesible. Por lo tanto existe una palabra $u_q \in A^*$ tal que $\delta^*(q_0, u_q) = q$. Entonces hacemos $f(q) = \delta'^*(q_0', u_q)$.

La demostración se obtiene fácilmente teniendo en cuenta que no hay parejas de estados indistinguibles en ningún autómata y que no puede ocurrir que al leer dos palabras $u, v \in A^*$, en uno de los autómatas pasemos a estados indistinguibles y en otro autómata pasemos a una pareja de estados distinguibles.

Identificación de estados indistinguibles

Dos estados p, q son distinguibles de nivel n si y solo si existe una palabra $u \in A^*$ de longitud menor o igual que n tal que en el conjunto $\{\delta^*(p,u), \delta^*(q,u)\}$ hay un estado final y otro no final.

Identificación de estados indistinguibles

Dos estados p, q son distinguibles de nivel n si y solo si existe una palabra $u \in A^*$ de longitud menor o igual que n tal que en el conjunto $\{\delta^*(p,u), \delta^*(q,u)\}$ hay un estado final y otro no final.

Una pareja de estados es distinguible si y solo si es distinguible a nivel n para algún $n \in \mathbb{N}$.

Identificación de estados indistinguibles

Dos estados p, q son distinguibles de nivel n si y solo si existe una palabra $u \in A^*$ de longitud menor o igual que n tal que en el conjunto $\{\delta^*(p,u), \delta^*(q,u)\}$ hay un estado final y otro no final.

Una pareja de estados es distinguible si y solo si es distinguible a nivel n para algún $n \in \mathbb{N}$.

Las parejas distinguibles a nivel 0 son las formadas por un estado final y otro no final.

Identificación de estados indistinguibles

Dos estados p, q son distinguibles de nivel n si y solo si existe una palabra $u \in A^*$ de longitud menor o igual que n tal que en el conjunto $\{\delta^*(p,u), \delta^*(q,u)\}$ hay un estado final y otro no final.

Una pareja de estados es distinguible si y solo si es distinguible a nivel n para algún $n \in \mathbb{N}$.

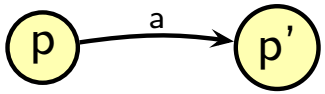
Las parejas distinguibles a nivel 0 son las formadas por un estado final y otro no final.

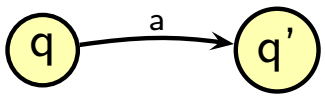
Las parejas $\{p, q\}$ distinguibles a nivel $n+1$ son las que son distinguibles a nivel n más aquellas tales que existe un $a \in A$ tal que $\{\delta(p,a), \delta(q,a)\}$ es distinguible a nivel n.

Cálculo de parejas de estados indistinguibles

Procedimiento básico

Las parejas de estados distinguibles se pueden calcular introduciendo las parejas de estados distinguibles a nivel 0, y haciendo un ciclo mientras se etiqueten nuevas parejas de estados distinguibles: para toda pareja $\{p, q\}$ no etiquetada como distinguible, y todo $a \in A$, se calcula $\{\delta(p, a), \delta(q, a)\}$. Si este conjunto es una pareja de estados distinguibles, entonces $\{p, q\}$ se hace distinguible.





Si p', q' son distinguibles hay que hacer distinguibles p, q.

Cálculo de parejas de estados indistinguibles

Cada pareja $\{p, q\}$ de estados distintos tiene asociada una variable booleana (ser distinguibles) inicialmente falsa. Cuando la variable booleana es verdadera diremos que la pareja está marcada.

1. Eliminar estados inaccesibles.
2. Para cada pareja de estados $\{q_i, q_j\}$
    3. Si uno de ellos es final y el otro no, hacer la variable booleana asociada igual a true.
4. Mientras haya cambios en las variables booleanas
    5. Para cada pareja de estados $\{q_i, q_j\}$
        6. Para cada símbolo a del alfabeto de entrada
            7. Calcular los estados q_k y q_l a los que evoluciona el autómata desde q_i y q_j leyendo a
            8. Si $q_k \neq q_l$ entonces
                9. Si la pareja $\{q_k, q_l\}$ está marcada entonces se marca $\{q_i, q_j\}$

Método alternativo

Pero es más efectivo calcular las parejas de nivel 0 y para toda pareja $\{p, q\}$ no etiquetada como distinguible, y todo $a \in A$, se calcula $\{\delta(p, a), \delta(q, a)\}$. Si esta es una pareja distinguible se pone $\{p, q\}$ como distinguible, y si no lo es se añade $\{p, q\}$ a una lista de parejas asociados a la pareja $\{\delta(p, a), \delta(q, a)\}$, para poner $\{p, q\}$ como distinguible, si alguna vez $\{\delta(p, a), \delta(q, a)\}$ resulta ser una pareja distinguible.

Identificación de estados indistinguibles

Pareja de estados $\rightarrow$ variable booleana (indistinguile) inicialmente false + lista de parejas de estados

Es inmediato demostrar por inducción que este procedimiento encuentra todas las parejas de estados indistiguibles.

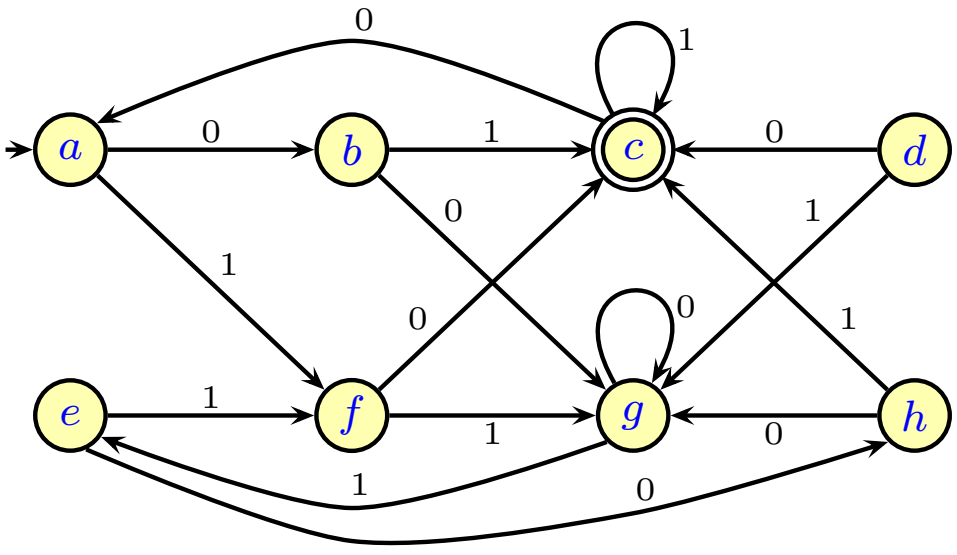
Identificación de estados indistinguibles

Pareja de estados → variable booleana (indistinguile) inicialmente false + lista de parejas de estados

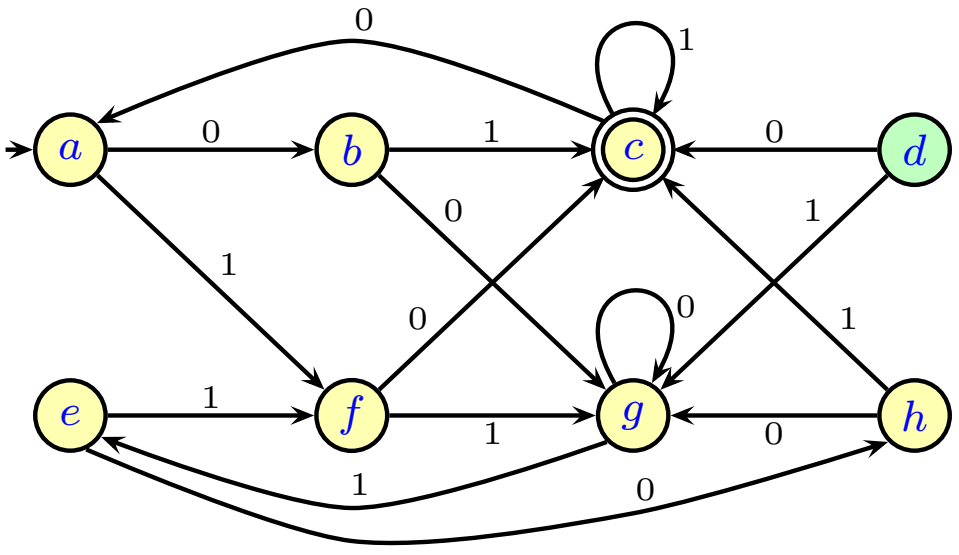
1. Eliminar estados inaccesibles.
2. Para cada pareja de estados accesibles $\{q_i, q_j\}$
   3. Si uno de ellos es final y el otro no, hacer la variable booleana asociada igual a true.
4. Para cada pareja de estados accesibles $\{q_i, q_j\}$
   5. Para cada símbolo a del alfabeto de entrada
      6. Calcular los estados q_k y q_l a los que evoluciona el autómata desde q_i y q_j leyendo a
      7. Si $q_k \neq q_l$ entonces
         8. Si la $\{q_k, q_l\}$ está marcada entonces se marca la pareja $\{q_i, q_j\}$ y recursivamente todas las parejas en la lista asociada.
      9. Si la pareja $\{q_k, q_l\}$ no está marcada, se añade la pareja $\{q_i, q_j\}$ a la lista asociada a la pareja $\{q_k, q_l\}$

Es inmediato demostrar por inducción que este procedimiento encuentra todas las parejas de estados indistiguibles.

Ejemplo

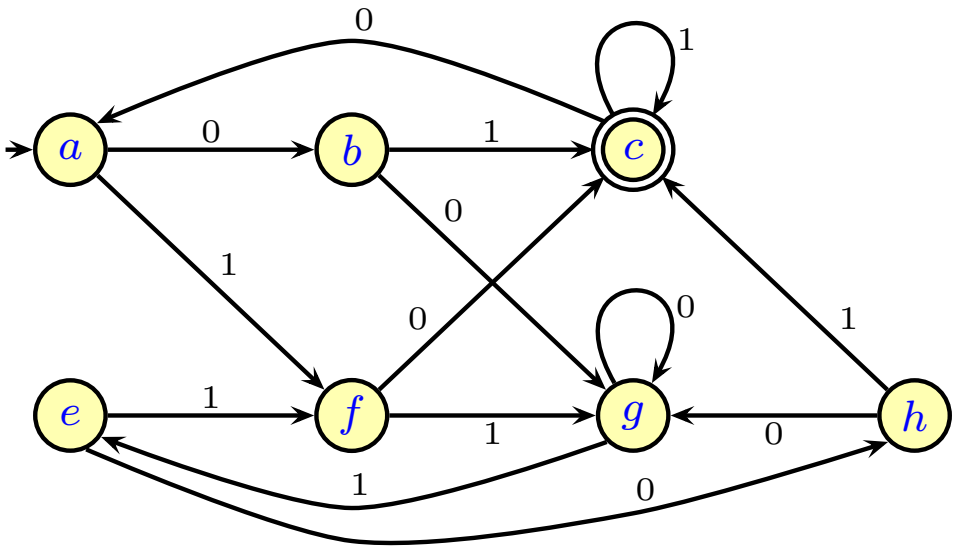


Ejemplo



El estado d es inaccesible

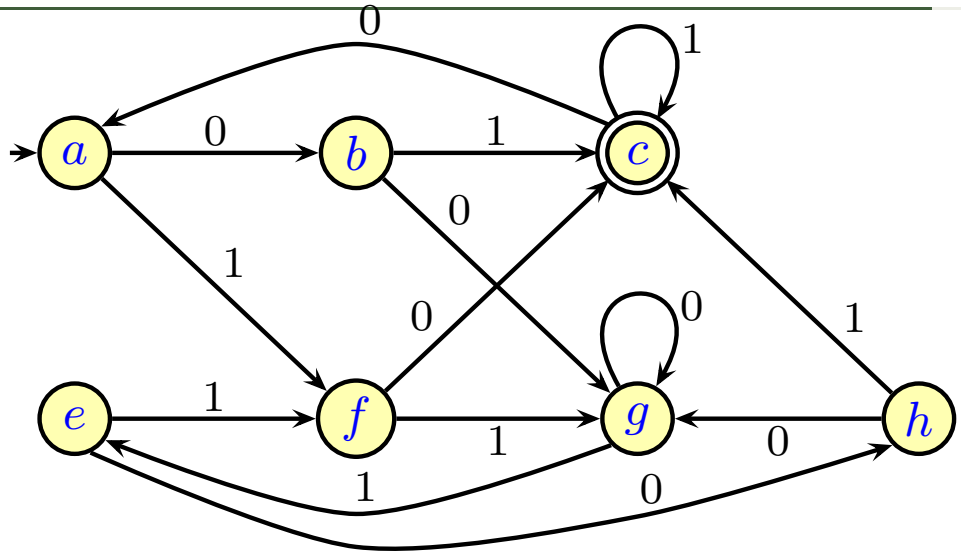
Ejemplo



El estado d es inaccesible

y se elimina

Ejemplo



a b c e f g

Ejemplo

b

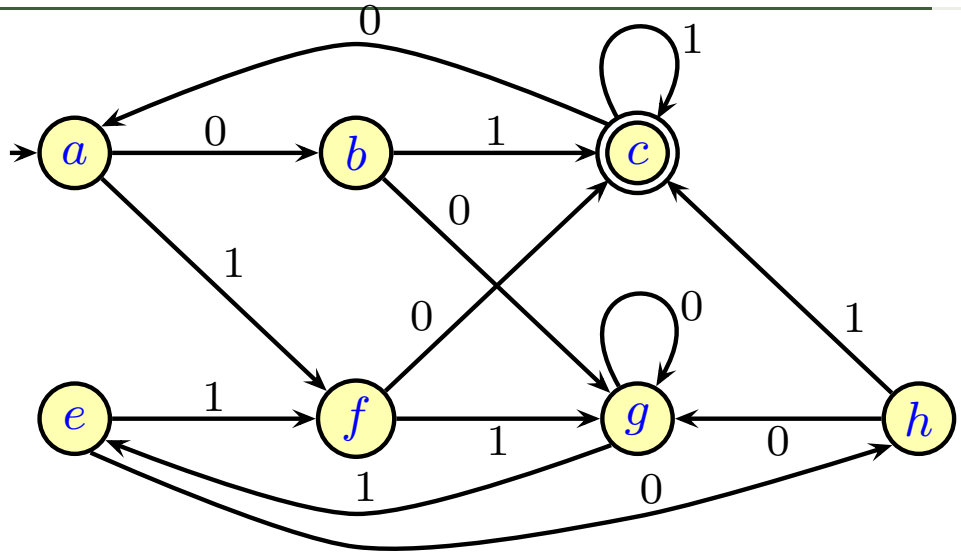
c

e

f

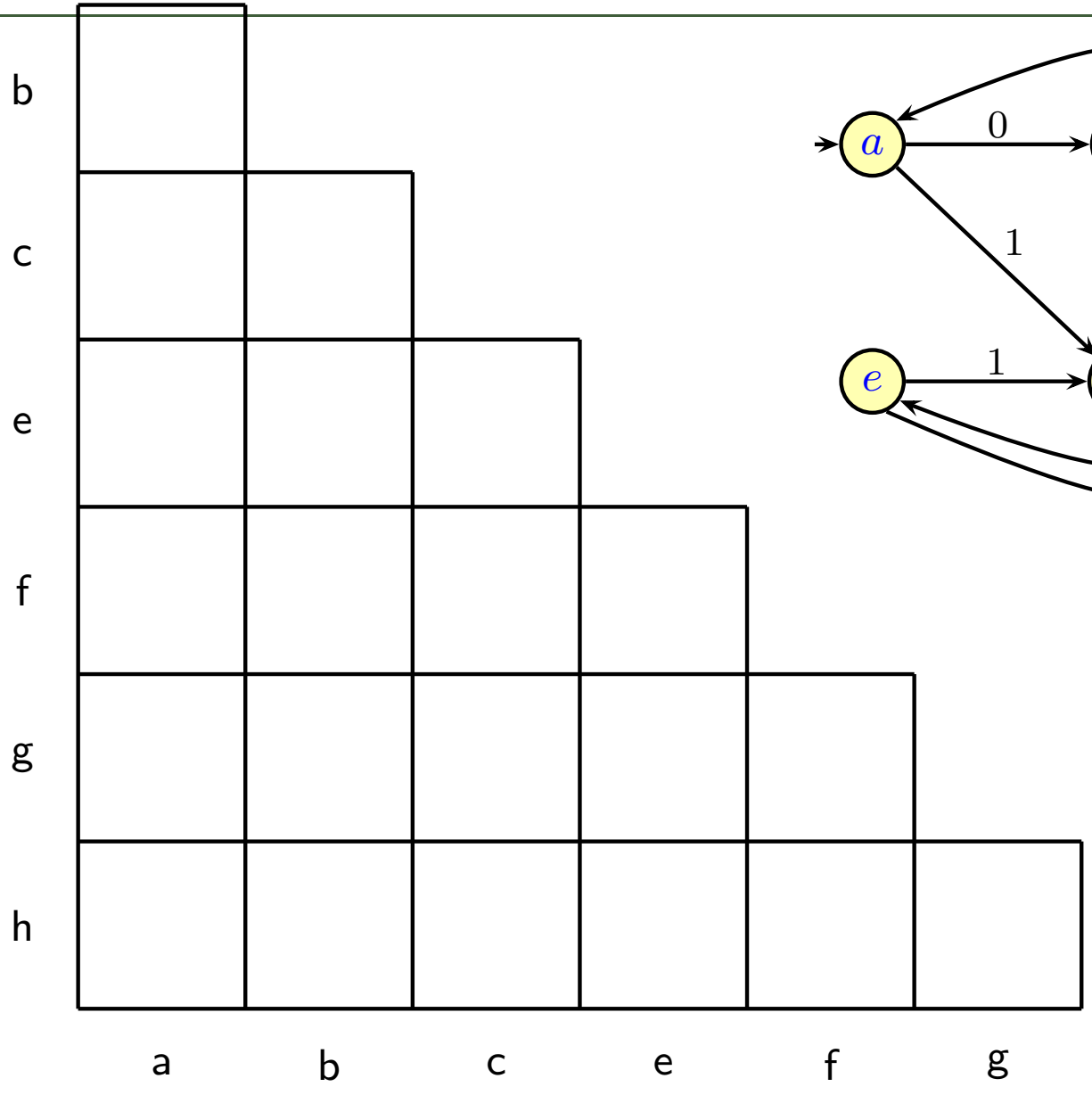
g

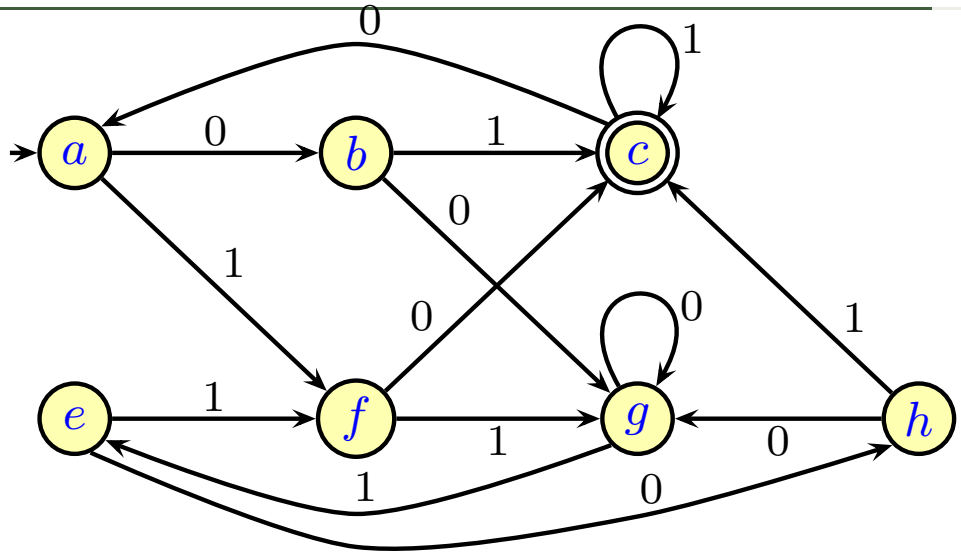
h



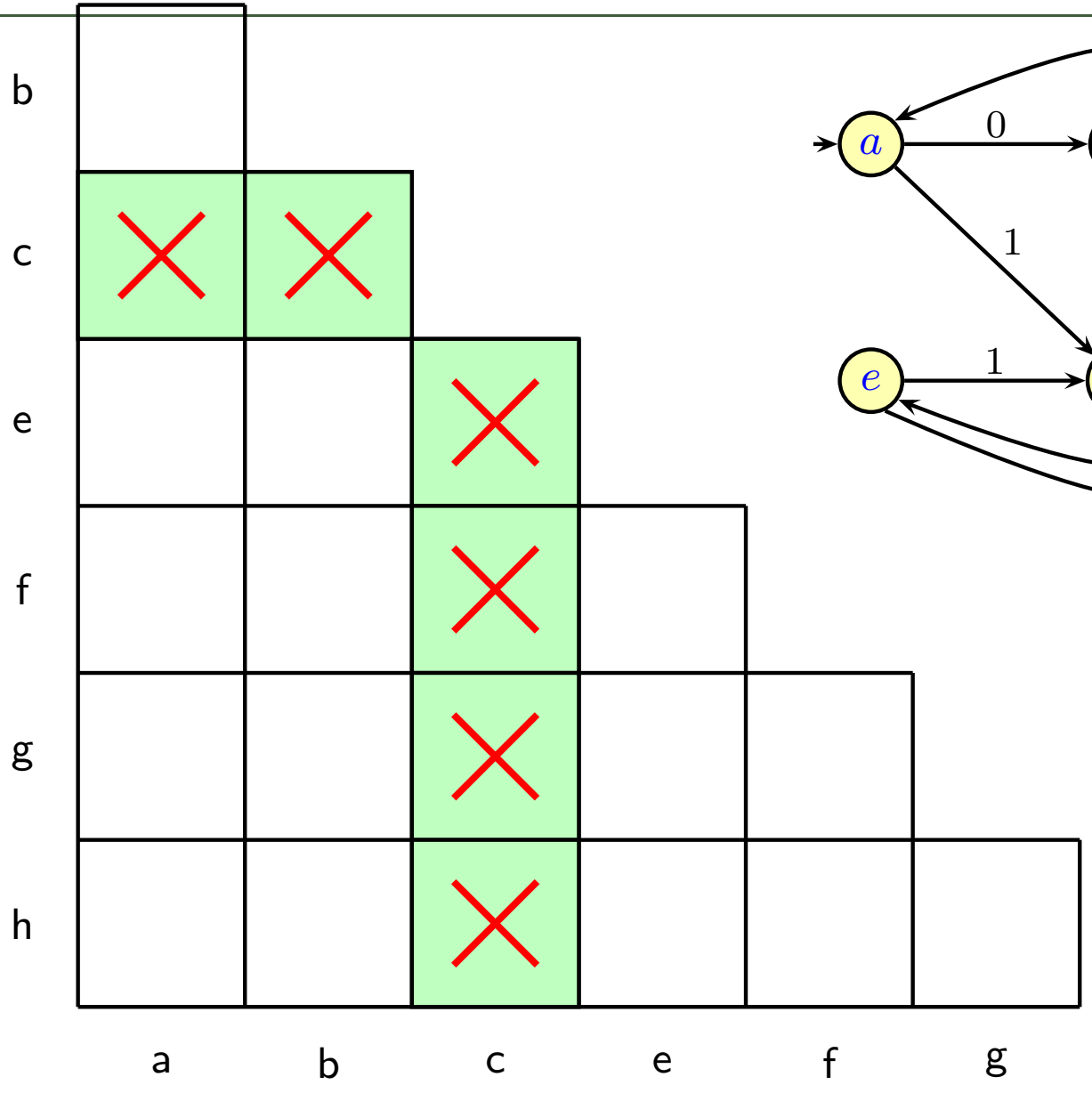
a b c e f g

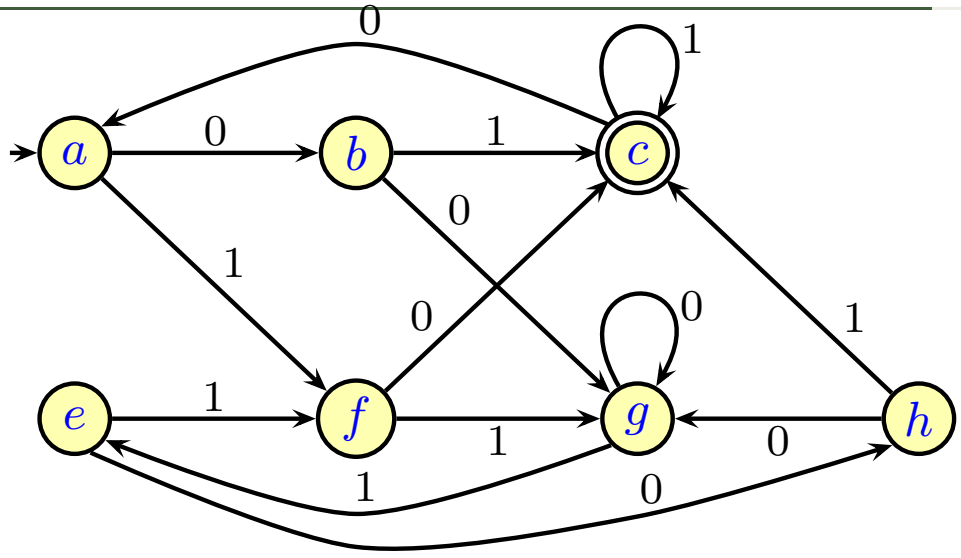
Ejemplo



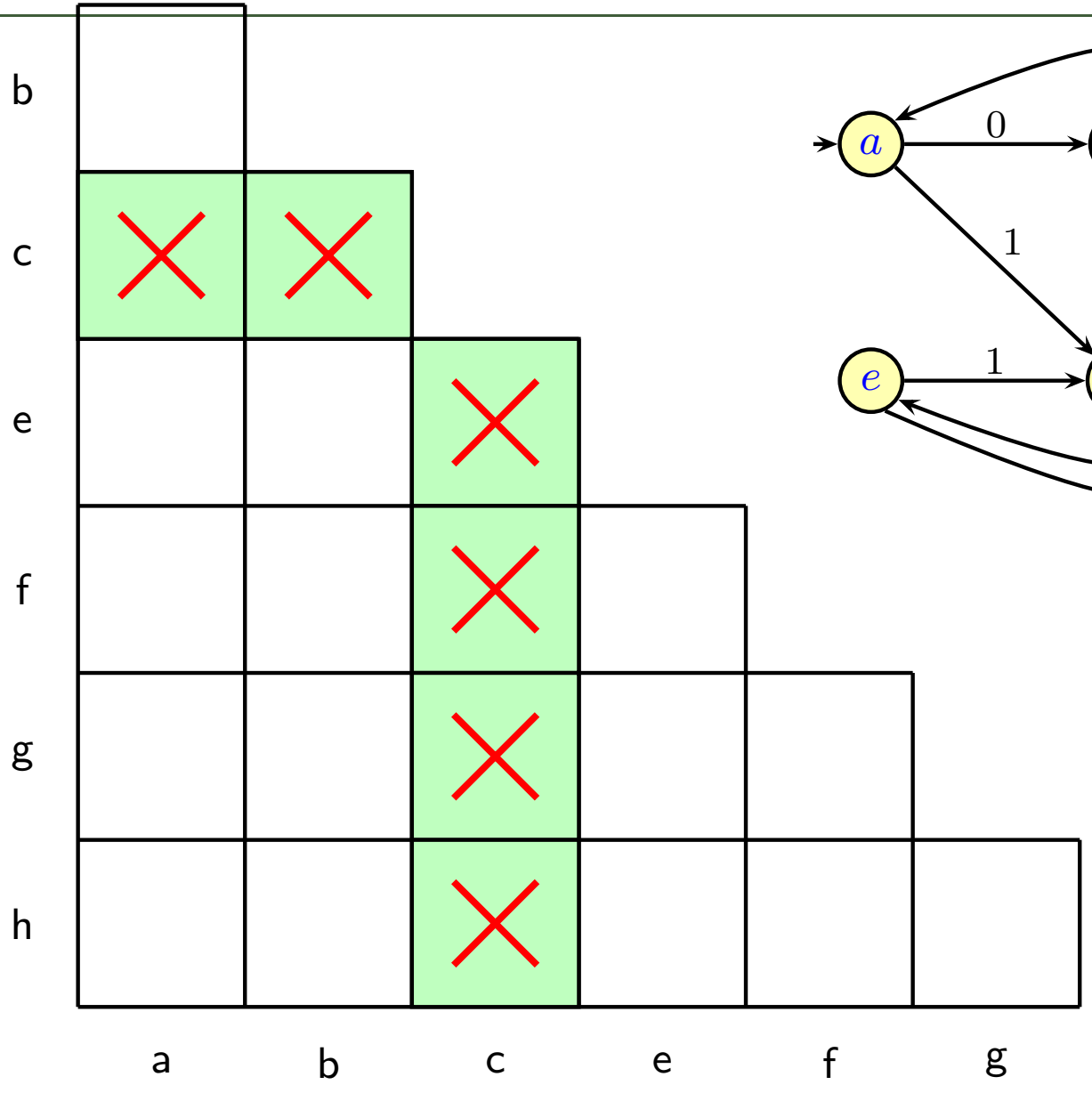


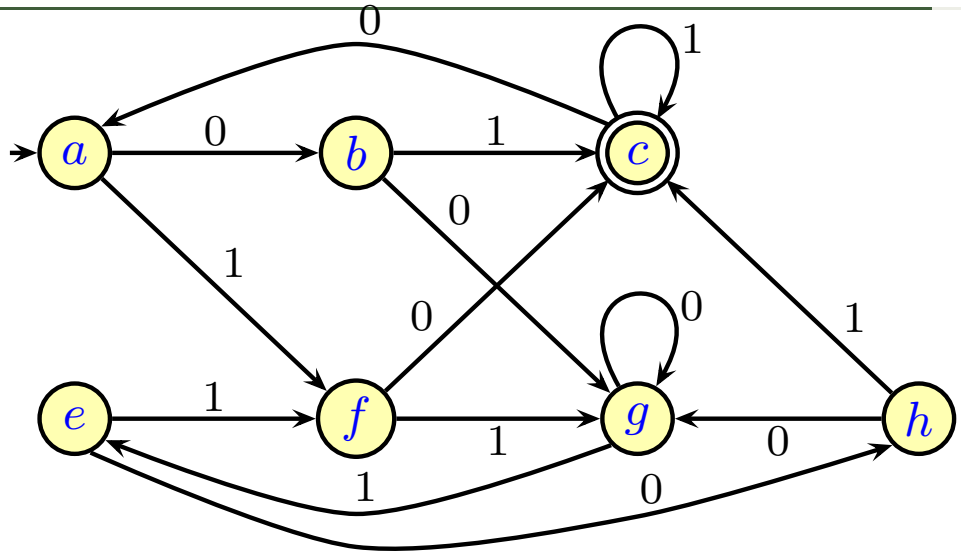
Ejemplo



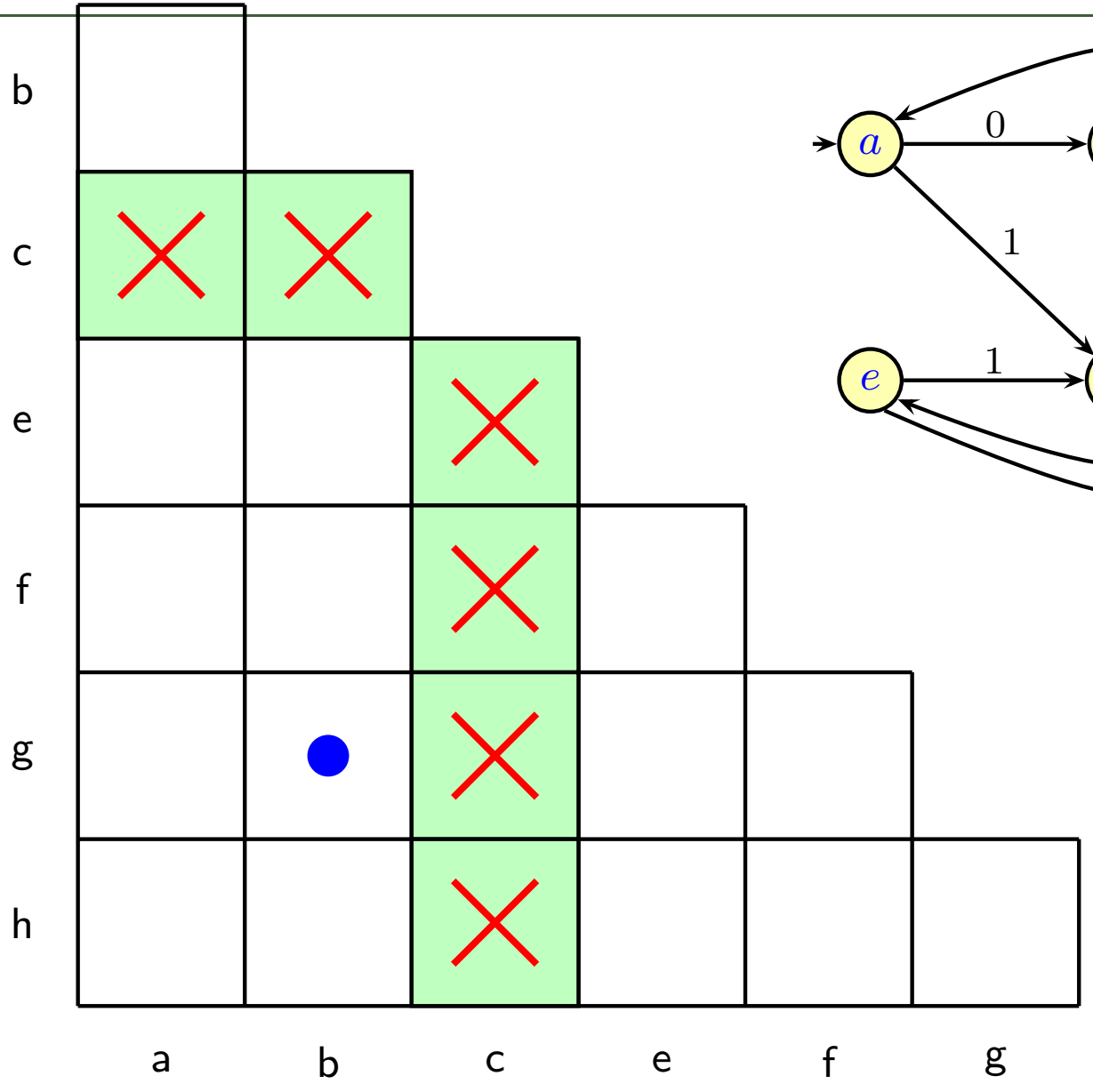


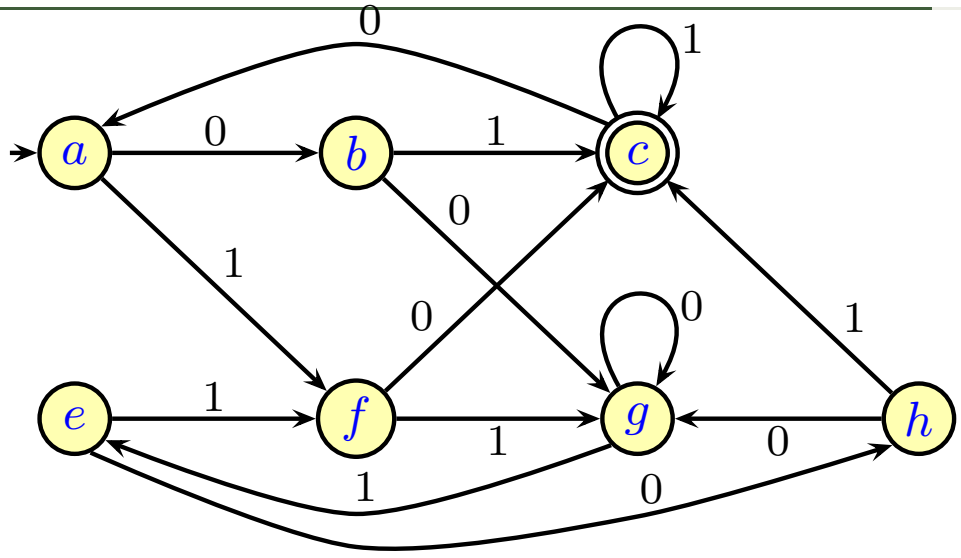
Ejemplo





Ejemplo

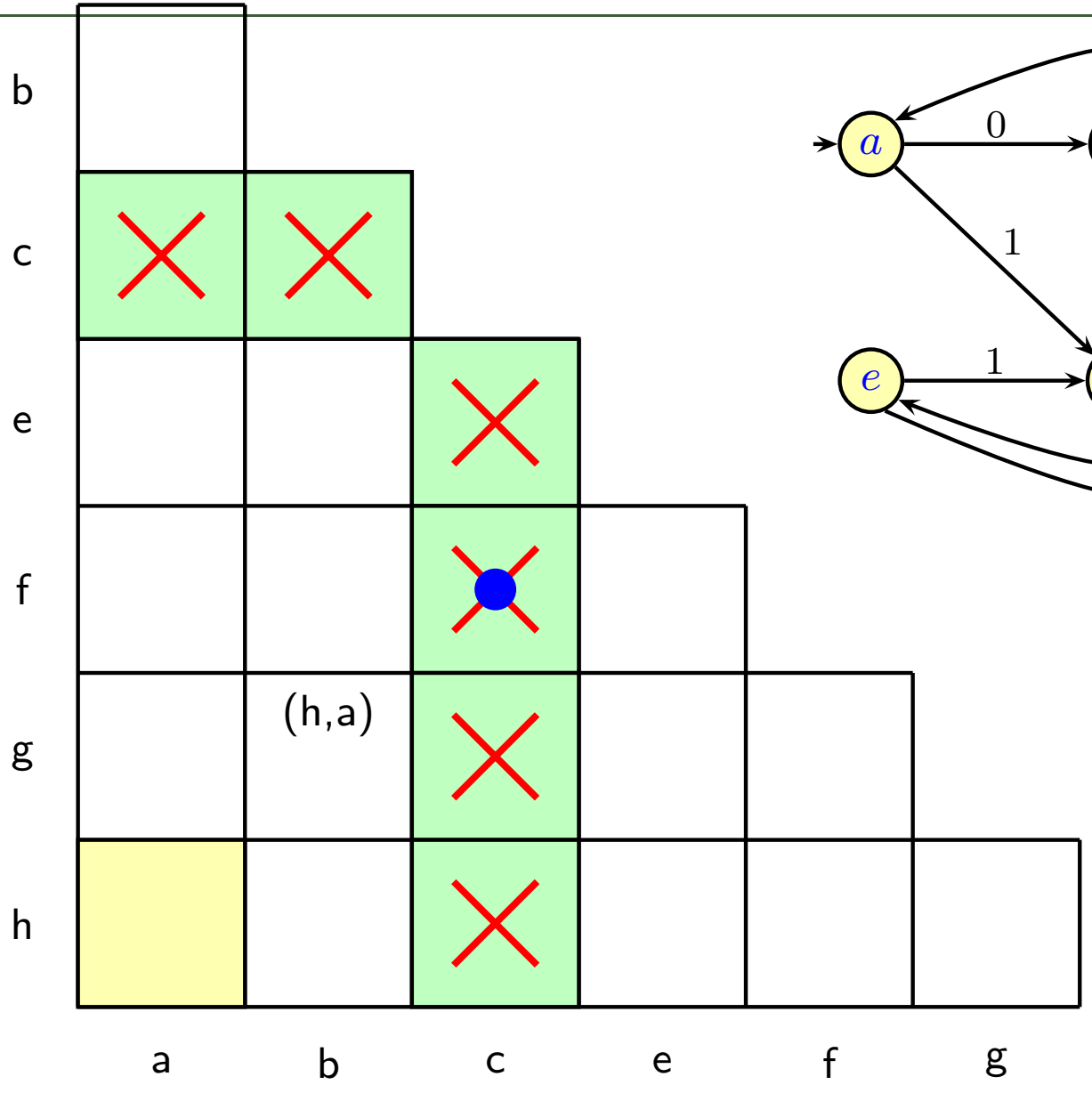


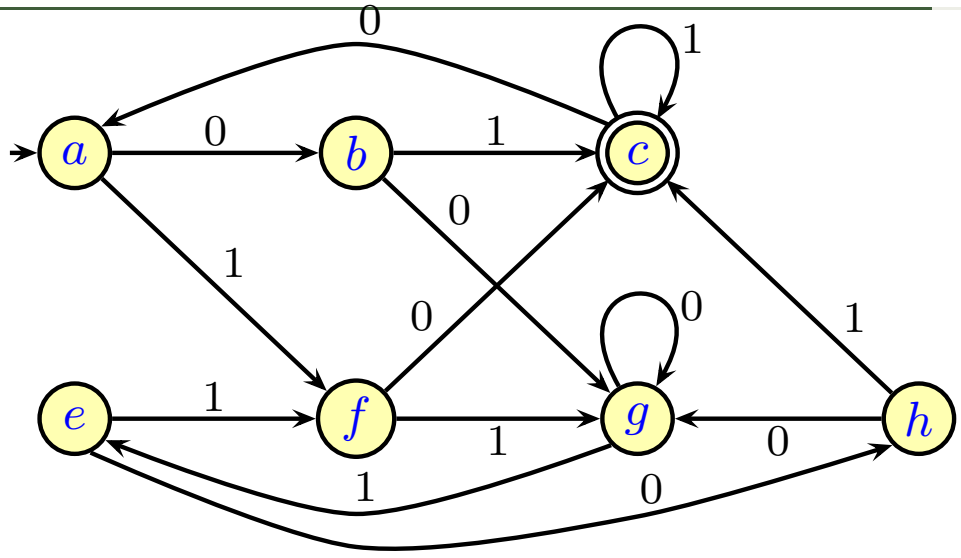


g

b

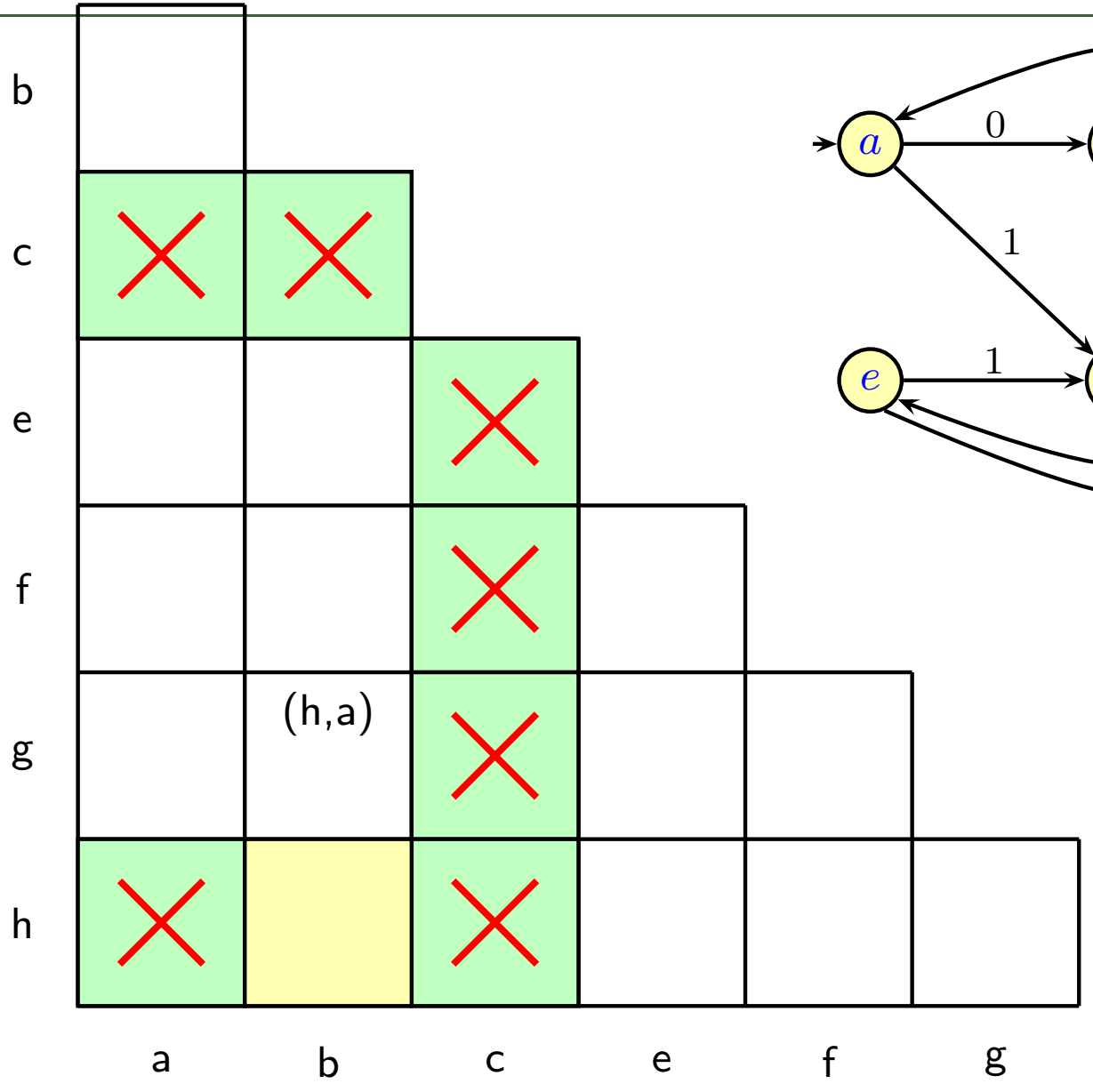
Ejemplo

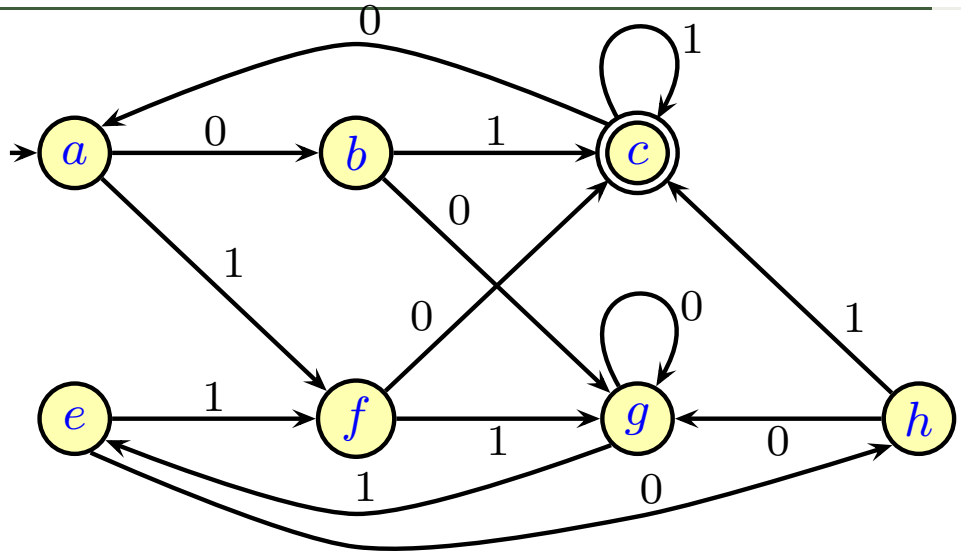




	0	1
h		c
a		f

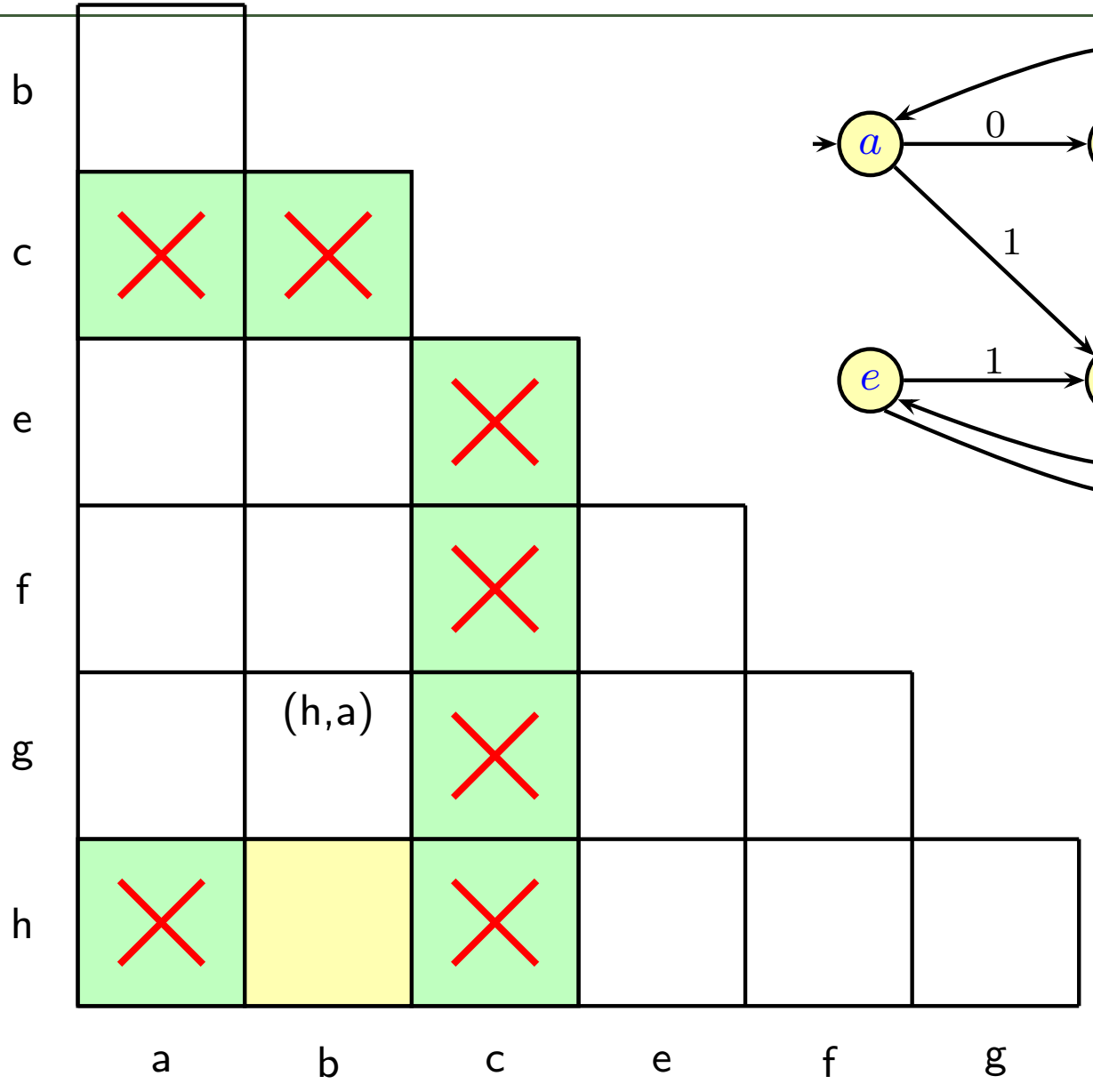
Ejemplo

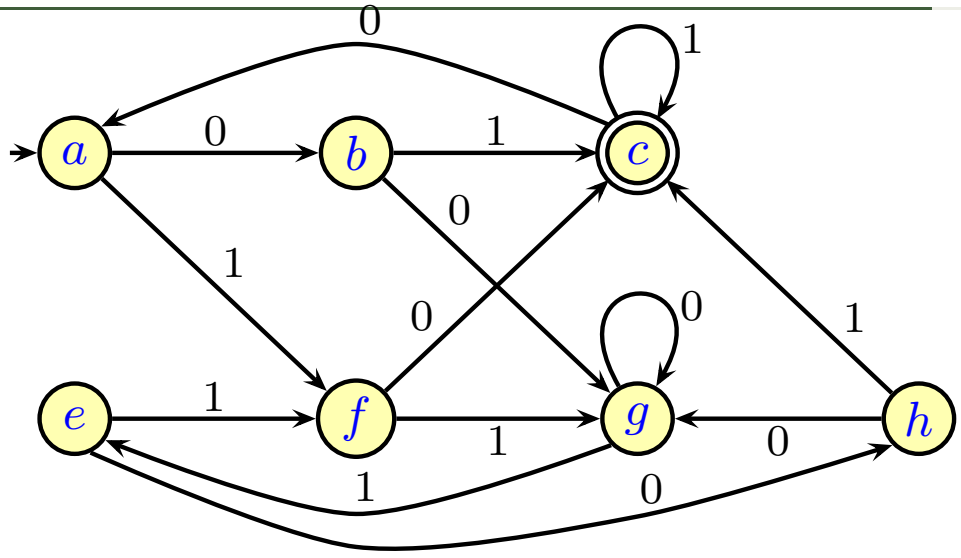




	0	1
h	g	
b	g	

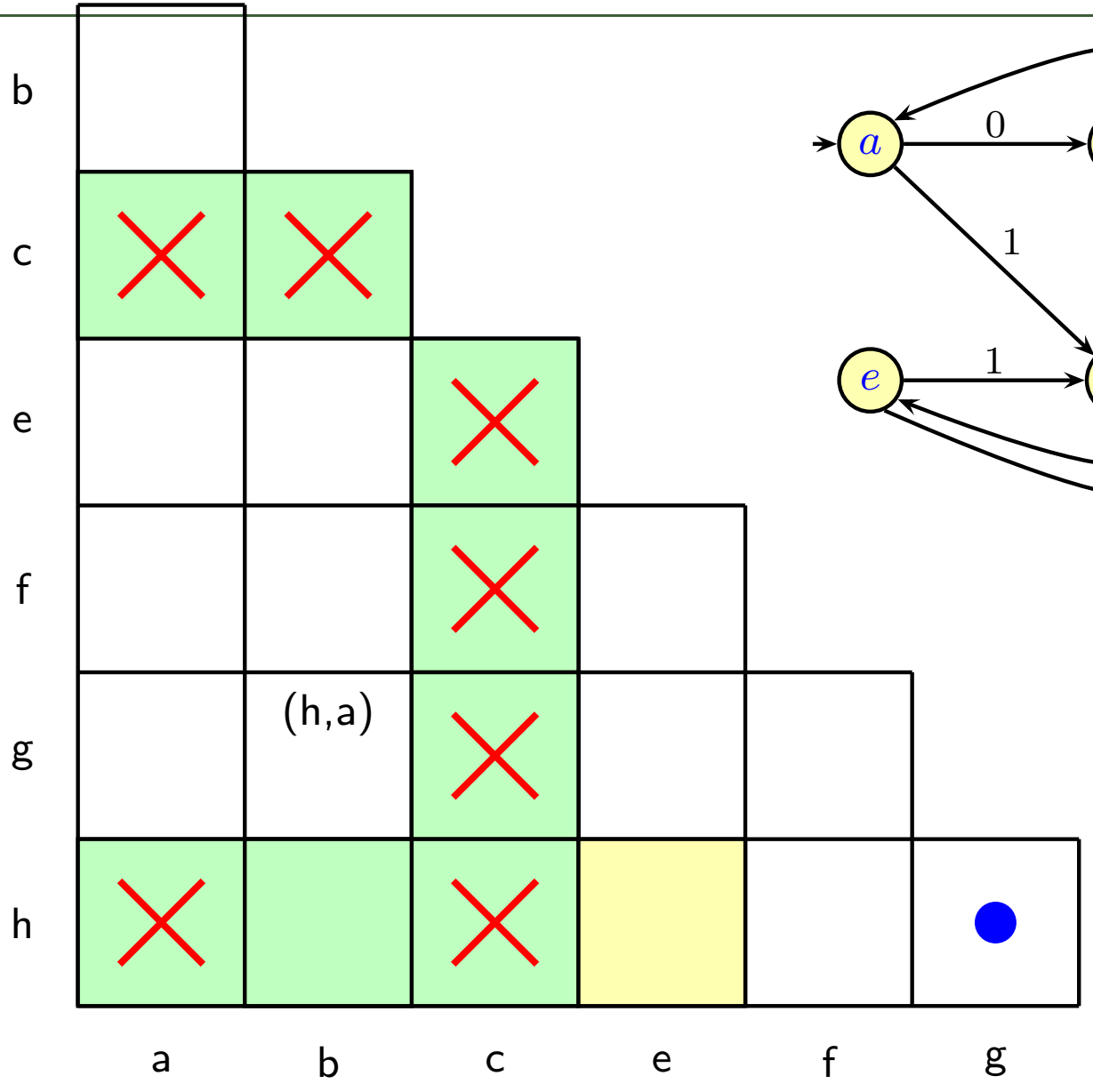
Ejemplo

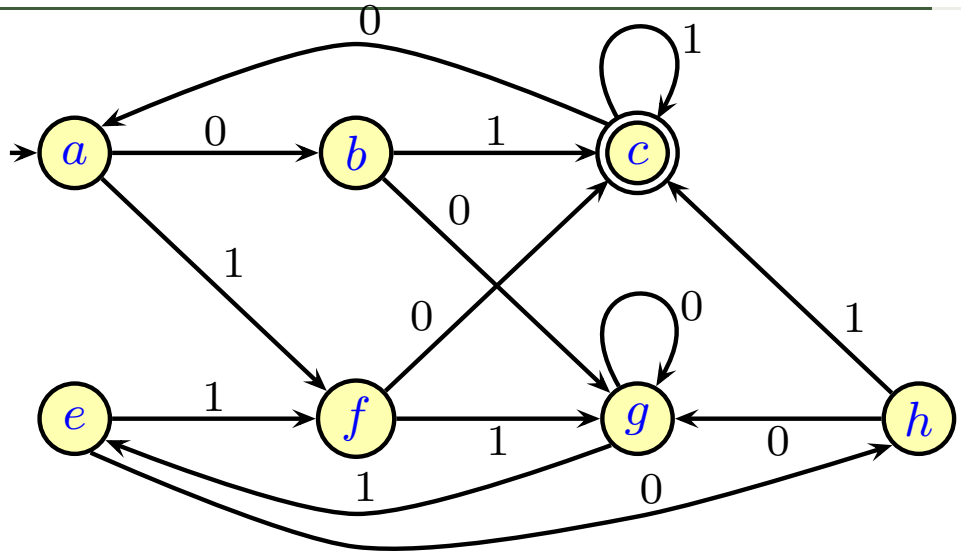




	0	1
h		c
b		c

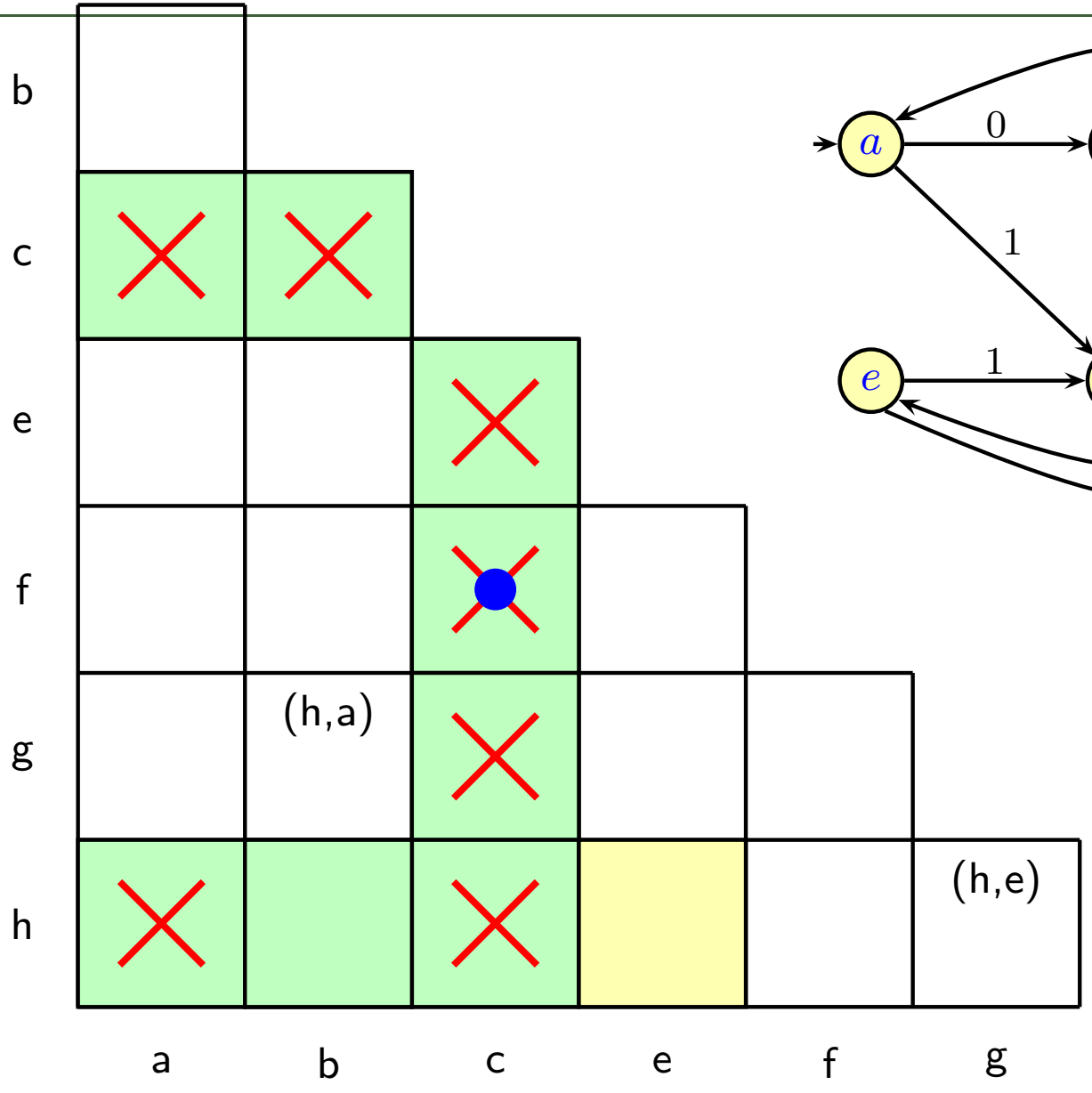
Ejemplo

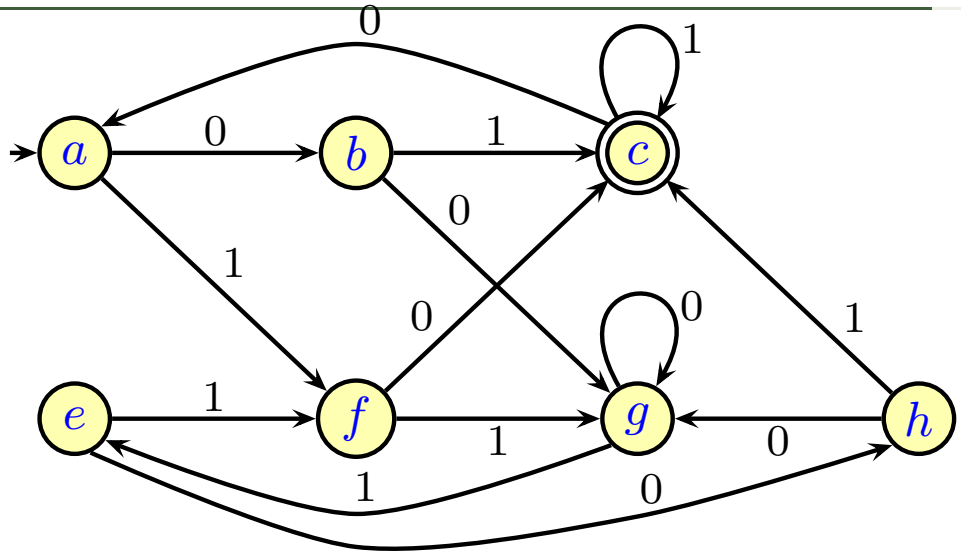




	0	1
h	g	
e	h	

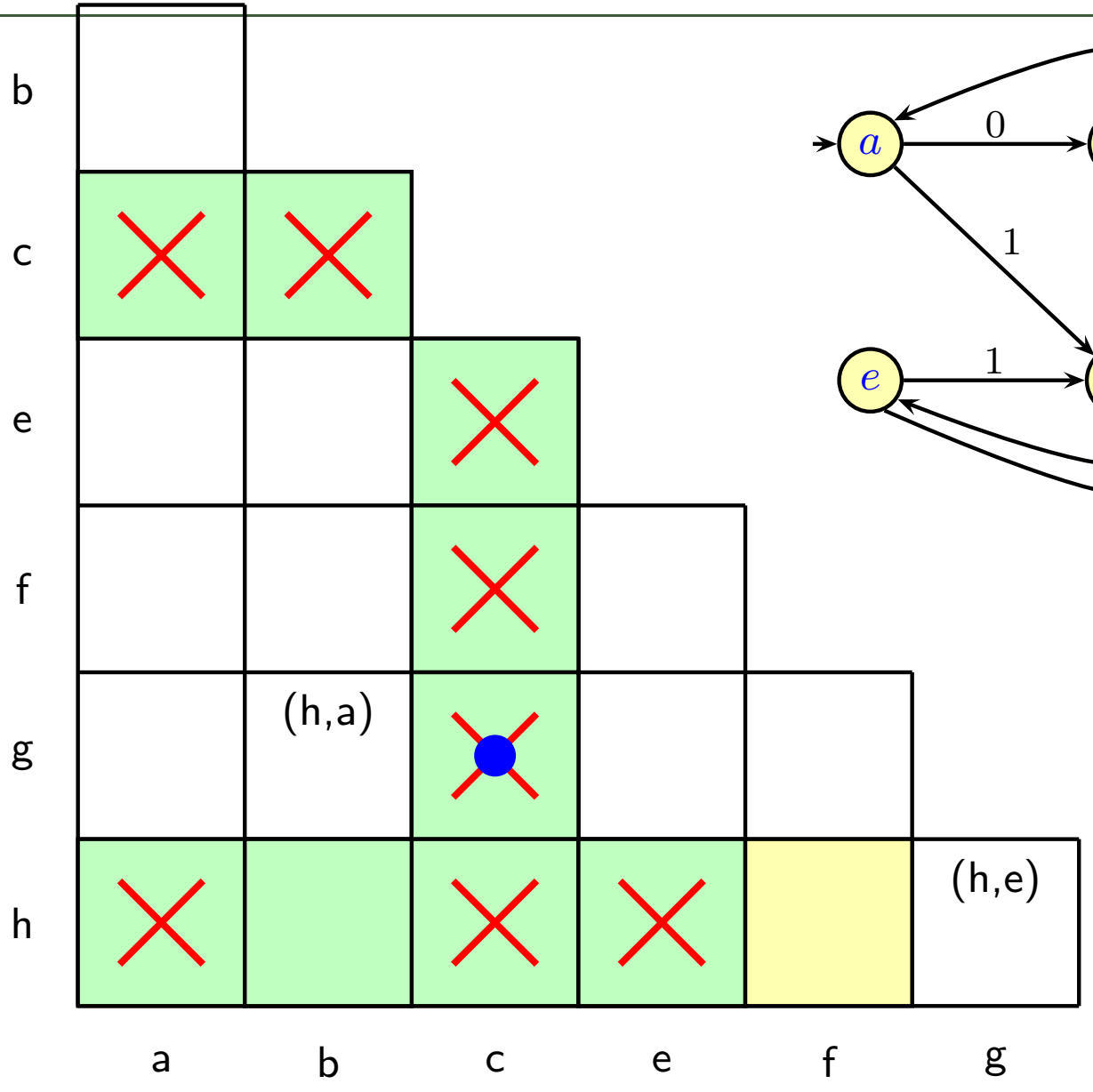
Ejemplo

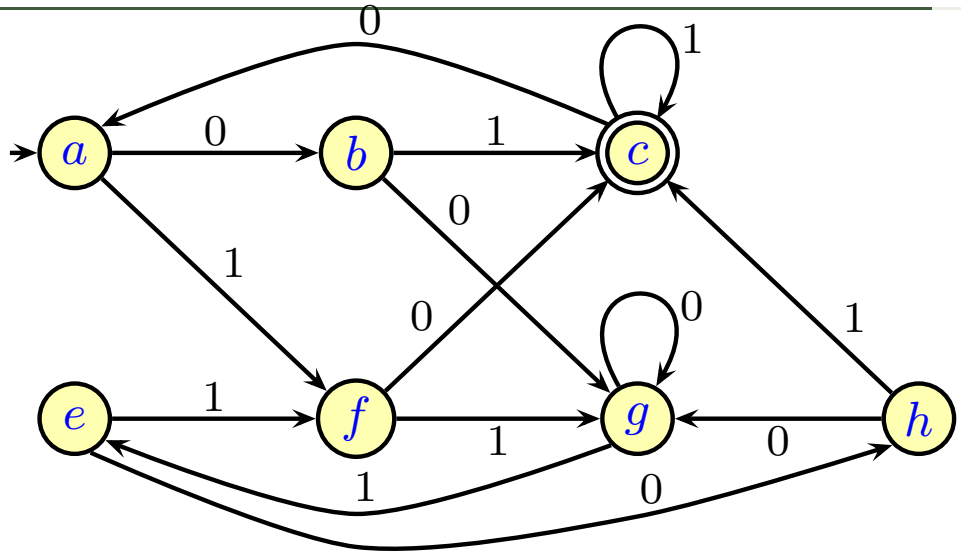




	0	1
h		c
e		f

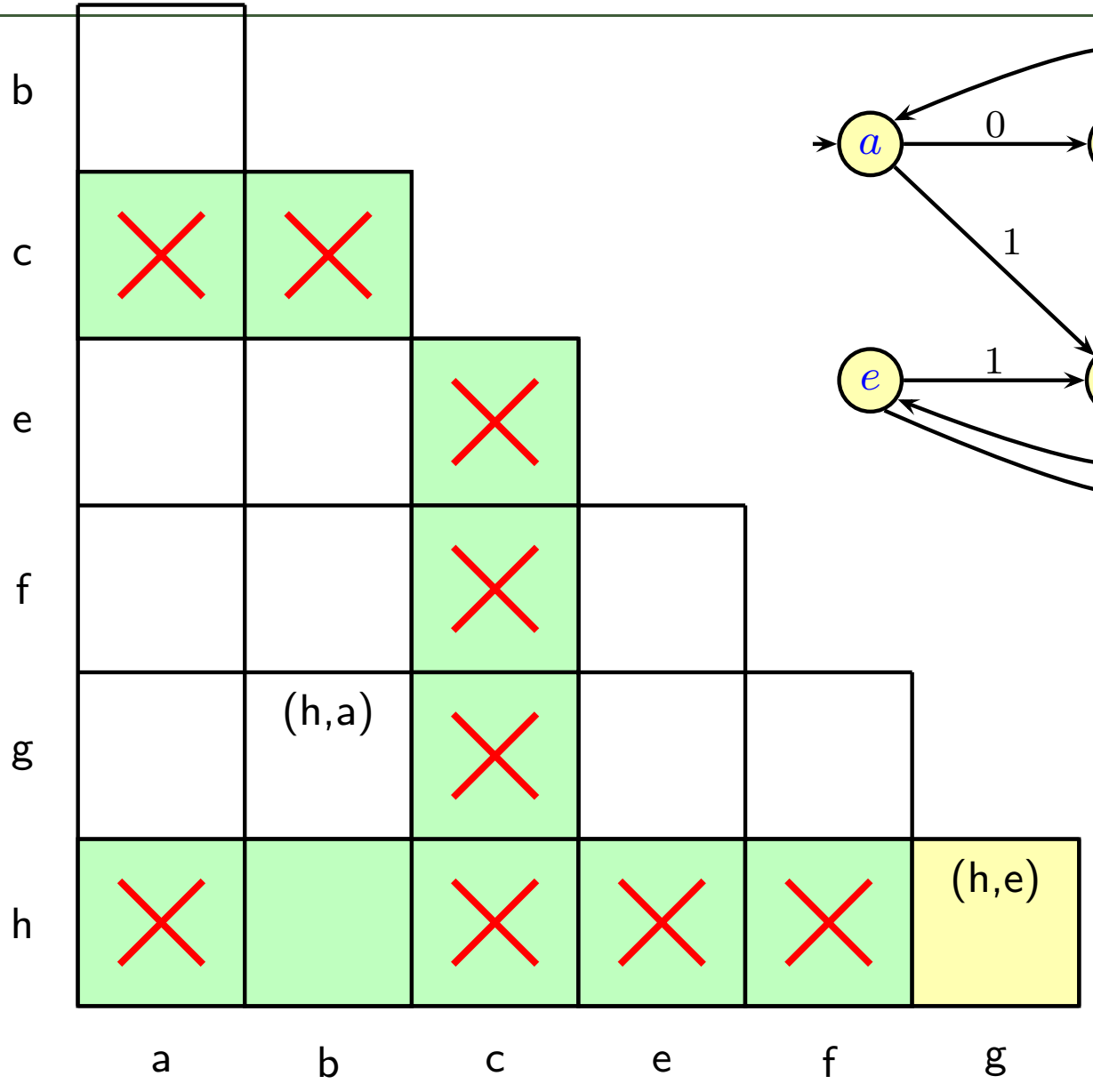
Ejemplo

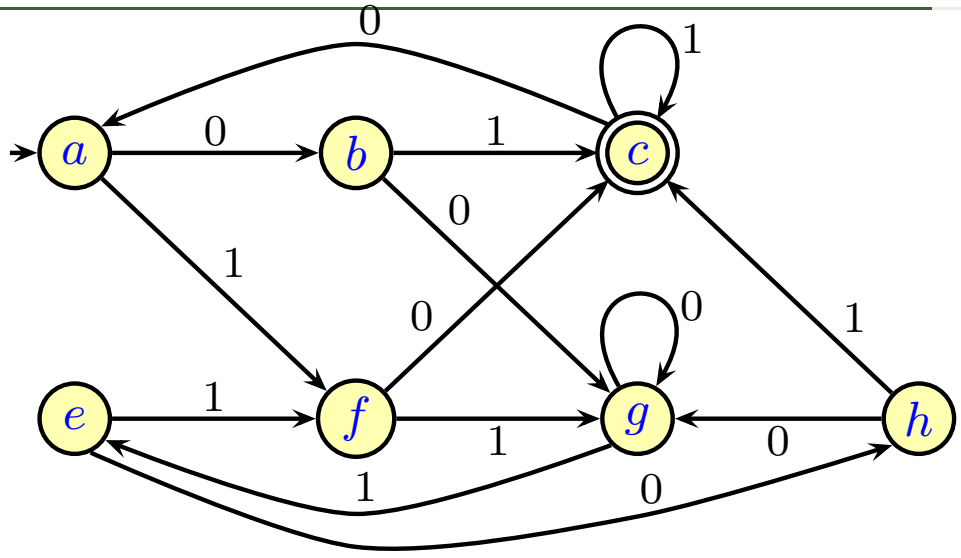




	0	1
h	g	
f	c	

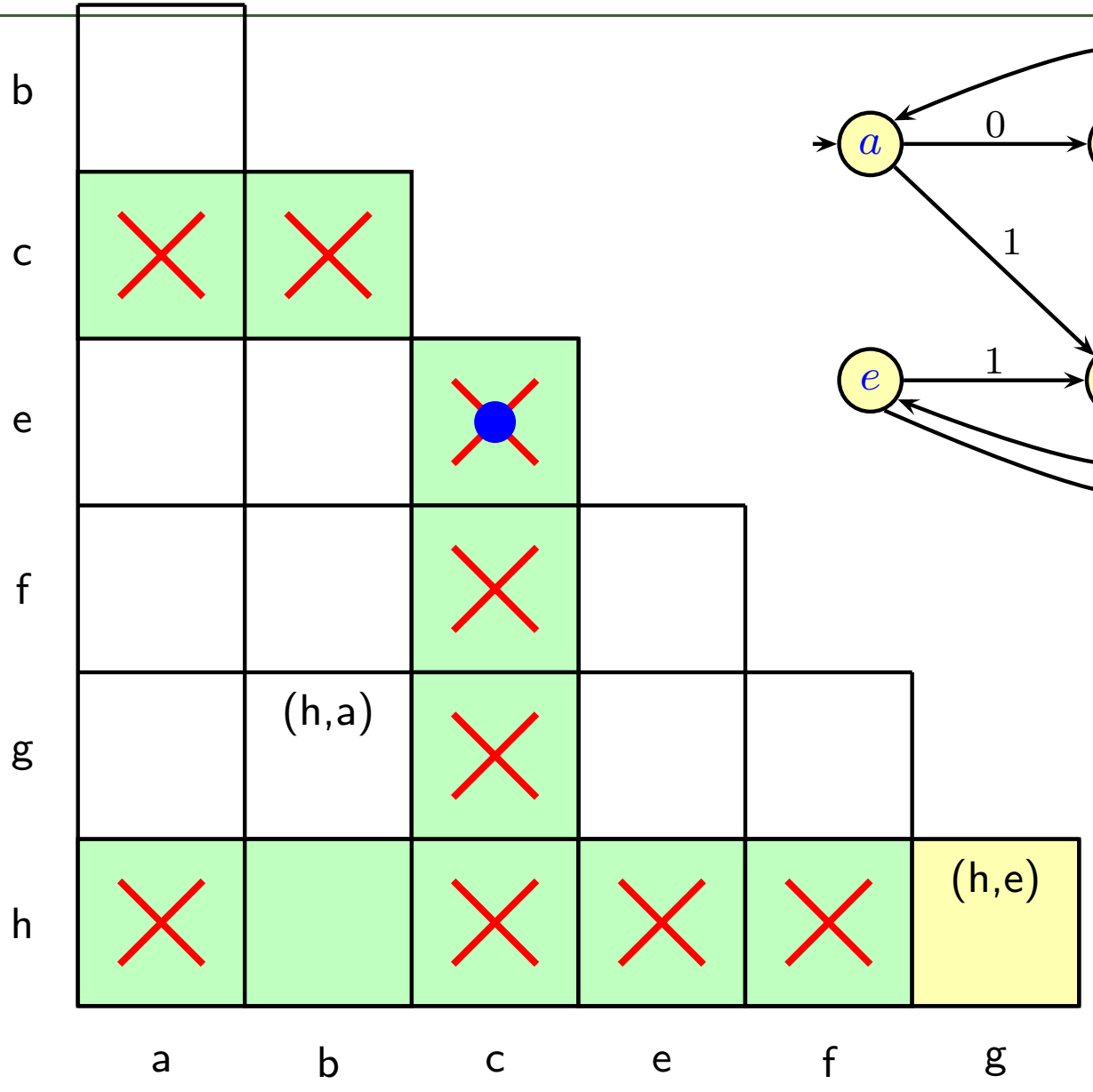
Ejemplo

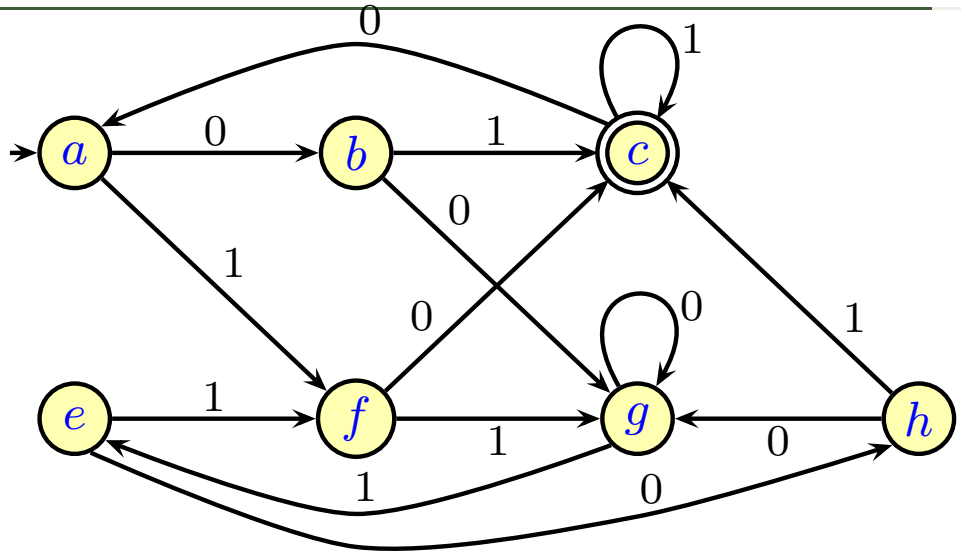




	0	1
h	g	
g	g	

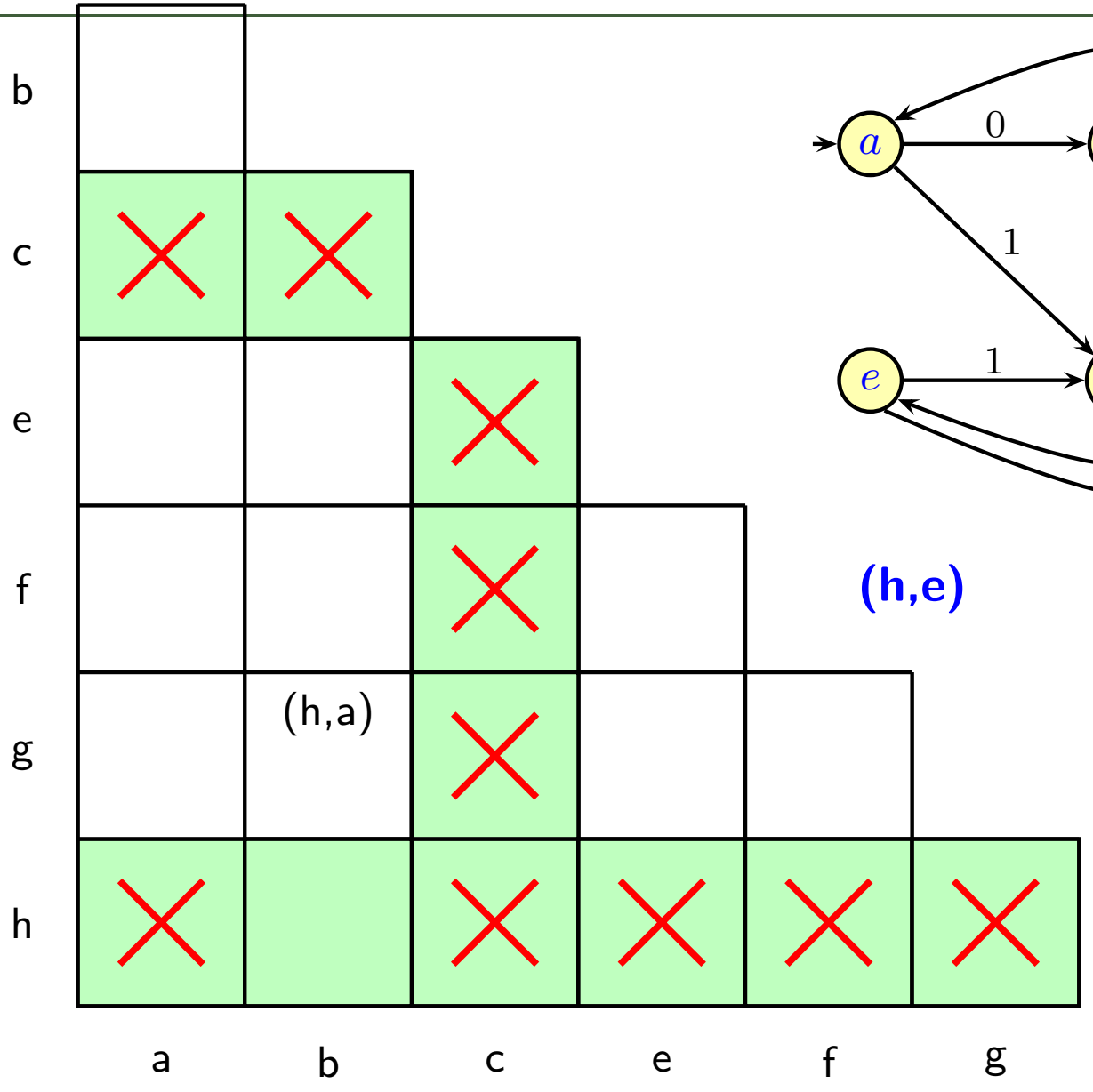
Ejemplo

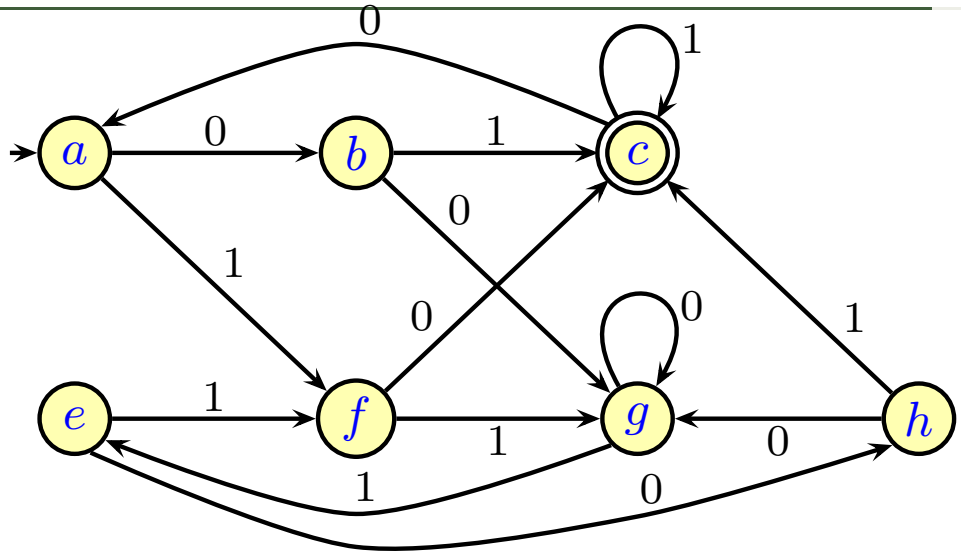




	0	1
h		c
g		e

Ejemplo

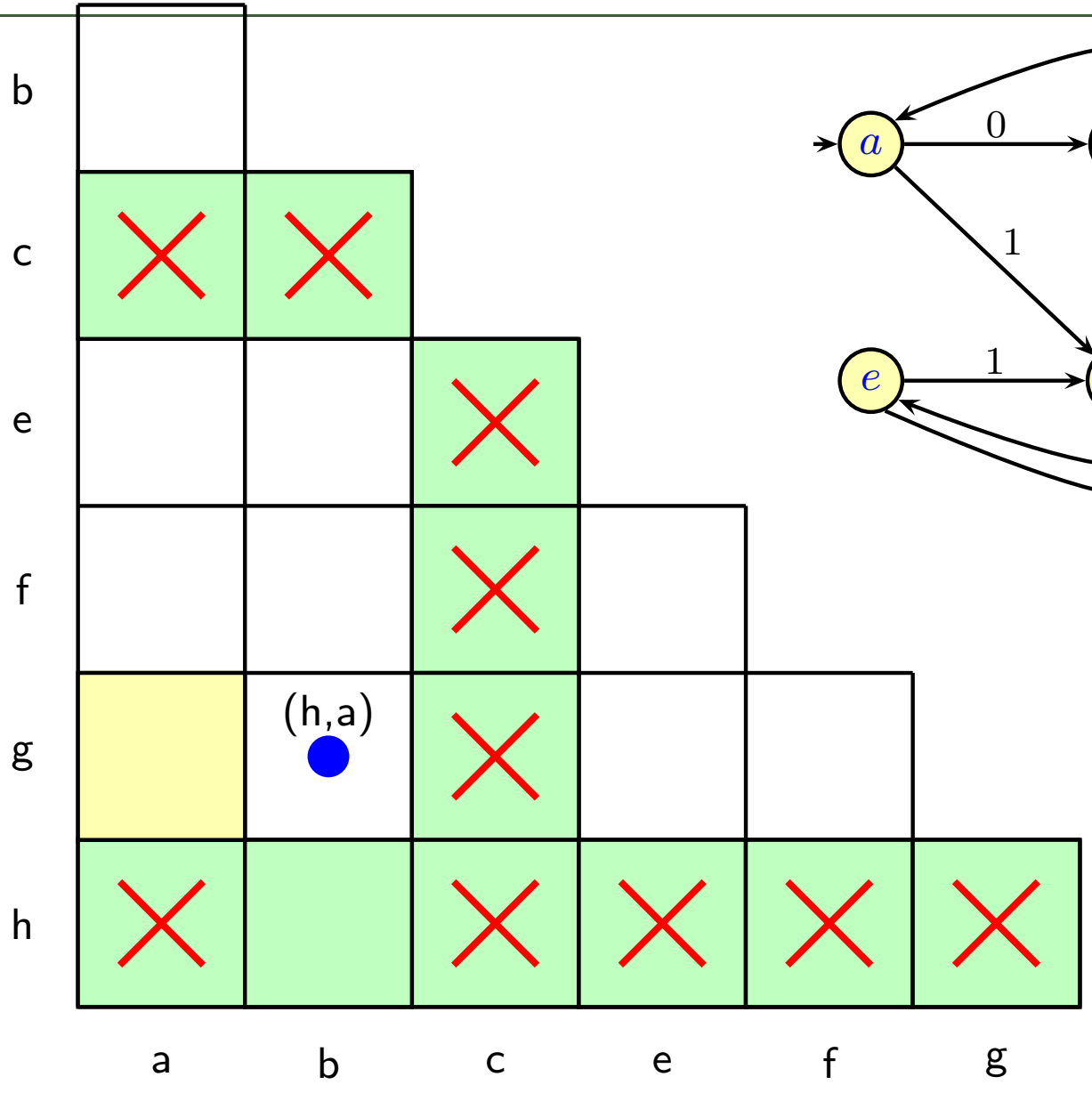


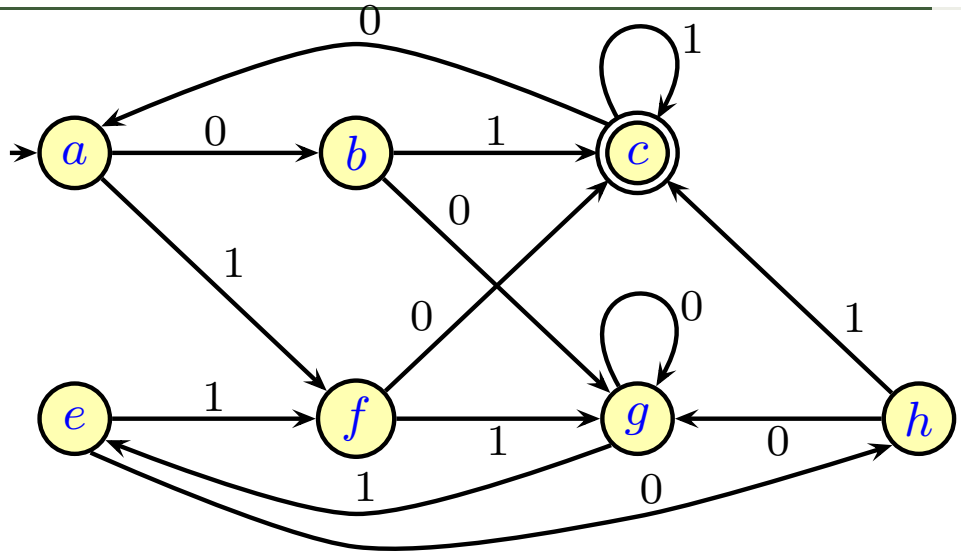


(h,e)

	0	1

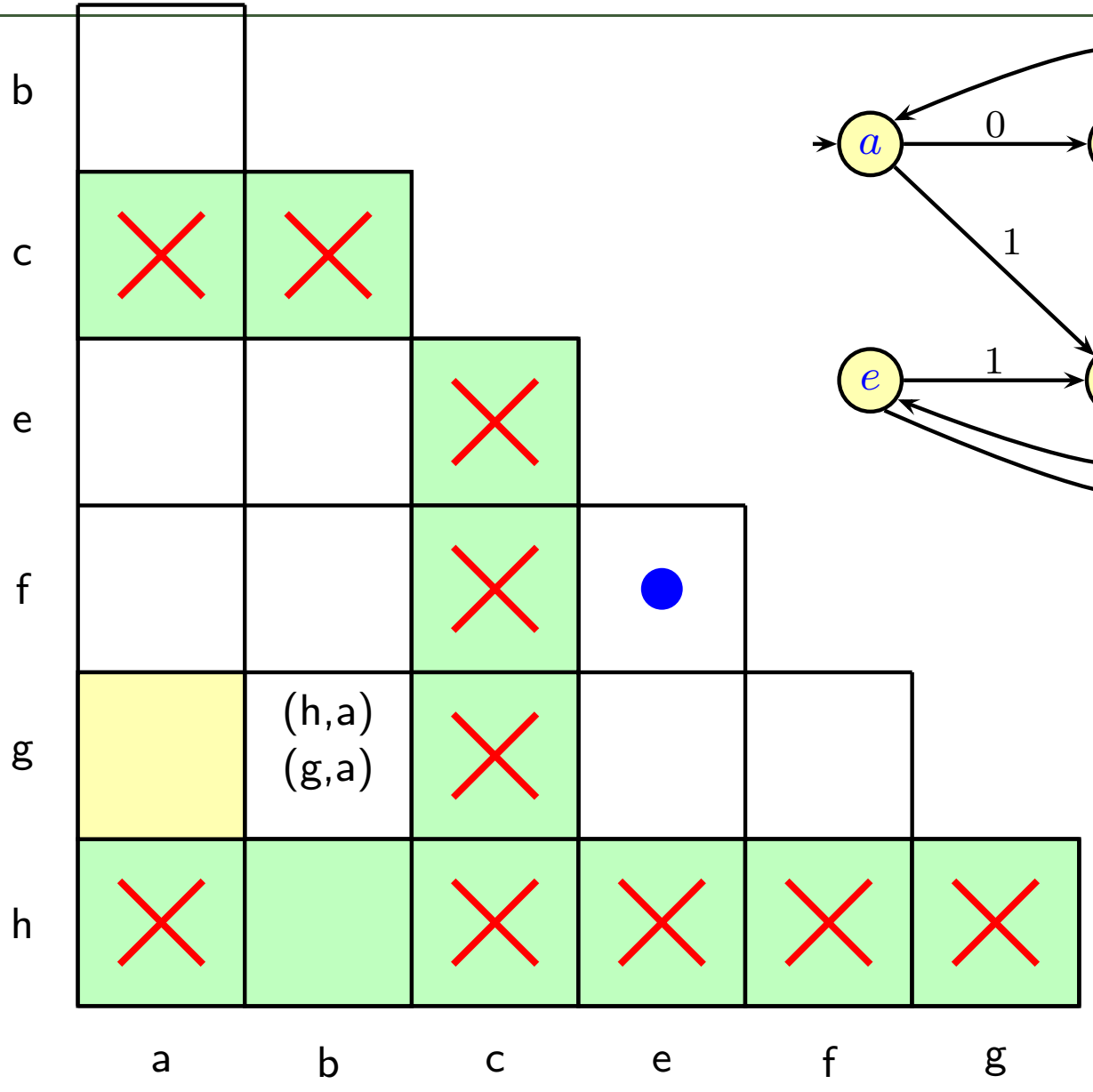
Ejemplo

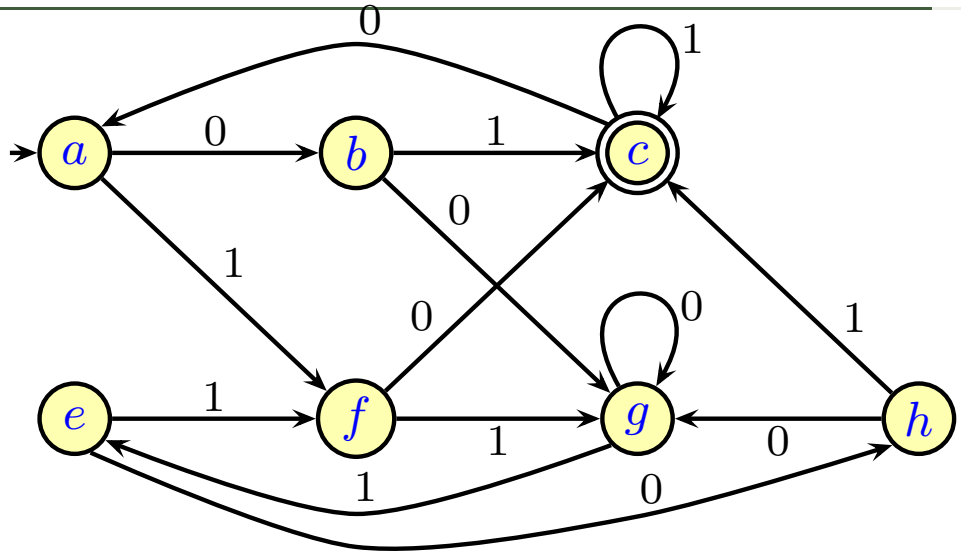




	0	1
g	g	
a	b	

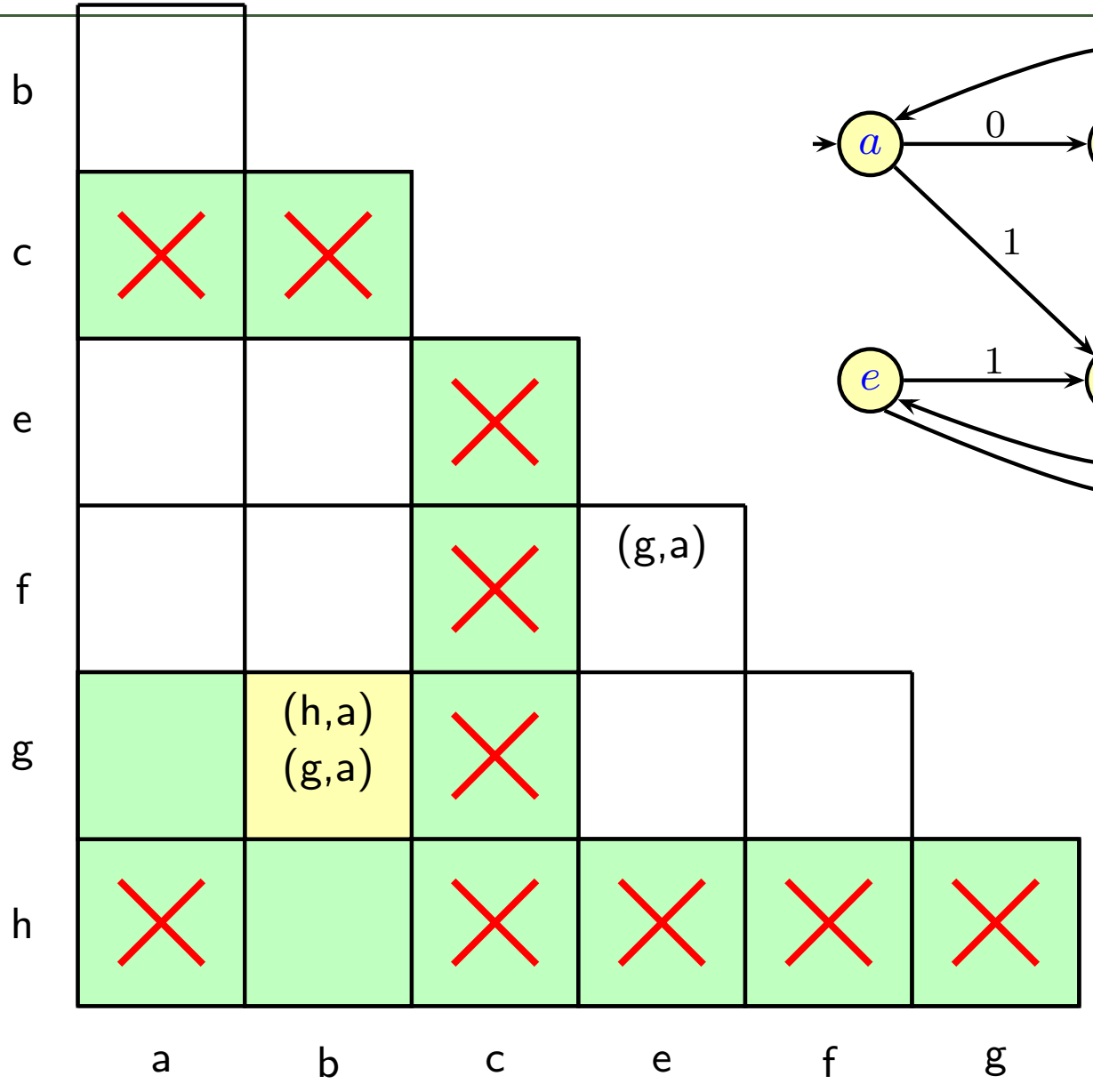
Ejemplo

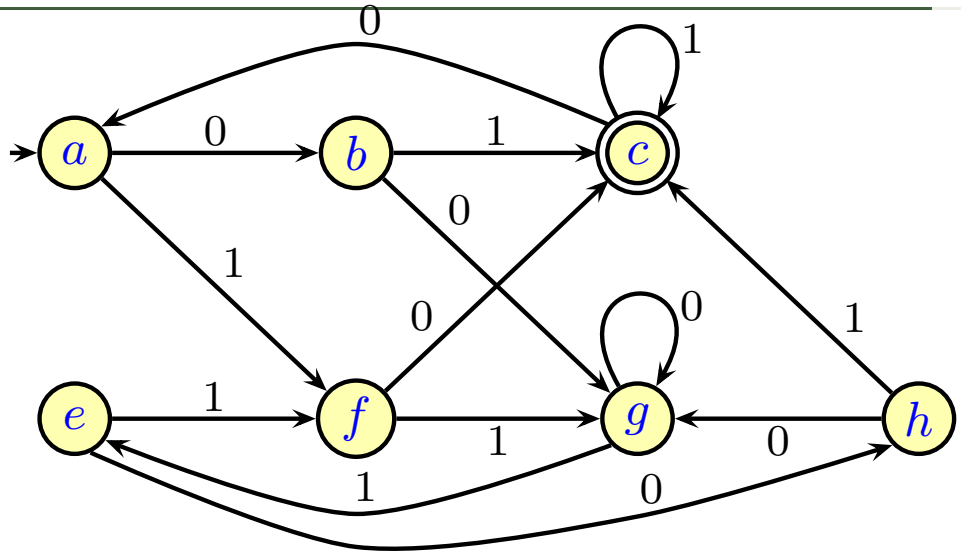




	0	1
g		e
a		f

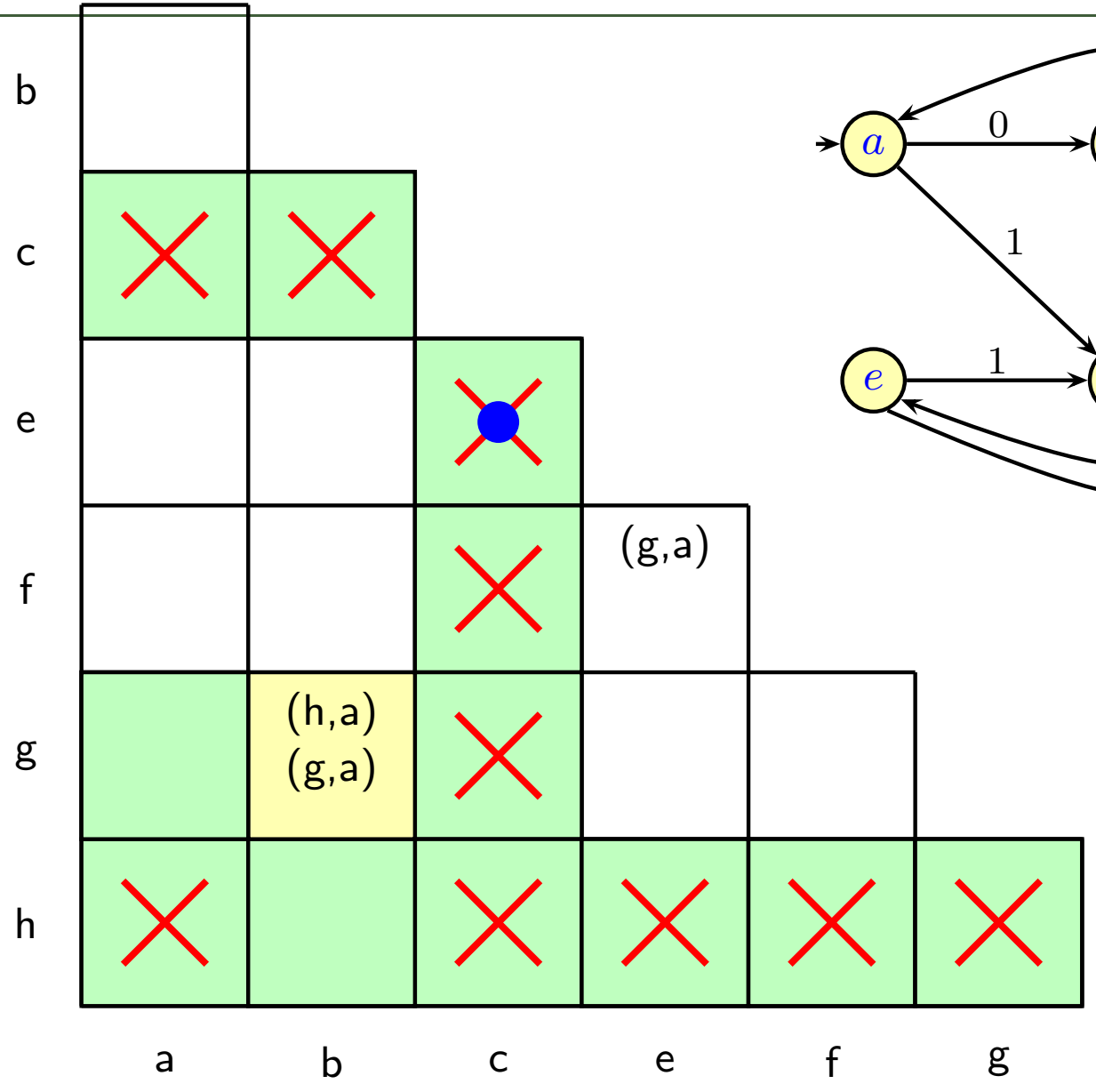
Ejemplo

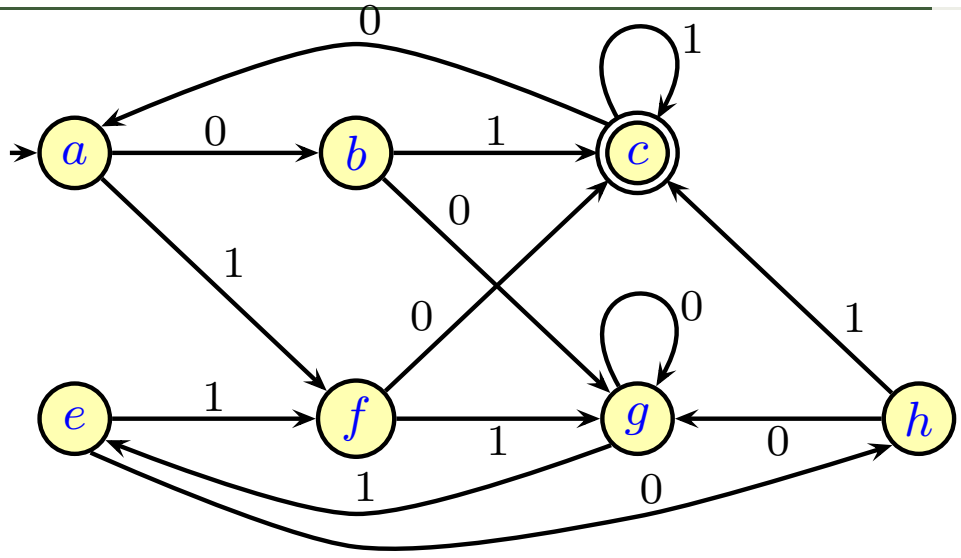




	0	1
g	g	
b	g	

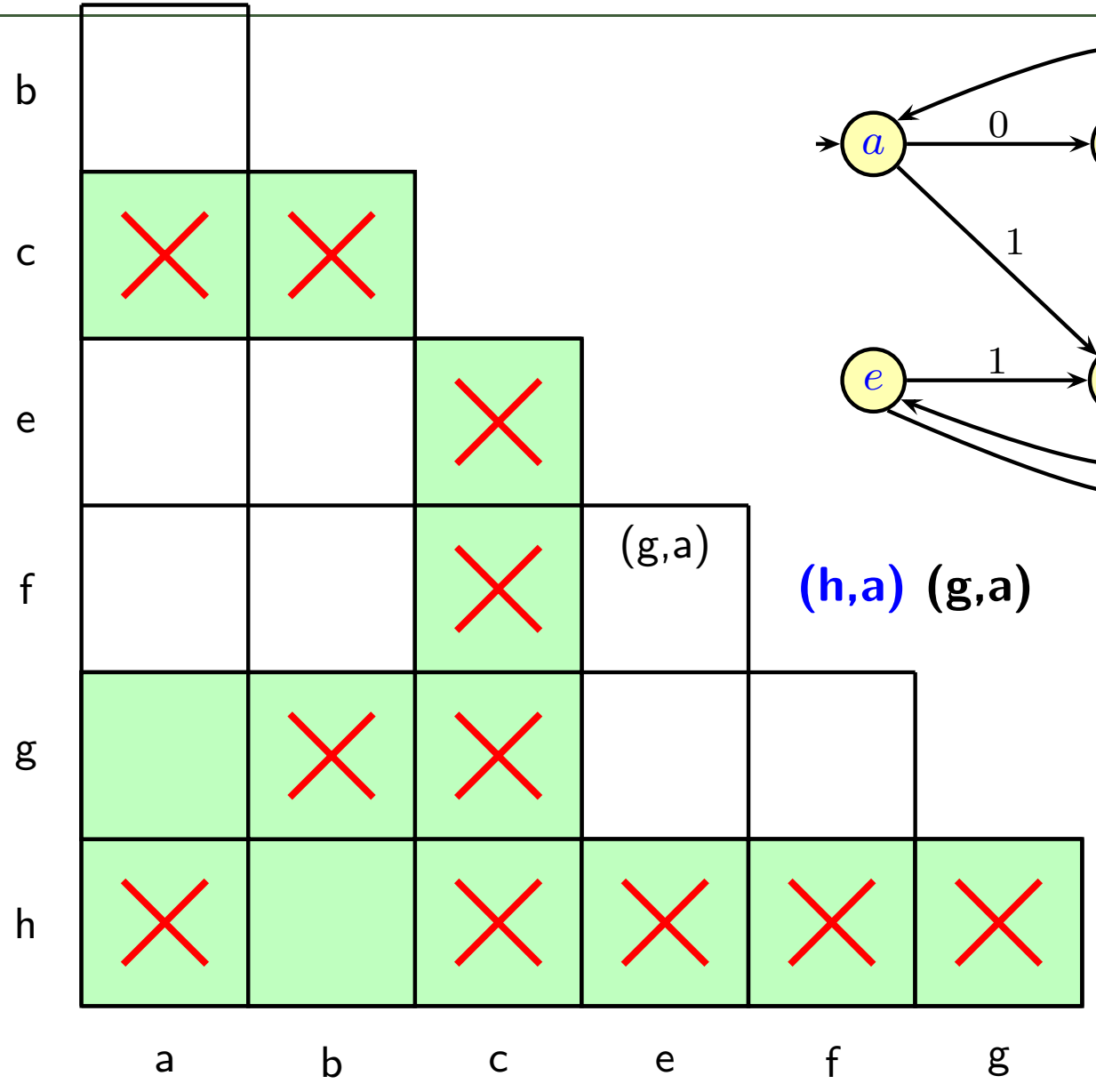
Ejemplo

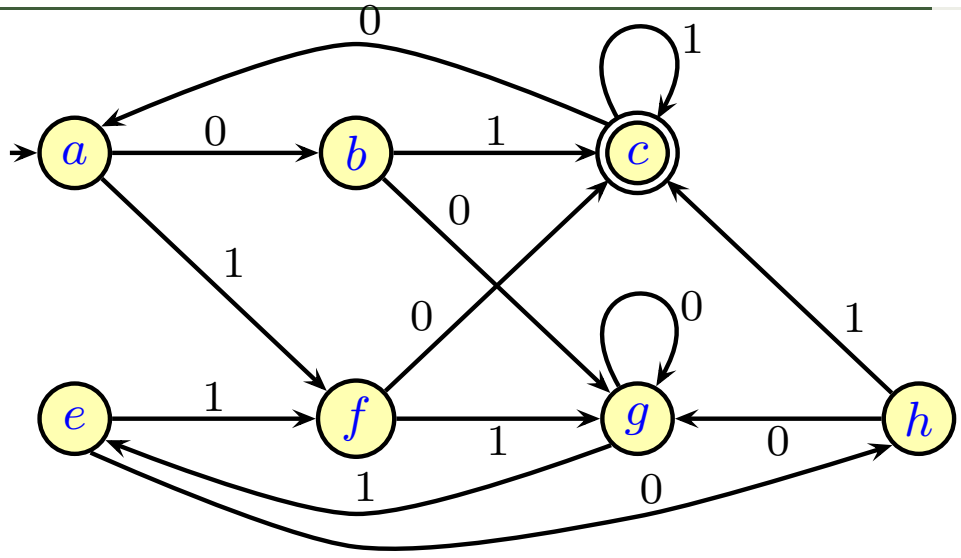




	0	1
g		e
b		c

Ejemplo

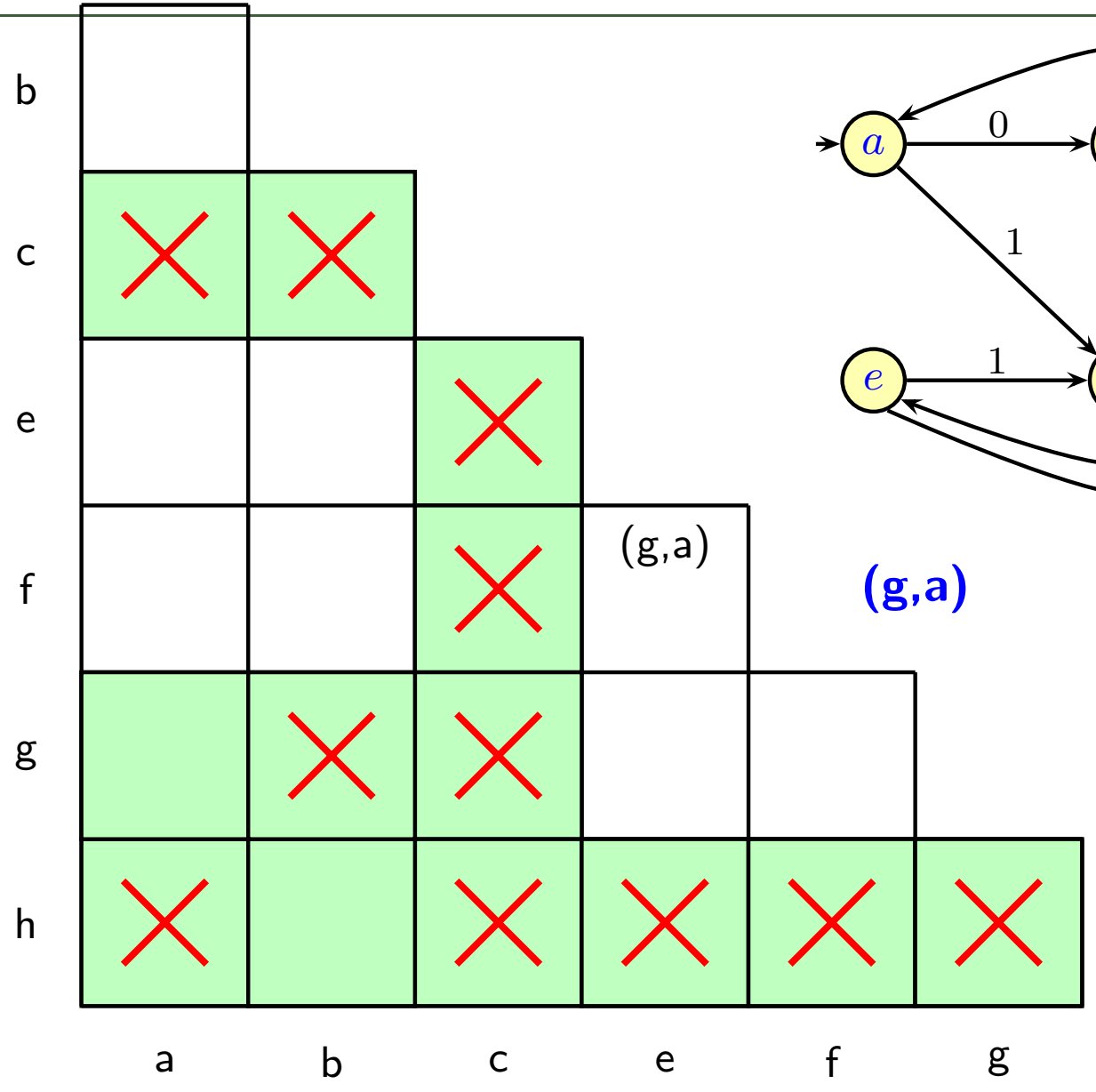


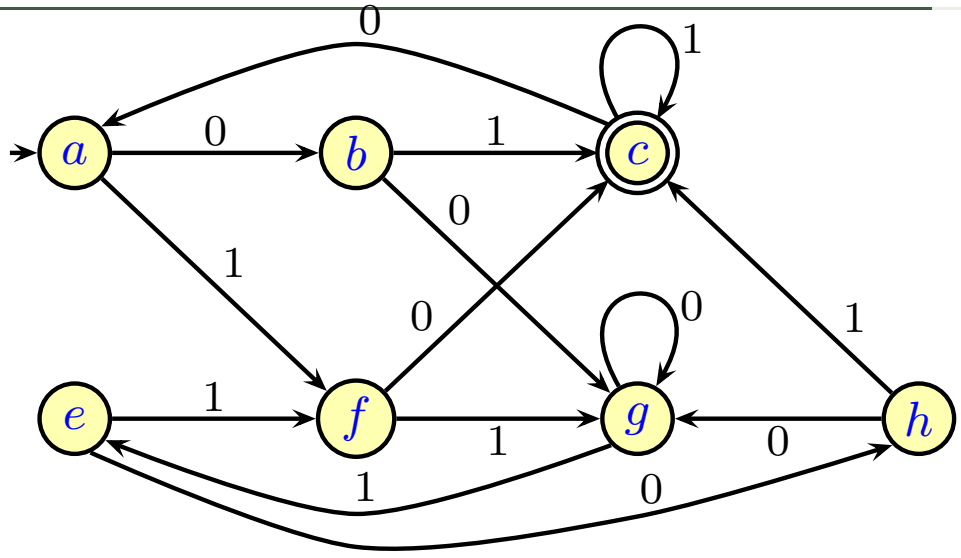


(h,a) **(g,a)**

	0	1

Ejemplo

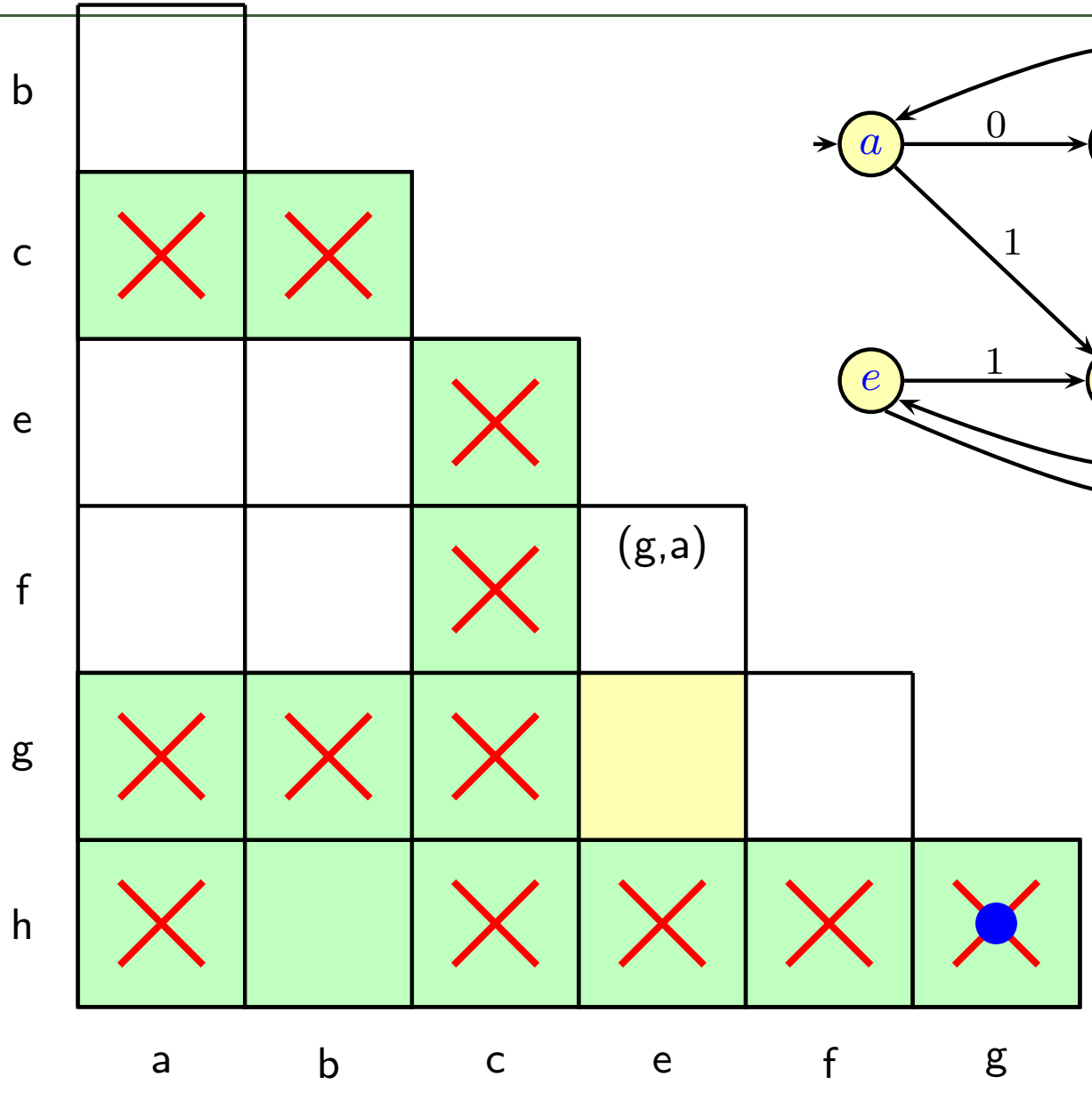


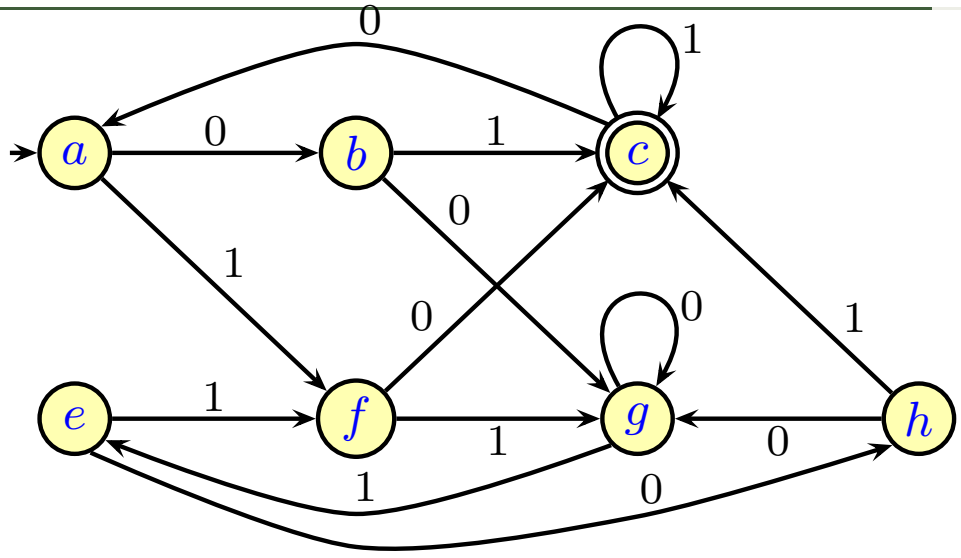


(g,a)

	0	1

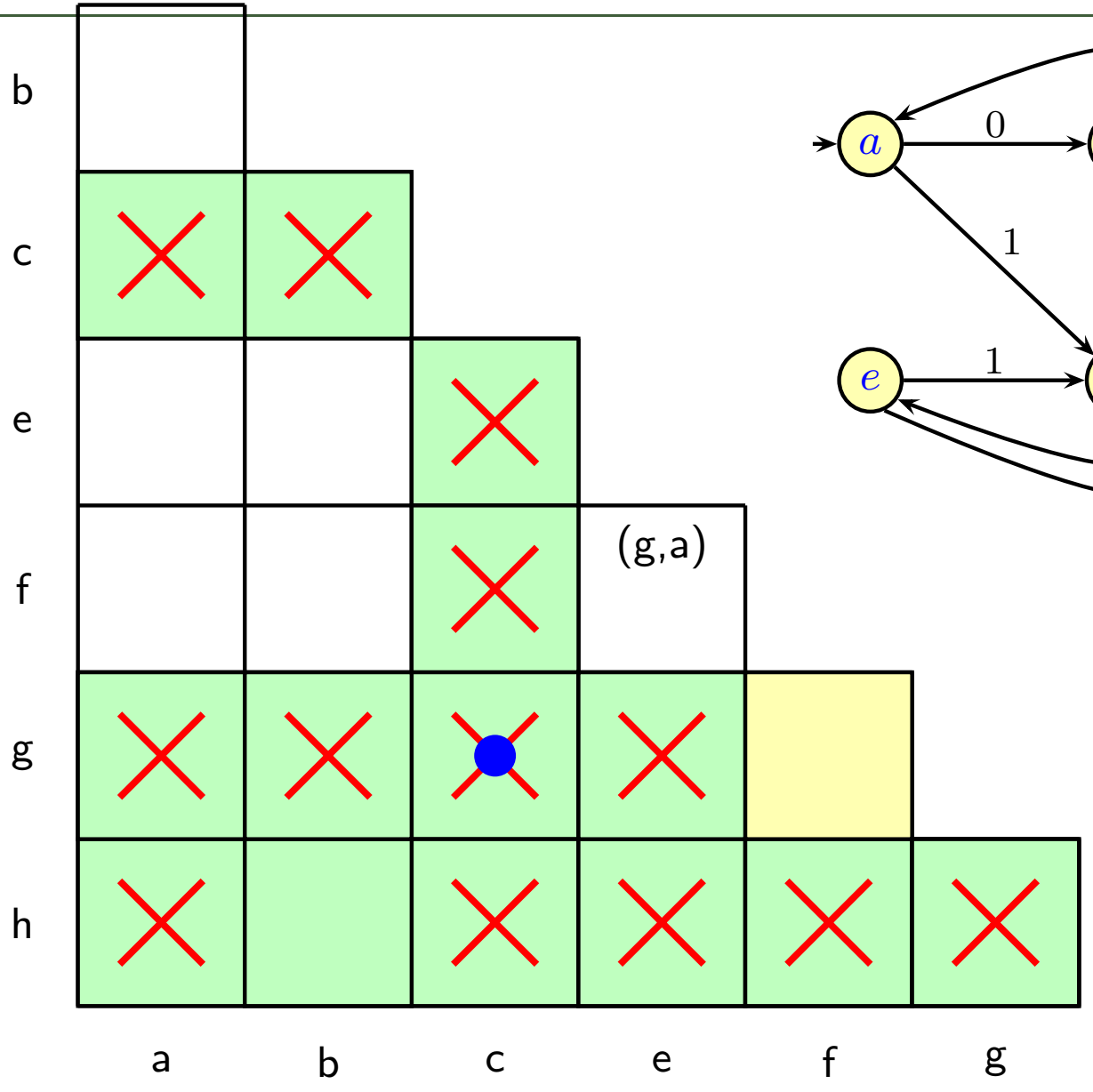
Ejemplo

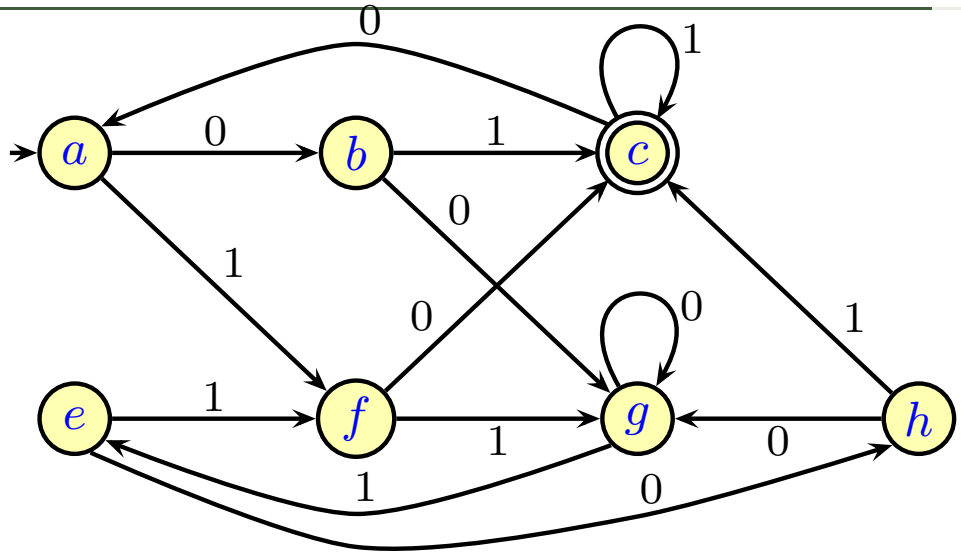




	0	1
g	g	
e	h	

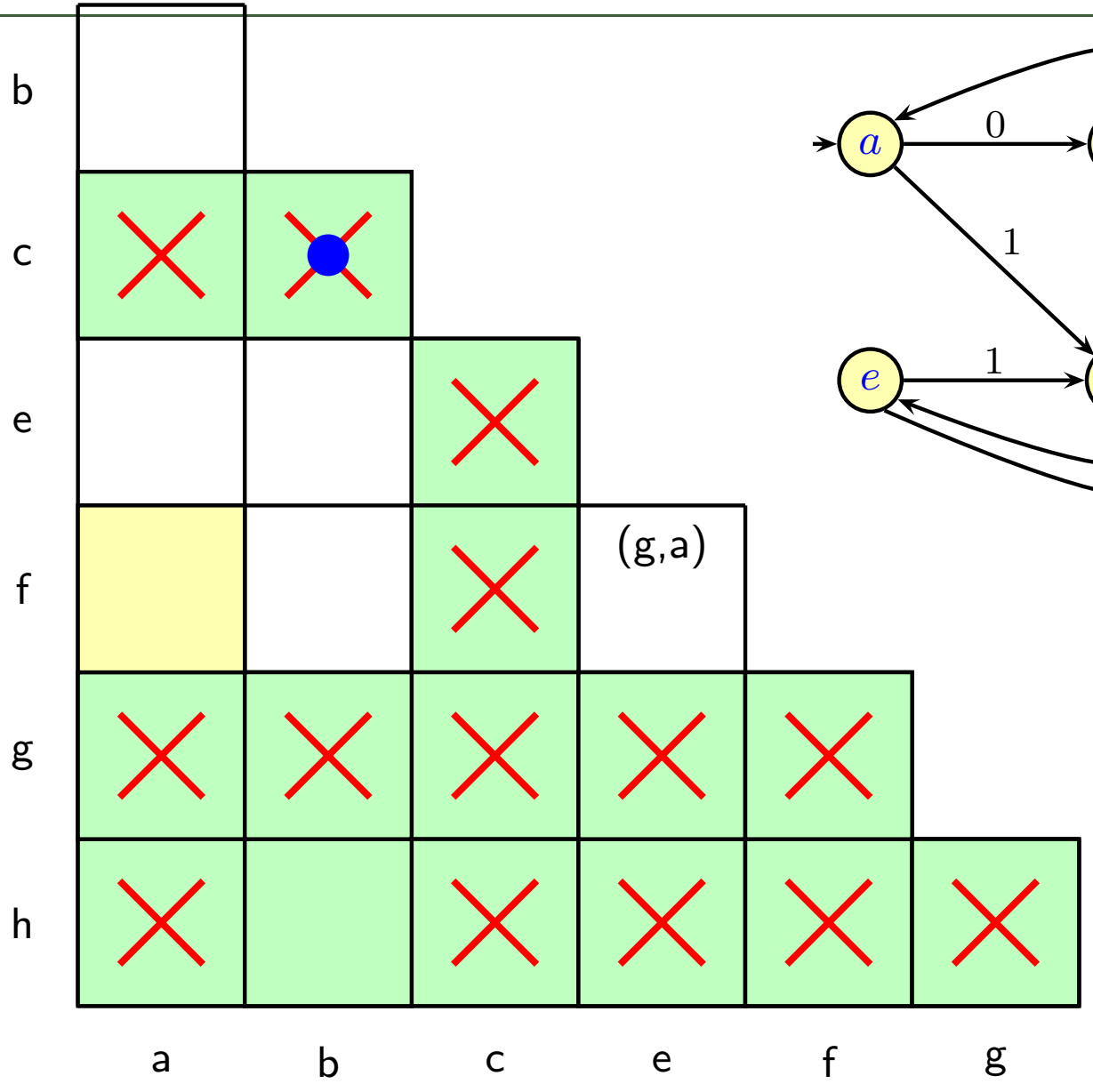
Ejemplo

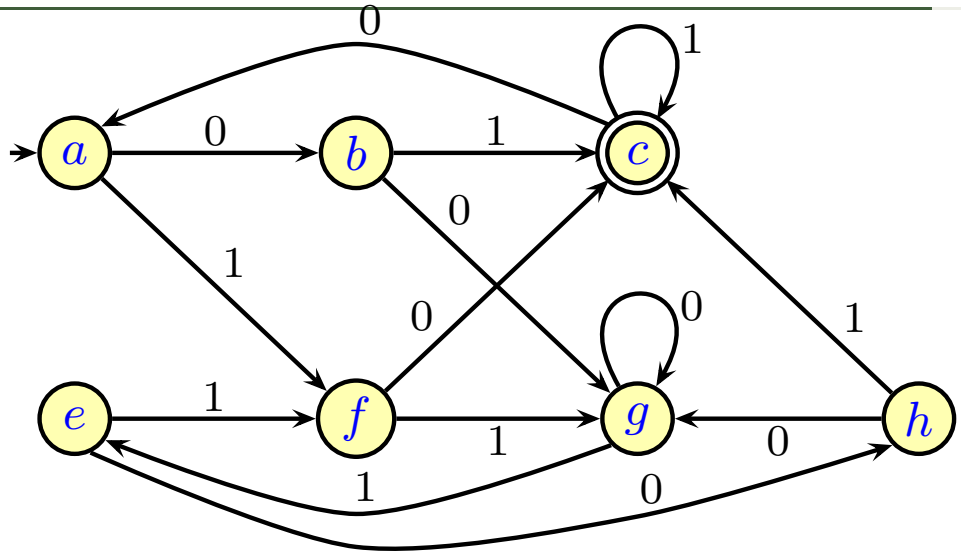




	0	1
g	g	
f	c	

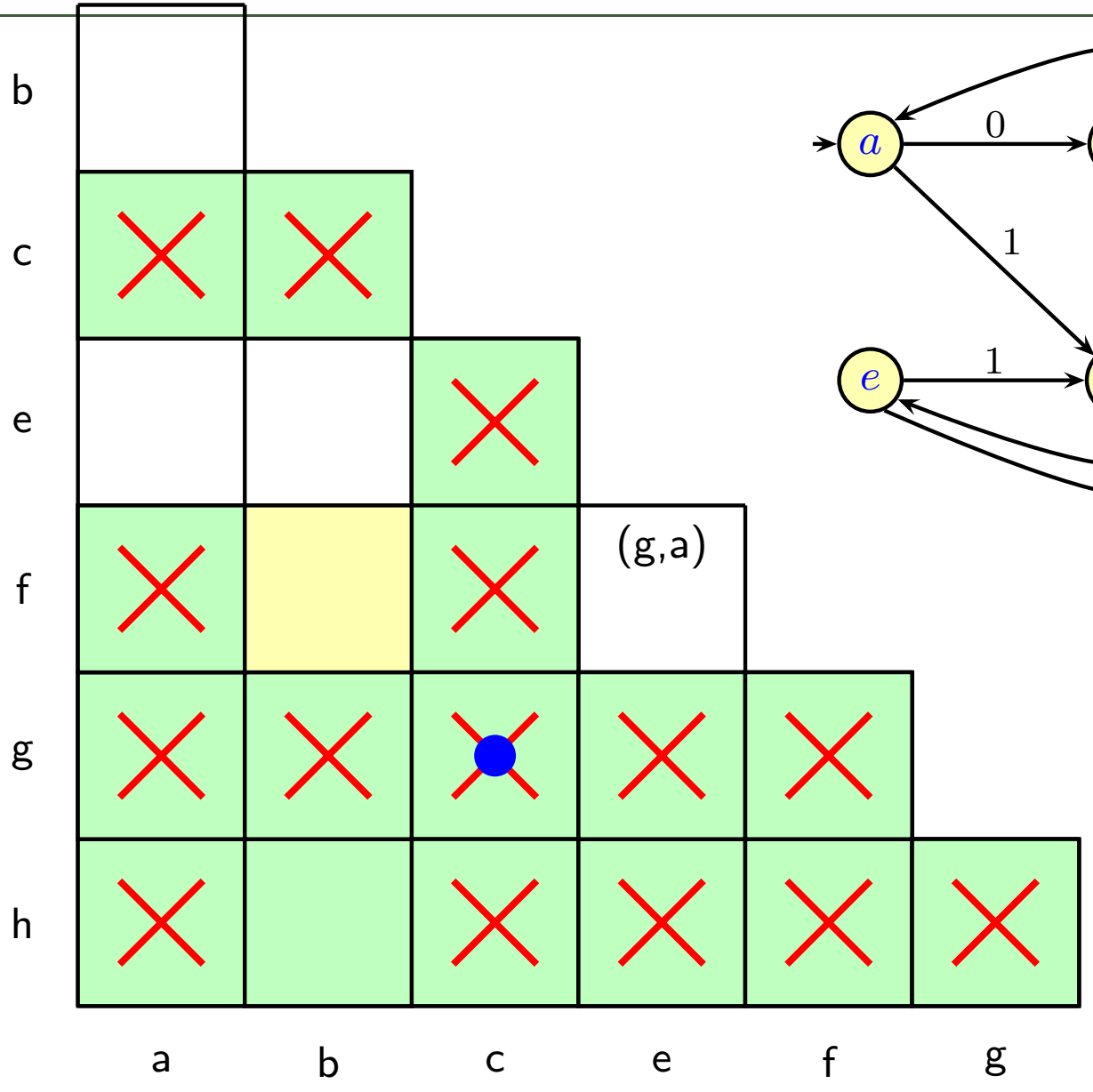
Ejemplo

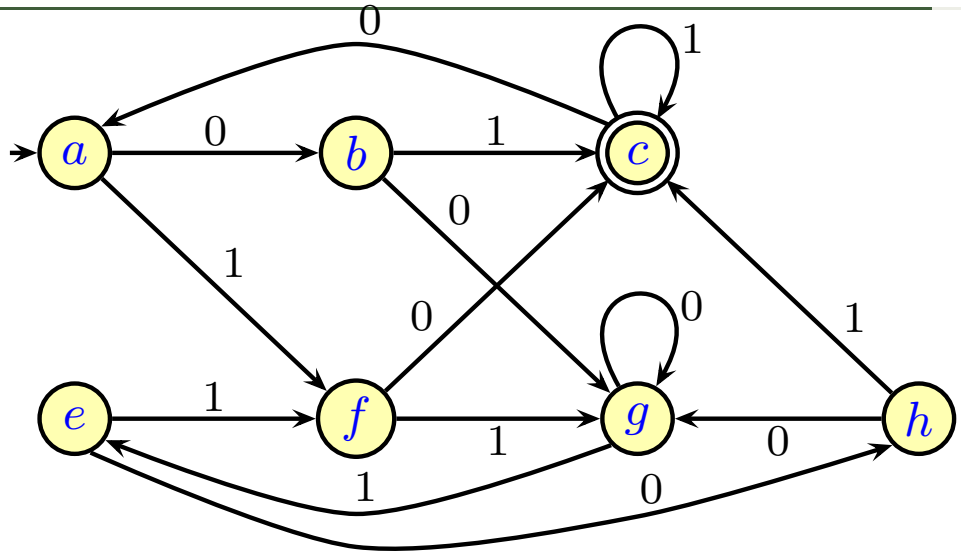




	0	1
f	c	
a	b	

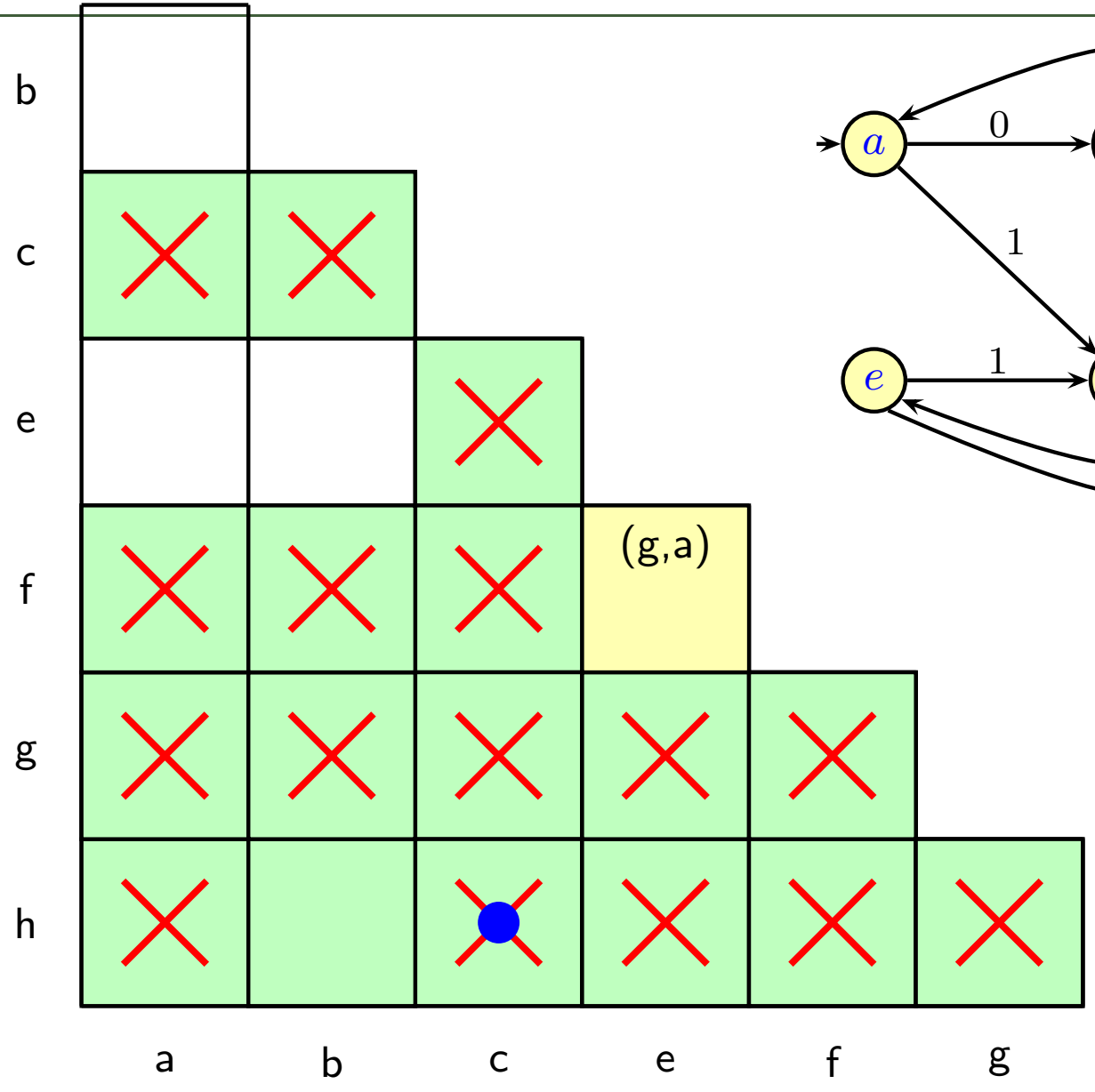
Ejemplo

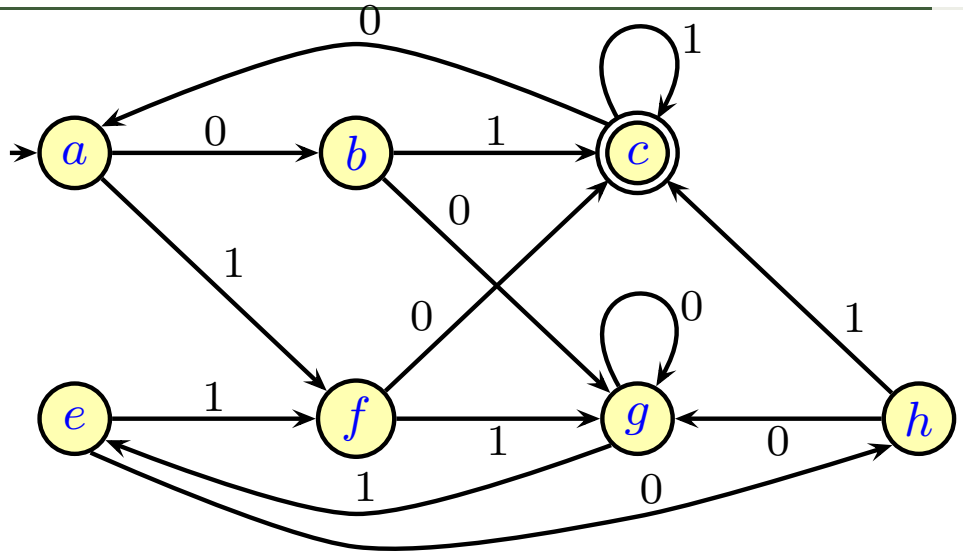




	0	1
f	c	
b	g	

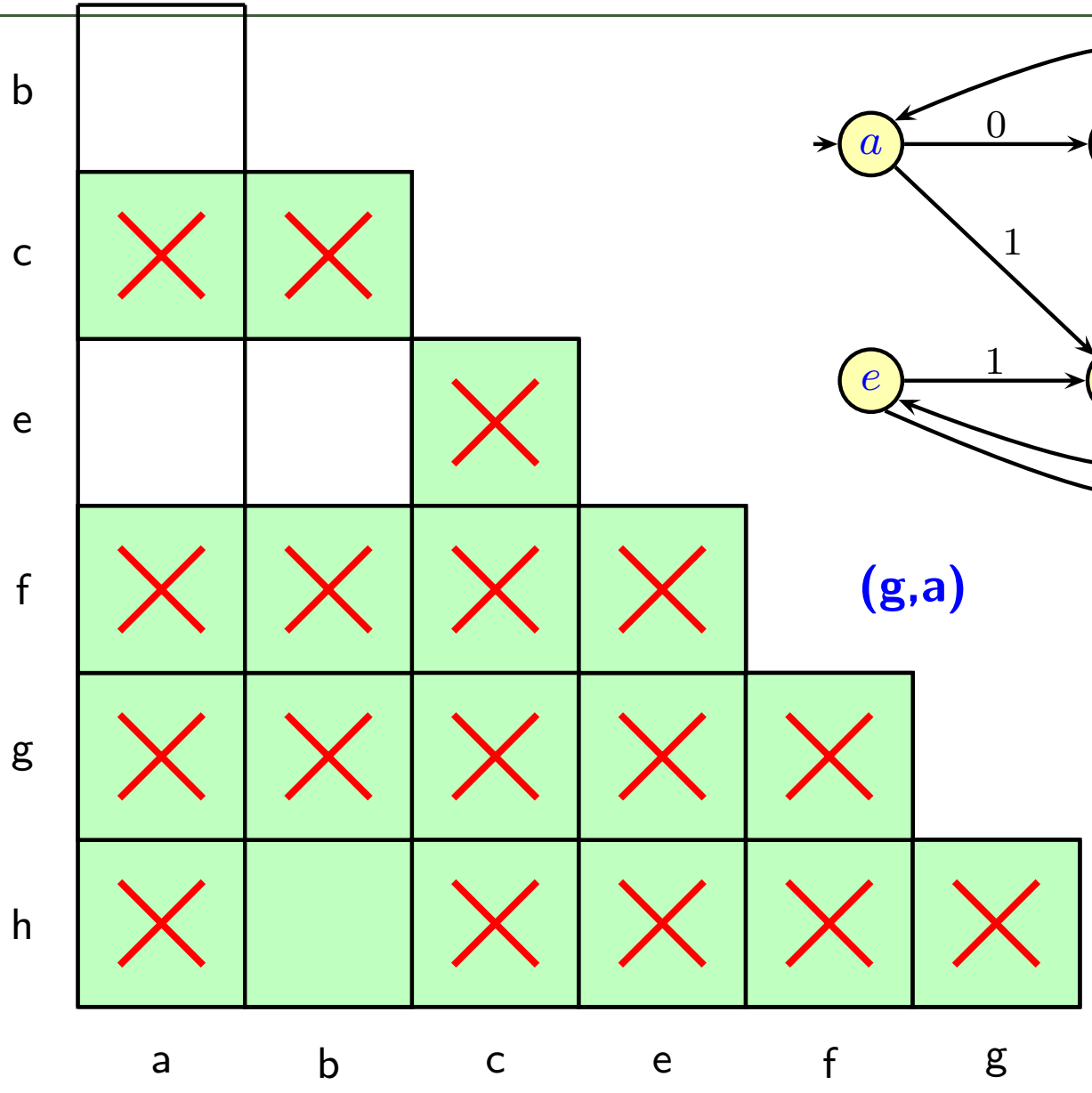
Ejemplo

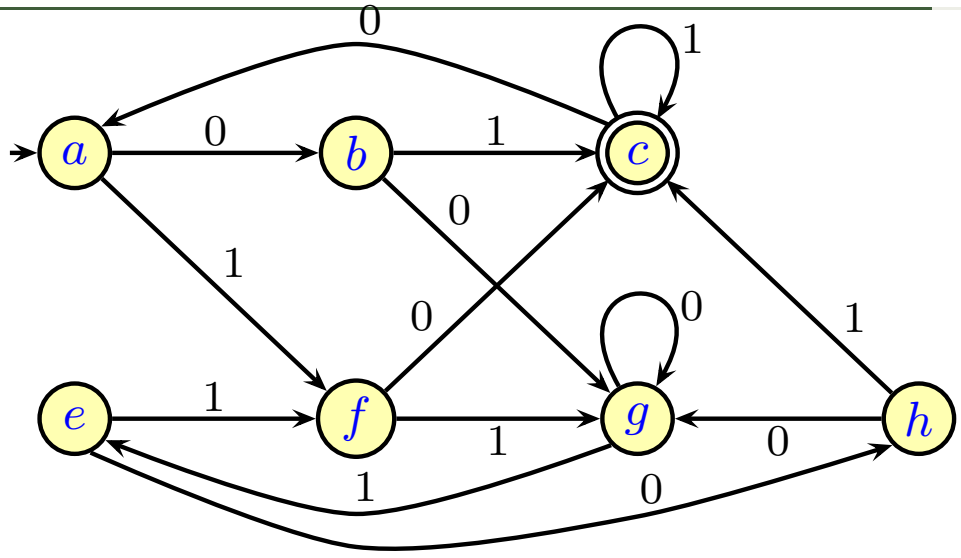




	0	1
f	c	
e	h	

Ejemplo

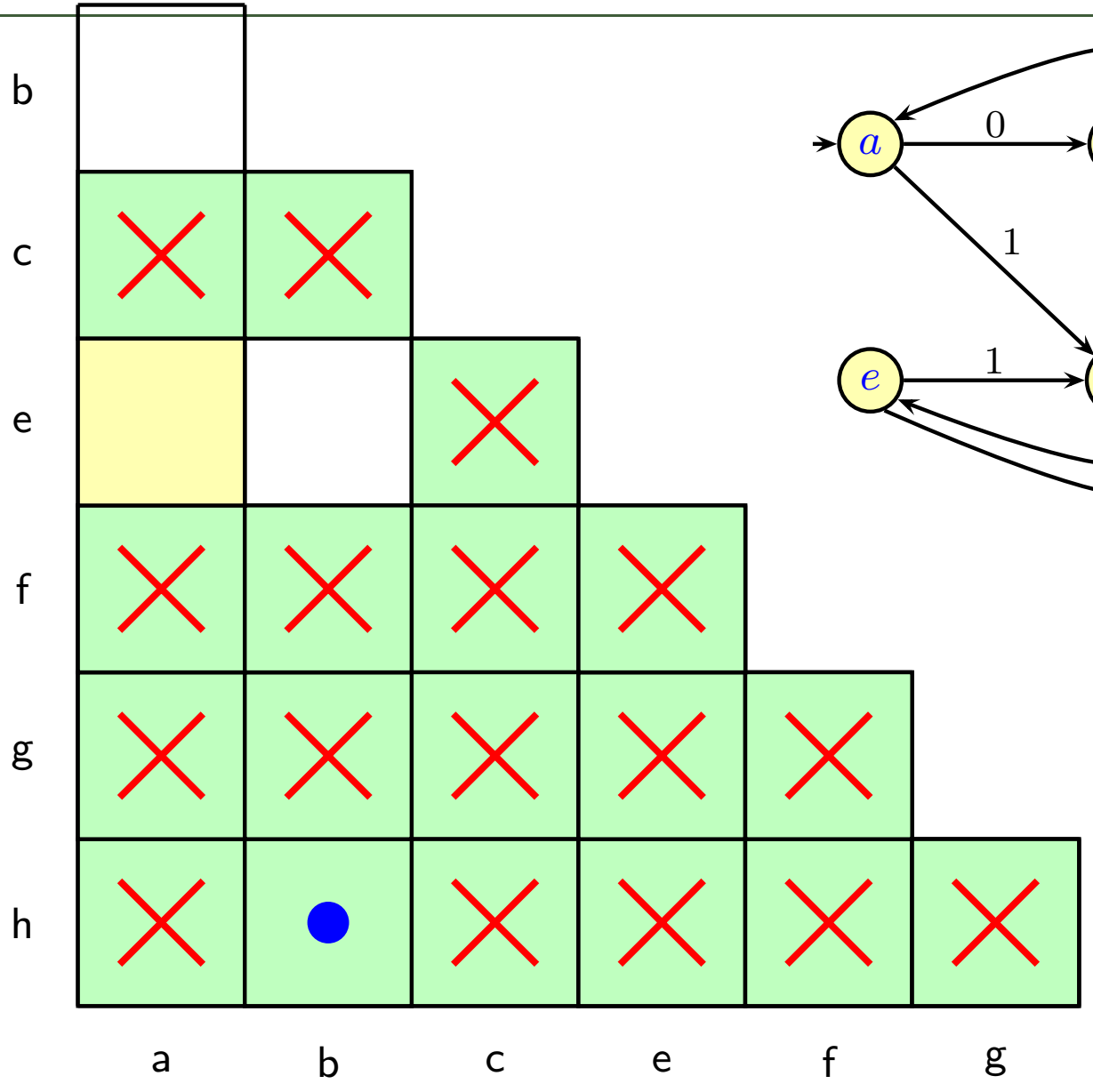


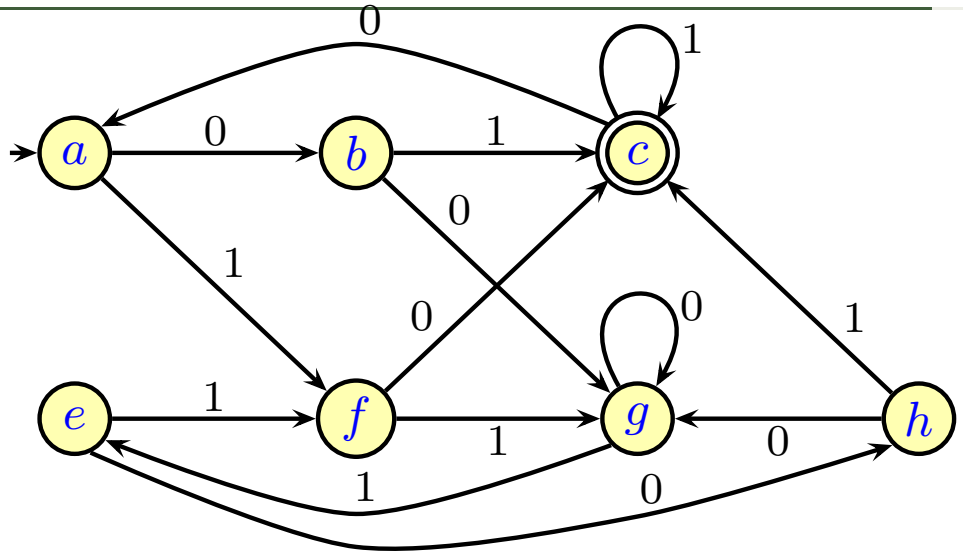


(g,a)

	0	1

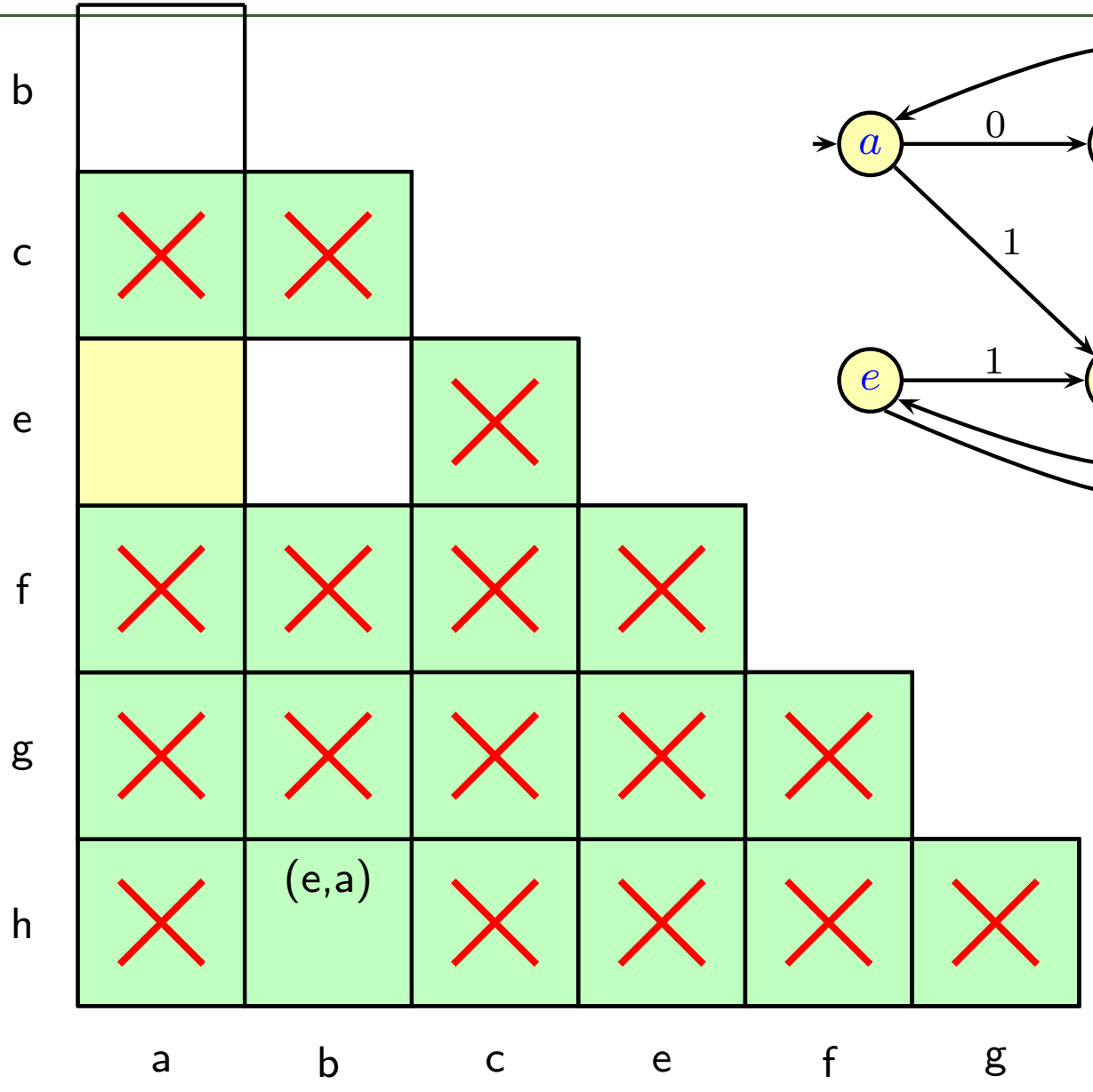
Ejemplo

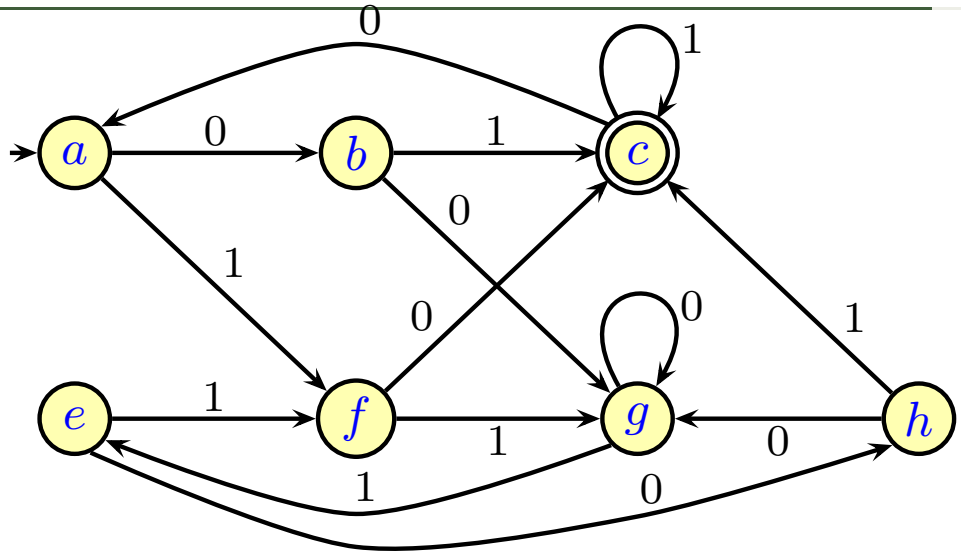




	0	1
e	h	
a	b	

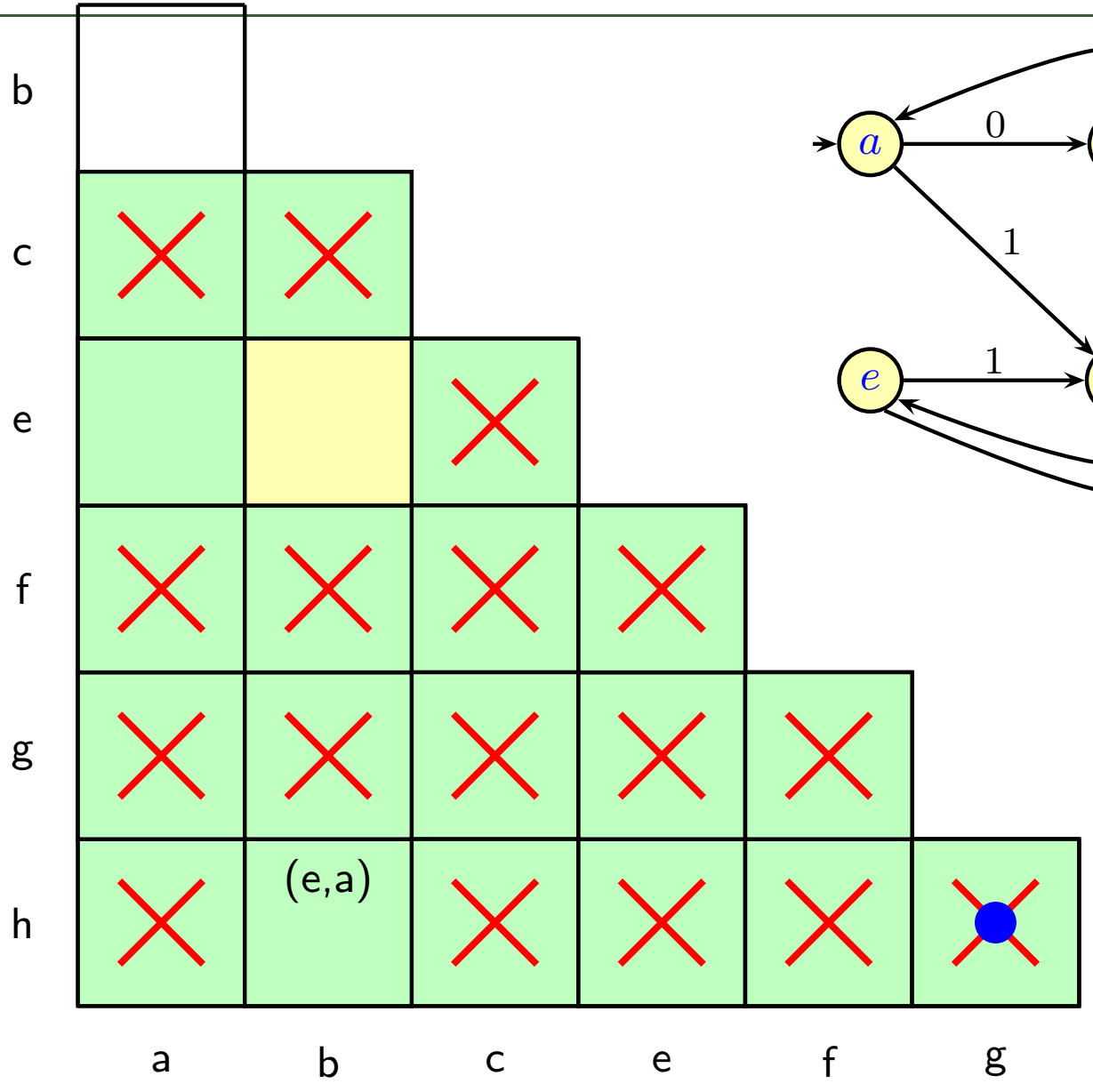
Ejemplo

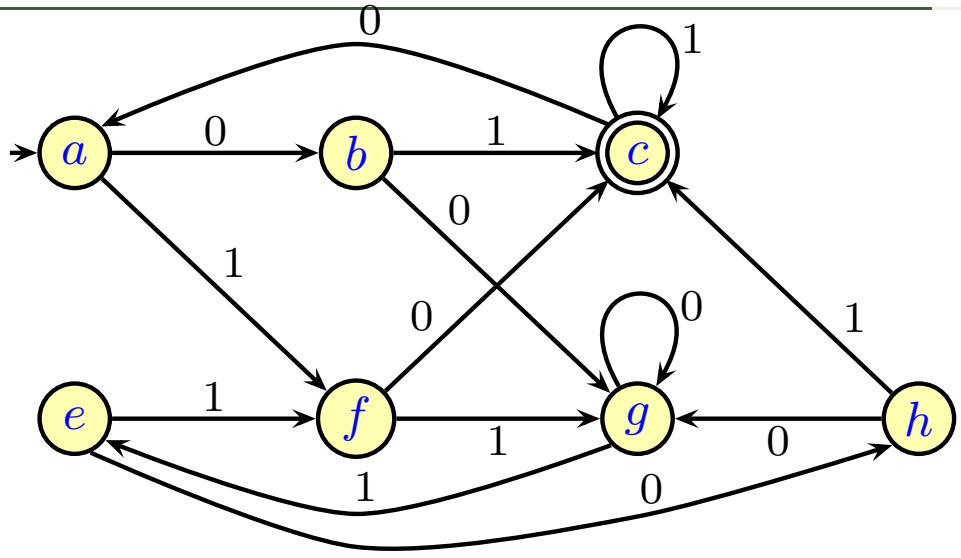




	0	1
e		f
a		f

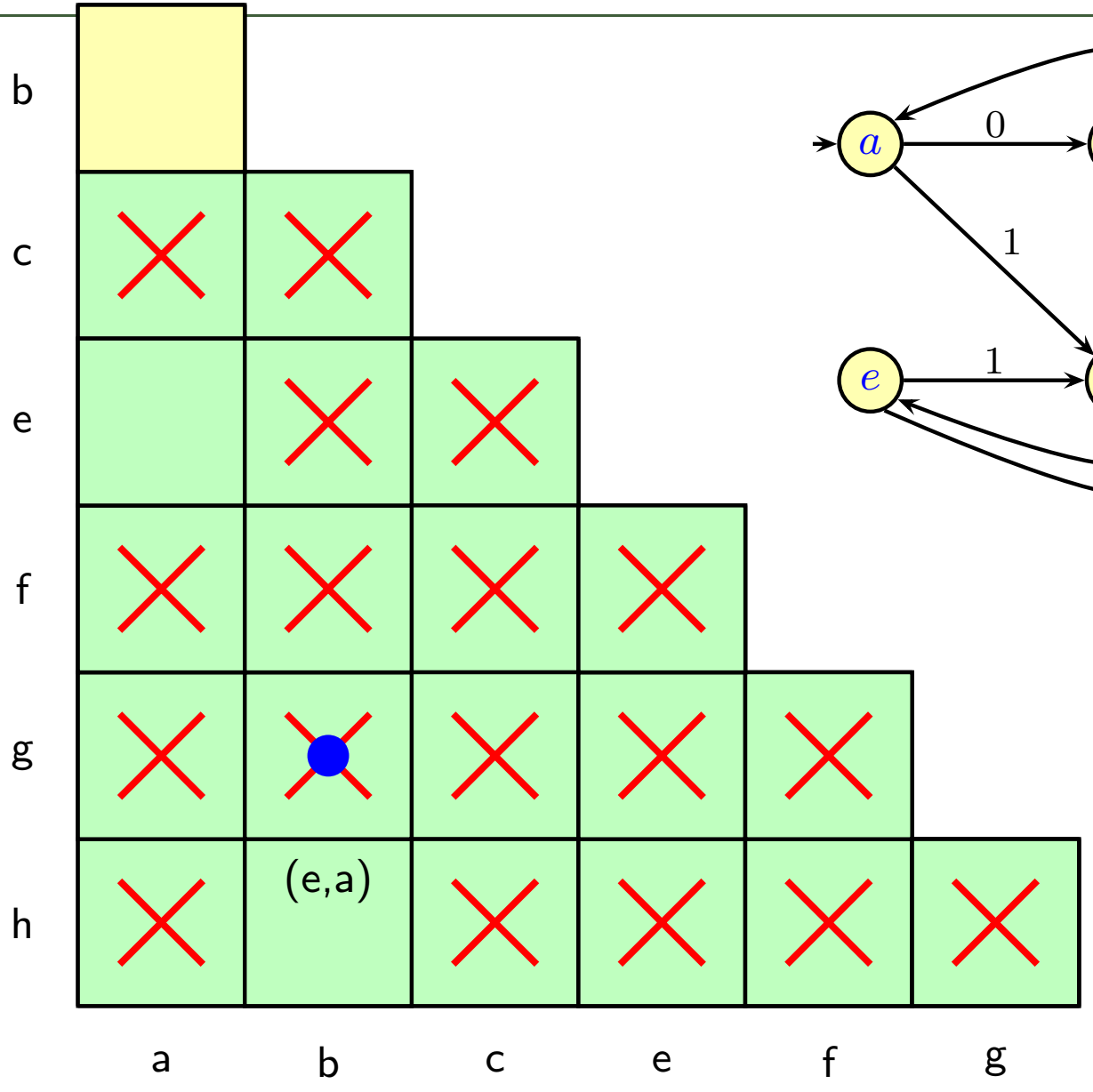
Ejemplo

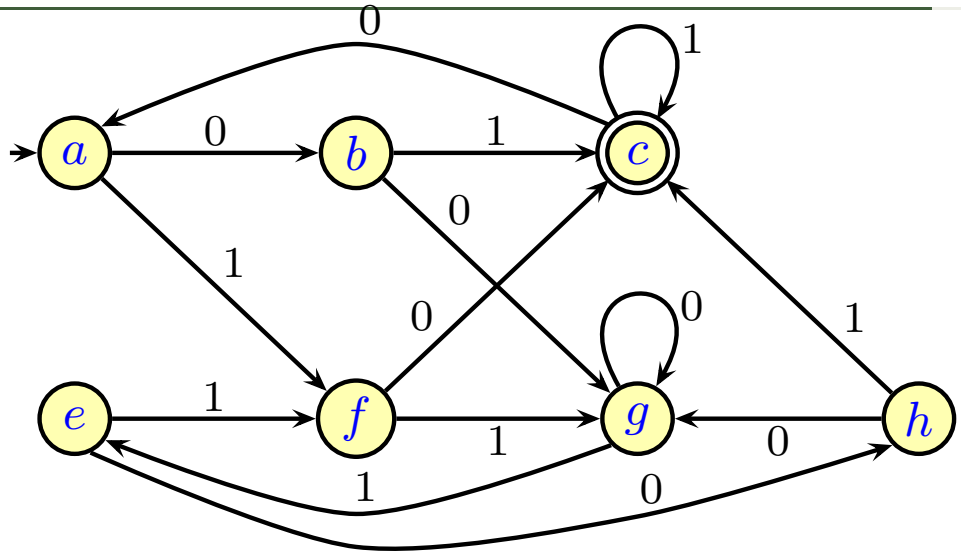




	0	1
e	h	
b	g	

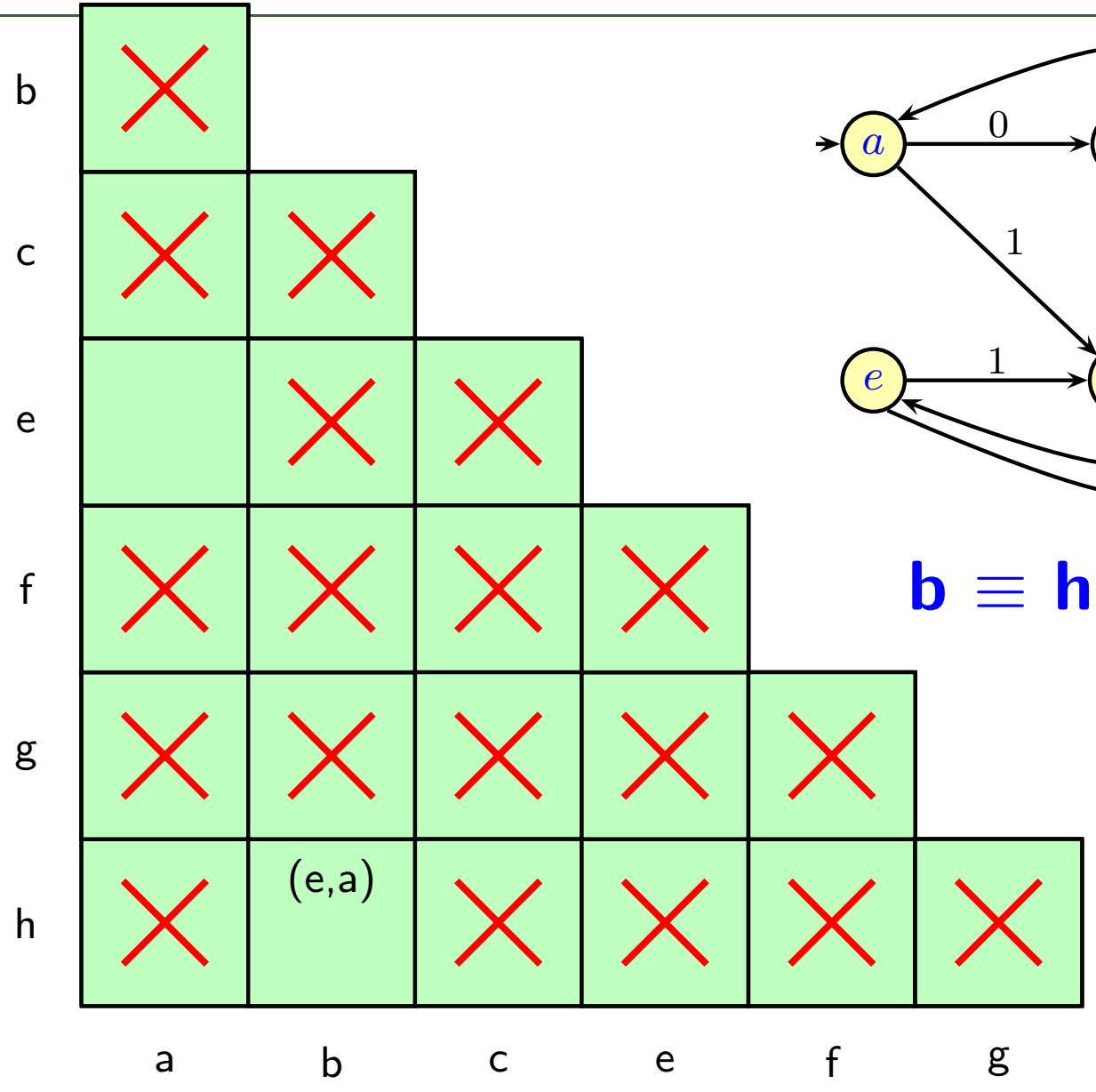
Ejemplo

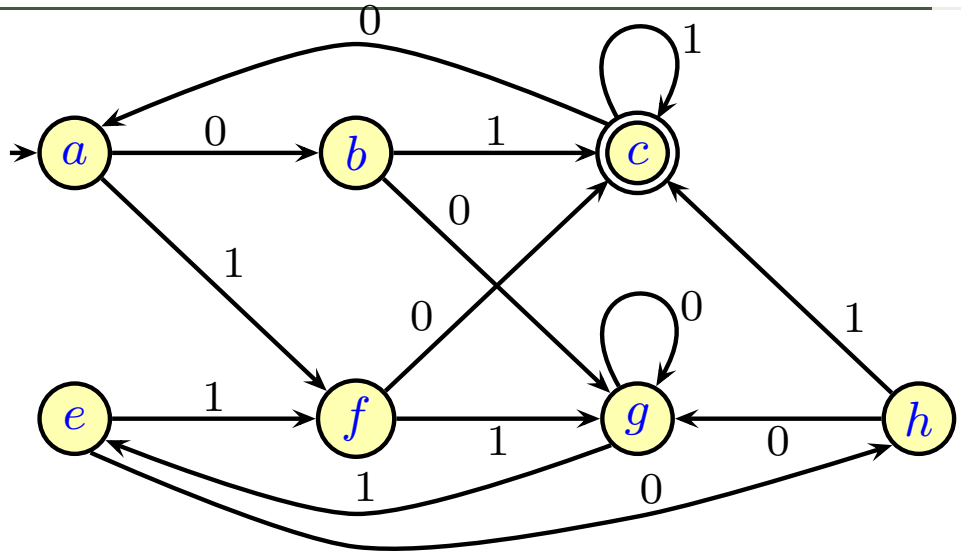




	0	1
b	g	
a	b	

Ejemplo





b ≡ h, **a ≡ e**

	0	1

Construcción del autómata minimal

El autómata minimal se construye identificando los estados indistinguibles.

Si el autómata original es $M = (Q, A, \delta, q_0, F)$,

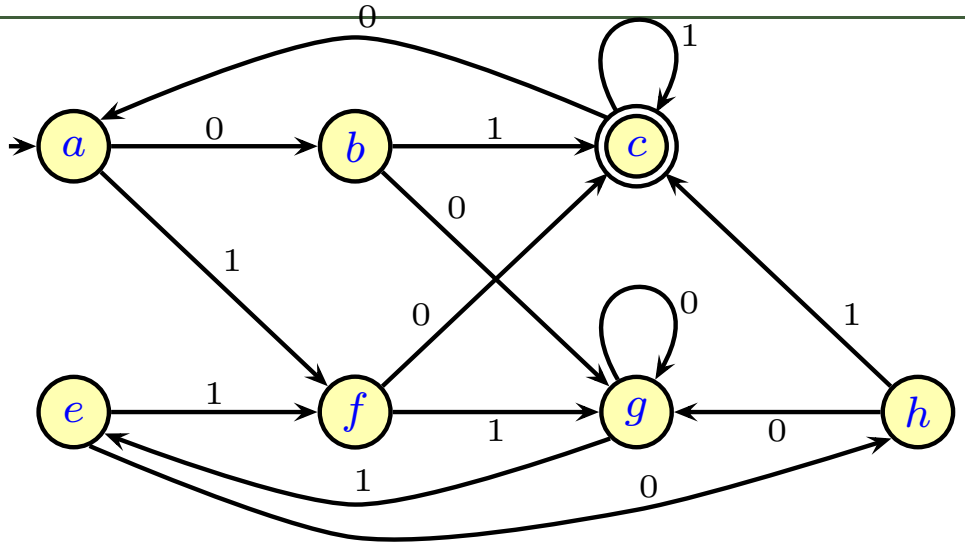
R es la relación de equivalencia de indistinguibilidad entre estados

$[q]$ la clase de equivalencia asociada al estado q,

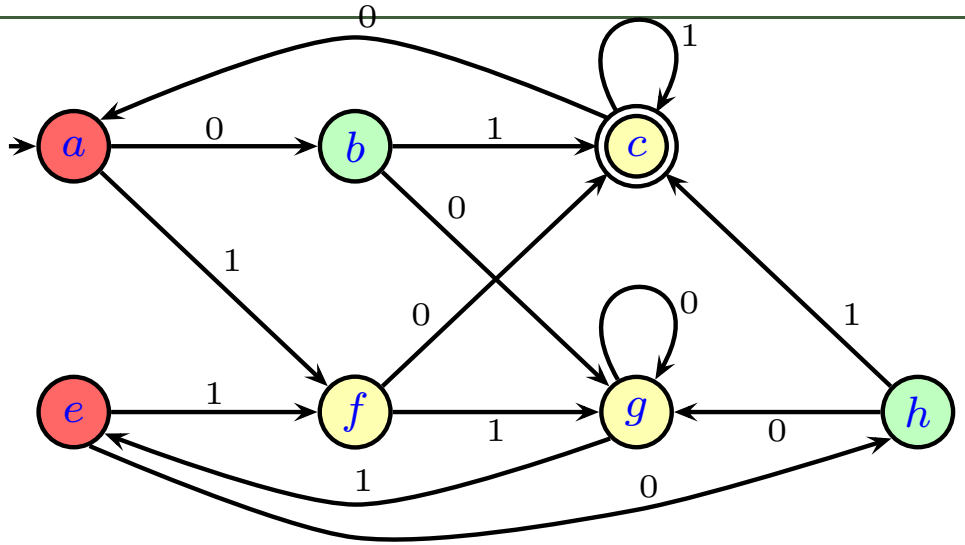
El nuevo autómata, $M_m = (Q_m, A, \delta_m, q_0^m, F_m)$ tiene los siguientes elementos,

- $Q_m = \{[q] : q \text{ es accesible desde } q_0\}$
- $F_m = \{[q] : q \in F\}$
- $\delta_m([q], a) = [\delta(q, a)]$
- $q_0^m = [q_0]$

Ejemplo: autómata minimal



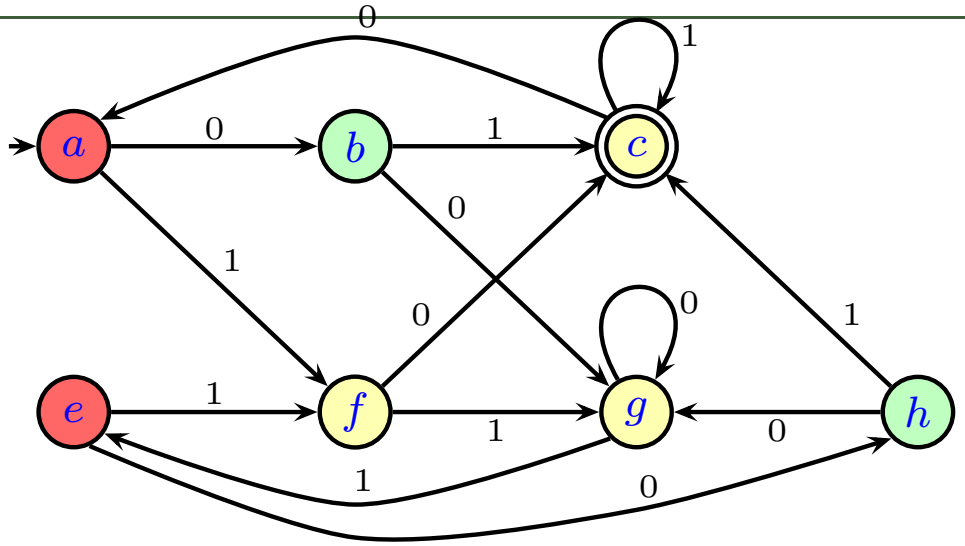
Ejemplo: autómata minimal



b ≡ h,

a ≡ e

Ejemplo: autómata minimal



b ≡ h,

a ≡ e

